

VELS





INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

EXPLAINABLE LOAN PREDICTION USING FINANCIAL BEHAVIOUR CONSISTENCY ENGINE (FBCE)

S PREM KUMAR - 23619129
A GUHAN - 23619148

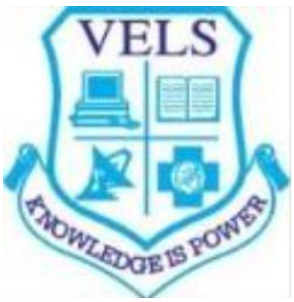
Under the guidance of
Dr. N. Meenakshi M.Tech, Ph.D

FINAL PROJECT REPORT

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY CSE –

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)
CHENNAI – 600 117
October 2026

BONAFIDE CERTIFICATE

Certified that this project report "Explainable Loan Prediction using Financial Behaviour Consistency Engine (FBCE)" is the bonafide work of **S PREM KUMAR (23619129)** and **A GUHAN (23619148)**, who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya M.E., Ph.D.
Associate Professor & Head
Department of CSE
School of Engineering
VISTAS
Chennai

SIGNATURE OF THE SUPERVISOR
Dr. N. Meenakshi M.Tech, Ph.D
Department of CSE
School of Engineering
VISTAS
Chennai

Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of our **FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH**, and it is with great pleasure that we take this opportunity to thank him.

We are grateful to our **HEAD OF THE DEPARTMENT**, Dr. K. Ulaga Priya, Department of Computer Science and Engineering, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. We extend our warmest thanks to our **INTERNAL GUIDE, Dr. N. Meenakshi M.Tech, Ph.D, Department of Computer Science and Engineering**, who has patiently guided us and provided invaluable advice and help throughout the work. We are thankful to our **Project Coordinator, Ms. S.J. Kavitha**.

We would like to express our greatest gratitude to our staff members, Librarian, and non-teaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not the least, we wish to thank our parents and friends for their individual support and interest, who inspired and encouraged us to go our own way; without them we would be unable to complete this project.

S PREM KUMAR - 23619129
A GUHAN - 23619148

TABLE OF CONTENTS

ABSTRACT ..... **v**
LIST OF FIGURES ..... **vi**
LIST OF TABLES ..... **vii**
LIST OF ABBREVIATIONS ..... **ix**
CHAPTER 1. INTRODUCTION **1**
1.1 OVERVIEW OF THE PROJECT 1
1.2 PROBLEM STATEMENT 3
1.3 MOTIVATION AND SIGNIFICANCE 3
1.4 AIM AND OBJECTIVES 4
1.5 SCOPE OF THE PROJECT 4
1.6 CONTRIBUTIONS 6
1.7 ORGANISATION OF THE REPORT 6
CHAPTER 2. LITERATURE REVIEW **8**
2.1 LITERATURE OVERVIEW 8
2.2 LITERATURE SURVEY 8
2.3 SUMMARY OF RELATED WORK 11
2.4 RESEARCH GAP 12
2.5 THEORETICAL BACKGROUND 12
CHAPTER 3. SYSTEM DESIGN **17**
3.1 EXISTING SYSTEM 17
3.2 PROPOSED SYSTEM 17
3.3 SYSTEM ARCHITECTURE 18
3.4 REQUIREMENTS ANALYSIS 20
3.5 HARDWARE REQUIREMENTS 22
3.6 SOFTWARE REQUIREMENTS 22
3.7 PROJECT FOLDER STRUCTURE 23
3.8 USE-CASE, DATA-FLOW AND INTERACTION DESIGN 24
3.9 MODULE DEPENDENCIES 27
CHAPTER 4. IMPLEMENTATION **29**
4.1 METHODOLOGY 29
4.2 FINANCIAL BEHAVIOUR CONSISTENCY ENGINE 33
4.3 EXPLAINABILITY LAYER AS IMPLEMENTED 41
4.4 IMPLEMENTATION OF THE MODULES 44
4.5 TESTING, RESULTS AND DISCUSSION 51
4.6 DISCUSSION 62
4.7 LIMITATIONS 64
4.8 RESPONSIBLE USE, FAIRNESS AND ETHICAL CONSIDERATIONS 65

CHAPTER 5. CONCLUSION AND FUTURE WORK 68
5.1 CONCLUSION 68
5.2 FUTURE WORK 69
5.3 SOURCE CODE 70
5.4 RESULTS 81
5.5 USER INTERFACE OUTPUTS 82
REFERENCES 88

ABSTRACT

Machine-learning models can screen loan applications quickly, but their decisions are hard to justify, and they rarely test whether the financial details of an application agree with one another. This project presents the Financial Behaviour Consistency Engine (FBCE), a decision-support prototype that places a benchmarked loan-approval classifier next to a rule-based consistency engine and an explanation layer based on SHapley Additive exPlanations (SHAP). A Streamlit dashboard shows the three outputs side by side without merging them. Four classifiers (logistic regression, random forest, XGBoost and LightGBM) are trained under one scikit-learn preprocessing pipeline, and the best model by F1-score and then accuracy is saved for the dashboard. In the project's earlier five-fold cross-validation on the public Loan Prediction dataset of 614 applications, logistic regression obtained the highest accuracy (80.30%) and F1-score (87.14%) with an area under the ROC curve of 0.751, comparable to the ensemble models; this evaluation is reported separately from the single 80/20 split implemented in the delivered training script. Independently of the classifier, FBCE scores income consistency, debt consistency, employment stability, credit consistency and repayment capacity, averages them into a 0–100 score, assigns a consistency level and raises contradiction warnings. The FBCE module was verified on thirteen executed test cases and ten executed validation checks, and the training pipeline was executed on a clearly labelled synthetic dataset to confirm that the software runs. That synthetic-data model also reproduces the probability and SHAP contributions of the case study (50.09%) and of the dashboard demonstration (76.5%), so these examples illustrate the prototype and are not results on real lending data. In the case study the classifier approved an applicant with a probability of only 50.09% while FBCE rated the profile at 90.89 and the SHAP view showed a small negative contribution for male gender; because gender plays no part in the rule that generates the synthetic target, this contribution reflects the fit to a random sample and not lending bias, but it shows why protected attributes should be audited. The report also documents the limitations of the prototype, including mixed units in the loan-to-income ratio, a score that cannot exceed 93 for dashboard inputs and the use of heuristic, unvalidated thresholds. The FBCE score is a project-defined consistency indicator and not a banking credit score.

Keywords: explainable artificial intelligence, loan approval prediction, SHAP, credit risk assessment, financial consistency scoring, logistic regression, decision support systems.

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1	Architecture of the FBCE prototype	19
Figure 2	Project folder structure (from README.md)	24
Figure 3	Use-case diagram of the FBCE prototype	25
Figure 4	Level-1 data-flow diagram	26
Figure 5	Sequence of interactions for one prediction request	27
Figure 6	Dependencies between the project files	28
Figure 7	Class balance and approval rate by credit history (earlier exploratory analysis)	30
Figure 8	Model-development workflow implemented in train_model.py	32
Figure 9	Derivation of the FBCE component scores, overall score and warnings (from fbce.py)	35
Figure 10	Behaviour of the scoring functions in fbce.py (computed by evaluating the code)	35
Figure 11	FBCE score as a function of total income and loan amount (computed by evaluating fbce.py; credit history 1, self-employed “No”, 360 months)	38
Figure 12	Ten largest linear contributions for the demonstration input (synthetic-data model; closed form)	43
Figure 13	Accuracy, F1-score and specificity of the four classifiers (plotted from Table 39)	55
Figure 14	FBCE component scores for the case-study applicant (computed by running fbce.py)	56
Figure 15	Signed SHAP contributions for the case-study applicant (values reproduced with the synthetic-data model)	57
Figure 16	Dashboard input form: Applicant Financial Details (earlier build)	82
Figure 17	Prediction result and key financial indicators (earlier build)	83
Figure 18	Financial health advisory and Financial Behaviour Analysis Profile (earlier build)	83
Figure 19	FBCE consistency components and financial consistency checks (earlier build)	84
Figure 20	SHAP explainability table (feature impact; earlier build)	84
Figure 21	Top SHAP feature contributions for the demonstration input (earlier build)	85
Figure 22	Positive and negative contributors with the academic-demonstration disclaimer (earlier build)	85

LIST OF TABLES

Table No.	Title	Page No.
Table 1	Intended users of the prototype	5
Table 2	Boundaries of the system	5
Table 3	Summary of related work	11
Table 4	Qualitative comparison of loan-decision approaches (assessment by the authors, not a measured result)	11
Table 5	Research gaps and the extent to which the project addresses them	12
Table 6	Existing practice compared with the proposed system	17
Table 7	Component interfaces in the delivered implementation	20
Table 8	Data items	20
Table 9	Functional requirements	21
Table 10	Non-functional requirements	21
Table 11	Input validation and error handling in the delivered code	22
Table 12	Hardware requirements used for development	22
Table 13	Software tools and libraries	22
Table 14	Role of each file and folder	24
Table 15	Dependency matrix (row depends on column)	28
Table 16	End-to-end workflow and the verification status of each stage	29
Table 17	Attributes of the Loan Prediction dataset (as recorded in the earlier evaluation phase)	30
Table 18	Classifier configurations in train_model.py	32
Table 19	Model inputs produced by the column transformer	33
Table 20	Consistency levels assigned by fbce.py	34
Table 21	Income and debt consistency against the loan-to-income ratio	36
Table 22	Repayment capacity against the EMI-to-income ratio	36
Table 23	Effect of the loan term (income 20,000; loan 500,000; rate 10%)	37
Table 24	Effect of the annual interest rate (income 20,000; loan 500,000; term 240 months)	37
Table 25	Score for a salaried-type applicant as the loan amount grows (income 5000; term 360; credit history 1; self-employed “No”)	37
Table 26	Inputs and outputs of calculate_fbce	39
Table 27	Symbols used in the FBCE equations	39
Table 28	Constants of fbce.py (all project-defined)	39
Table 29	Illustrative sensitivity of the score to the unit of the loan amount (computed by evaluating fbce.py; not experimental data)	40
Table 30	Range of the FBCE score (computed by evaluating fbce.py)	41
Table 31	Ten largest linear contributions for the demonstration input (synthetic-data logistic regression; closed form, log-odds)	42

Table No.	Title	Page No.
Table 32	Functions in fbce.py	46
Table 33	Functions in train_model.py	46
Table 34	Structure of app.py	47
Table 35	Inputs, outputs, side effects and errors of the modules	51
Table 36	Evidence used in this chapter and its status	51
Table 37	FBCE test cases executed on the delivered fbce.py	52
Table 38	Results of the training pipeline on the synthetic dataset (160 test records, 12 approved; logistic regression and random forest only)	54
Table 39	Stratified five-fold cross-validation results, fold means (Experiment A, earlier evaluation phase)	54
Table 40	Output of train_model.py on the real dataset (values to be inserted from models/model_results.csv)	55
Table 41	Case-study inputs and prototype outputs (synthetic-data model; FBCE values from fbce.py)	56
Table 42	Differences between the earlier dashboard screenshots and the delivered code	57
Table 43	Reading the three signals together (interpretive guide, not an empirical result)	63
Table 44	Safeguards present in the prototype and safeguards still needed	67
Table 45	Summary of the delivered source files	70
Table 46	Execution sequence of the project files	71
Table 47	Verification status of the main evidence	81
Table 48	Known mismatches between report, screenshots and code	81
Table 49	Prerequisites	86
Table 50	Messages produced by train_model.py and what to do	87
Table 51	Troubleshooting	87

LIST OF ABBREVIATIONS

Abbreviation	Full form
AI	Artificial Intelligence
AUC	Area Under the (ROC) Curve
CSE	Computer Science and Engineering
CSV	Comma-Separated Values
EMI	Equated Monthly Instalment
FBCE	Financial Behaviour Consistency Engine
FN / FP	False Negative / False Positive
FR / NFR	Functional / Non-Functional Requirement
FOIR	Fixed Obligation to Income Ratio
LIME	Local Interpretable Model-agnostic Explanations
LR	Logistic Regression
ML	Machine Learning
RF	Random Forest
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SME	Small and Medium Enterprise
SMOTE	Synthetic Minority Oversampling Technique
TN / TP	True Negative / True Positive
XGBoost	Extreme Gradient Boosting

CHAPTER 1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Lending institutions receive a continuous stream of loan applications, and each decision affects two parties at once: the applicant, whose plans may depend on the outcome, and the lender, whose exposure to credit risk depends on it. Manual appraisal of income, employment, credit history and repayment ability is thorough but slow, and two officers can reach different conclusions on the same file. Institutions therefore look for tools that provide a consistent first-level assessment while leaving the final judgement with people.

Supervised machine learning is the usual answer. Classifiers such as logistic regression, random forest, XGBoost and LightGBM are trained on earlier lending decisions and, in published studies, are compared on loan-approval data [1]–[3]. A predicted label, however, carries no reason. A refused applicant expects an explanation, and lenders must be able to defend automated decisions before auditors and regulators [4], [5]. Post-hoc explanation methods, namely SHapley Additive exPlanations (SHAP) [7] and Local Interpretable Model-agnostic Explanations (LIME) [8], partly address this by showing how each input moved an individual prediction.

This project, Explainable Loan Prediction Using Financial Behaviour Consistency Engine (FBCE), builds an academic decision-support prototype that examines every loan application in three complementary ways. First, a benchmarked machine-learning classifier predicts the outcome and its approval probability. Second, a rule-based consistency engine, the FBCE, scores how well the financial details of the application agree with one another and raises warnings when they do not. Third, an explanation layer based on SHAP shows how each feature pushed the classifier towards approval or rejection. A Streamlit dashboard displays the three views side by side so that a reviewer can see what the model predicts, why it predicts it, and whether the application itself holds together.

The FBCE does not replace the classifier and is not merged into it. It is a separate, transparent calculation over the raw application values, built from five components: income consistency, debt consistency, employment stability, credit consistency and repayment capacity. Keeping the signals apart means that any disagreement between them stays visible to the reviewer instead of being averaged away.

1.1.1 Background to loan prediction

A loan application normally passes through a sequence of stages: submission of the application form, verification of the supporting documents, appraisal of the applicant's income, existing obligations, credit history and the purpose of the loan, a decision by an authorised officer, and monitoring of repayment after disbursal. Statistical credit scoring automates part of this sequence by estimating, from historical records, how a new application is likely to be decided

or to perform [16], [17]. Benchmark studies compare many classification algorithms on credit data, and they report that simple models such as logistic regression often perform competitively with more complex ones [16].

It is important to state what the model in this project predicts. In the Loan Prediction dataset the label records whether the lender approved the application, not whether the borrower later repaid. A model trained on this label therefore estimates how a lender *would have decided*, and it inherits the strengths and the weaknesses of those past decisions. This distinction is repeated in the discussion of limitations and fairness (Section 4.7 and Section 4.8).

1.1.2 Challenges of financial decision support

- **Incomplete and noisy data.** Seven of the twelve attributes of the dataset used by the project contain missing values (Table 17), so imputation choices influence every later step.
- **Small and unbalanced samples.** The dataset has 614 applications, of which 68.7% are approved, and a single attribute, credit history, almost determines the label (79.6% approval with a good record against 7.9% without). Models trained on such data can look accurate while learning very little beyond that attribute.
- **Labels that reflect past practice.** Approval labels carry the preferences and the biases of earlier decision makers [21].
- **Heterogeneous units.** Income, loan amount and term are recorded in different units, and a rule that combines them must handle the units explicitly; Section 4.5.2 shows what happens when it does not.
- **Demand for reasons.** Applicants, auditors and regulators expect an automated decision to be explained [4], [5], and explanation methods have limits of their own [20].
- **Over-reliance.** A confident-looking score can be accepted without question by a busy reviewer. A decision-support tool should therefore show where its signals disagree and remind the reviewer that the decision remains human.

1.1.3 Purpose of behavioural consistency analysis

In this project the word *behaviour* has a specific and limited meaning. FBCE examines the financial behaviour that an application *implies*: how the requested borrowing, the declared income, the loan term, the credit record and the employment type fit together. It is a static analysis of the values entered for one application at one moment. The prototype has no access to transaction histories, bank statements or repayment histories over time, which is the kind of behavioural data used in some research on consumer credit risk [18]; the name of the engine should not be read as a claim to analyse such data.

The motivation is practical. A loan officer does not read income, loan amount, term and credit history in isolation but asks whether they make sense together: whether the instalment is affordable, whether the amount requested is plausible for the household, and whether a clean

credit record is consistent with the rest of the file. FBCE turns these questions into five transparent scores and a short list of warnings, so that the same questions can be asked for every application in the same way.

1.1.4 Importance of explainable predictions

Explanations serve different audiences. The applicant wants to know what could change the outcome, the reviewer wants to check that the model is relying on sensible information, and the developer wants to detect leakage or bias. Surveys of explanation methods emphasise that the choice of method depends on the audience and on the type of model [19], and an influential argument holds that for high-stakes decisions interpretable models should be preferred to explanations of black boxes [11]. FBCE follows this line in a modest way: the model loaded in the dashboard in all documented runs is a logistic regression, whose weights are directly readable, and SHAP is used to present those weights for one application at a time.

1.2 PROBLEM STATEMENT

Traditional loan-prediction models may return an approval or rejection outcome without sufficiently transparent financial reasoning. This creates difficulties for trust, auditability, interpretability, fairness review and human decision support. A second gap receives less attention: income, requested amount, loan term and credit history reach a classifier as separate columns, and nothing in the training objective asks whether they make sense together. A large request against a modest household income, or a clean credit record next to a heavy repayment burden, is exactly the kind of mismatch a loan officer looks for, yet a classifier can overlook it when such combinations are rare in its training data. Explaining the model does not close this gap, because an explanation only describes what the model has learned. The problems addressed by the project are:

- A classifier's label or probability alone does not tell a reviewer why a decision was made.
- Explanations describe the model, not the internal consistency of the application itself.
- Biased patterns in historical lending decisions, such as the influence of a protected attribute, may remain hidden behind an approval label.
- Borderline probabilities, close to the 0.5 decision threshold, are easily read as firm decisions.
- Reviewers need a single interface that brings the prediction, the consistency evidence and the explanation together.

1.3 MOTIVATION AND SIGNIFICANCE

Loan decisions are high-impact decisions. An unjustified refusal harms the applicant, while an unjustified approval exposes the lender to credit risk. Regulators and auditors expect automated credit decisions to be explainable [4], [5], and it has been argued that high-stakes decisions

should rely on interpretable models wherever possible [11]. The motivation for this work is to show that a lightweight and transparent prototype can give a reviewer more than a bare label: a probability, a consistency score with its components, a list of contradiction warnings and a ranked list of the features that drove the prediction.

The significance of the project lies in the separation of signals. In a design where heuristic penalties are merged into the model's own score, a low or high final value cannot be attributed either to the classifier or to a rule [6]. FBCE keeps the consistency score apart from the classifier output, so each can be inspected on its own. The prototype is explicitly positioned as support for human review and not as a replacement for it, and its score is a project-defined indicator and not a banking credit score.

1.4 AIM AND OBJECTIVES

The aim of the project is to design and implement an explainable loan-prediction prototype that combines machine-learning classification, a financial consistency engine and SHAP-based explanation in a single decision-support dashboard. The specific objectives are:

1. To build a loan-prediction model on a public loan-approval dataset.
2. To compare multiple classifiers (logistic regression, random forest, XGBoost and LightGBM) under one preprocessing pipeline and to select one for deployment.
3. To develop the FBCE scoring mechanism, which produces five component scores, an overall score, a consistency level and contradiction warnings.
4. To provide explainable predictions using SHAP feature contributions.
5. To integrate the workflow into a Streamlit dashboard for interactive use.
6. To support transparent human review and to document limitations and fairness concerns honestly.

1.5 SCOPE OF THE PROJECT

The prototype is a local academic demonstration. It runs in a web browser through Streamlit, collects eleven application attributes through a form and reports its outputs in separate panels. It has no connection to any banking system and provides no official credit decision. The scope includes:

- Data preprocessing (imputation, one-hot encoding and scaling) of a public loan-approval dataset in the common Kaggle format.
- Benchmarking of four classifiers: logistic regression, random forest, XGBoost and LightGBM.
- Five-component financial consistency scoring with consistency levels and contradiction warnings.
- SHAP-based explanation with ranked positive and negative contributors.

- A Streamlit dashboard presenting the prediction, probability, FBCE score, component scores, warnings and SHAP drivers.
- A synthetic-data generator that lets the software pipeline be tested when the real dataset is not available.

The following are outside the scope of the work reported here: training on repayment-outcome data, probability calibration, formal fairness metrics, a usability study with loan officers and production deployment. They are listed as future work in Chapter 5.

1.5.1 Intended users

Table 1. Intended users of the prototype

User	Interaction with the prototype	What the user needs	Not intended for
Loan officer or reviewer (illustrative role)	Enters one application in the form and reads the prediction, FBCE result and SHAP contributions.	A second opinion that shows reasons and warnings, not only a label.	Taking a lending decision on the basis of the output alone.
Developer (the student team)	Places or generates the dataset, runs the training script and starts the dashboard.	Repeatable commands, readable error messages, a saved model file.	Operating the system on real applicants.
Academic evaluator	Reads the report and may run the delivered code.	Evidence of what was executed and what is only reported.	–

The loan-officer role is an intended usage scenario of the design. The dashboard has not been evaluated with real loan officers (Section 4.7).

1.5.2 System boundaries

Table 2. Boundaries of the system

Inside the boundary	Outside the boundary
Reading one application entered in a browser form	Retrieving applicant data from banks, credit bureaus or documents
Predicting with a saved scikit-learn pipeline	Retraining or monitoring the model while the dashboard is running
Computing the FBCE score and warnings from raw values	Validating the FBCE thresholds against repayment outcomes
Computing SHAP contributions for the current application	Auditing the model for fairness with formal metrics
Showing the three outputs side by side	Producing an official credit decision, authentication, access control or an audit trail

1.5.3 Assumptions

- The dataset follows the column names of the common Kaggle Loan Prediction file, because the dashboard builds its input table with exactly these names.

- Incomes are monthly amounts and the loan amount is entered in the unit of the training data (thousands in the Kaggle file). The code does not convert units, so this assumption is not enforced.
- The FBCE repayment component assumes a fixed annual interest rate of 10%, a project assumption and not a market rate.
- The prototype runs on one local machine for one user at a time with a trusted model file.
- The synthetic generator is used only to test the software, and its results are not treated as evidence about real lending.

1.6 CONTRIBUTIONS

The contributions of the project, as supported by the delivered source code and the evidence in this report, are:

1. A single-pipeline benchmark script that compares four classifiers and saves the selected model together with its feature list (`train_model.py`).
2. The FBCE module (`fbce.py`), a transparent five-component consistency score with equal weights, four consistency levels and rule-based contradiction warnings, kept separate from the classifier output.
3. A Streamlit dashboard (`app.py`) that shows the prediction, the FBCE result and the SHAP contributions of one application side by side.
4. An evidence-based account of what was executed and what is reported from earlier work, including thirteen executed FBCE test cases, a sensitivity analysis of the scoring functions and a discussion of fairness and limitations.
5. A clearly labelled synthetic-data generator (`generate_demo_data.py`) for testing the software pipeline.

Of these, items 1–5 are implemented, and the evidence for each is stated in Section 4.5: the FBCE module and its warnings were executed and tested, the training script was executed for logistic regression and random forest only, the dashboard was reviewed but not executed, and the generator was executed. Fairness metrics, calibration, unit conversion and field evaluation are proposed work only (Section 5.2).

1.7 ORGANISATION OF THE REPORT

Chapter 2 reviews related work, identifies the research gap and summarises the theoretical background on which the prototype rests: classification, evaluation metrics, preprocessing, SHAP and credit-assessment concepts. Chapter 3 analyses the existing and proposed systems, states the requirements and presents the architecture and design diagrams. Chapter 4 explains the methodology (dataset, model pipeline and FBCE scoring rules), describes the implementation file by file and reports testing, results and discussion. Chapter 5 gives the

conclusion and future work. It also contains the complete source code, the results with a statement of the verification status of the reported evidence, and the user interface outputs, including the dashboard screenshots and the user guide. The references follow.

CHAPTER 2. LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

Work relevant to FBCE falls into three strands: Shapley-value explanations of credit models, classifier benchmarks for loan approval, and hybrid systems that pair a model with rule-based checks. This chapter summarises the sources used in preparing the project and relates each to the proposed system, and then gives the theoretical background used in the later chapters. Table 3 gives a compact comparison.

2.2 LITERATURE SURVEY

2.2.1 Explanations of credit models

Lundberg and Lee [7] introduced SHAP as a unified approach that decomposes a prediction into additive per-feature contributions, and Ribeiro et al. [8] proposed LIME, which fits a sparse local surrogate model around a single prediction. SHAP is the explanation technique used in the dashboard of this project; LIME is the other method considered in the project design. Rudin [11] argues that for high-stakes decisions inherently interpretable models should be preferred over post-hoc explanations of black boxes. This supports the choice of logistic regression as the model loaded in the dashboard, whose coefficients and SHAP values have a direct reading.

Bussmann et al. [4] applied Shapley-value explanations to credit data from about 15,000 small and medium enterprises seeking peer-to-peer credit and then used correlation networks over the Shapley values to group borrowers with similar explanations. De Lange et al. [5] proposed an explainable model for credit default on unsecured consumer loans from a Norwegian bank, combining LightGBM with SHAP; the LightGBM model clearly outperformed the bank's logistic-regression scoring model, and the most important explanatory variables were the volatility of the utilised credit balance, the remaining credit in percentage of total credit and the duration of the customer relationship. Oyeyemi et al. [9] report an explainable credit-risk study that uses SHAP and LIME.

2.2.2 Classifier benchmarks for loan approval

Putra and Rumini [1] compared logistic regression, random forest and XGBoost on merged application, previous-application and bureau data, with missing-value imputation, encoding, scaling and SMOTE class balancing. Logistic regression obtained the highest area under the ROC curve (0.7415), above random forest (0.7138) and XGBoost (0.7159). Ghahremani et al. [2] study machine-learning-based loan approval automation with attention to efficiency, accuracy and fairness in credit decisions, and Gawande et al. [3] describe an explainable ensemble framework for loan approval prediction.

These benchmarks show that model complexity does not guarantee better results on loan data of modest size. The comparison obtained in this project, in which logistic regression matched the ensembles (reported in Section 4.5.4), is consistent with the ranking in [1], although the datasets, preprocessing and metrics differ and the two results should not be compared numerically.

2.2.3 Hybrid systems with rule-based checks

The closest precedent known to the authors is the system of Shah et al. [6], described as a calibrated XGBoost model whose confidence is modified by rule-based penalties drawn from financial heuristics such as the debt-to-income ratio, with SHAP used for interpretation. If the penalties are applied to the model's own score, as described, a low final value cannot be attributed either to the classifier or to a rule. FBCE is designed to avoid this by keeping the rule-based score apart from the classifier output.

2.2.4 Dataset reference

The experiments use the public Loan Prediction Problem dataset hosted on Kaggle [10], which is widely used for classroom and benchmark work on loan approval. Reference [10] is a dataset page and not a peer-reviewed paper, so the dataset is described by its documented properties (Section 4.1.3) and not by any claim about real banking portfolios.

2.2.5 Traditional credit scoring and machine-learning approaches

Classical credit scoring uses statistical classifiers, above all logistic regression and discriminant analysis, to build scorecards. Baesens et al. [16] compared a wide range of classifiers on eight real credit-scoring data sets and found that neural networks and least-squares support vector machines performed very well, but also that simple classifiers such as logistic regression and linear discriminant analysis performed very well. Lessmann et al. [17] later updated this benchmark with novel classification algorithms and better performance measures, giving an independent baseline for scorecard development. Khandani et al. [18] applied machine-learning techniques to consumer credit risk by combining customer transaction data with credit bureau data, and reported out-of-sample forecasts that improved classification of delinquencies and defaults.

For this project the lesson is twofold. First, a logistic regression is a legitimate and competitive baseline, which supports its selection for the dashboard. Second, richer behavioural data (such as transactions) can improve credit models [18], but such data are not available in the public loan-approval dataset, so FBCE can only work on the declared application fields.

2.2.6 Feature engineering and financial behaviour indicators

Credit models do not use raw fields only. Derived indicators such as the ratio of debt or instalment to income express affordability, and Khandani et al. describe a measure of consumer

credit risk that combines traditional factors such as debt-to-income ratios with transaction data [18]. FBCE uses two derived ratios of this kind: the ratio of the requested loan to total income, and the ratio of the estimated monthly instalment (EMI) to total income. These ratios are the project's own design decisions; the sources reviewed here motivate the use of affordability indicators but do not prescribe the particular formulas, scale factors or thresholds of FBCE, which are listed in Table 28.

2.2.7 Explainable AI and model transparency

Guidotti et al. [19] classify methods for explaining black-box models by the type of explanation and the type of model and note that the lack of explanation is both a practical and an ethical issue. Rudin [11] argues that interpretable models should be used for high-stakes decisions instead of explaining black boxes after the fact. SHAP [7] and LIME [8] are model-agnostic post-hoc methods that explain individual predictions; both describe the model, not the world. The project therefore combines a model that is readable by construction (logistic regression) with a post-hoc presentation of its contributions (SHAP) and adds a separate rule-based view (FBCE).

2.2.8 Limitations of Shapley-value explanations

Kumar et al. [20] show that mathematical problems arise when Shapley values are used as feature-importance measures, that remedies add complexity such as the need for causal reasoning, and that Shapley-based explanations may not suit human-centric goals of explanation. Three practical consequences follow for this report. The value of a contribution depends on the chosen reference input, which in the dashboard is a vector of zeros (Section 4.3). Contributions of correlated or redundant features, such as the paired one-hot columns of a two-valued attribute, are shared in a way that need not match a human reading. And a contribution is not a causal effect: it does not say what would happen if the applicant changed the value.

2.2.9 Fairness and responsible decision support

Fuster et al. [21] studied U.S. mortgage data and found that machine-learning models predicted default more accurately than simpler models but that Black and Hispanic borrowers were disproportionately less likely to gain from their introduction. Ghahremani et al. [2] address fairness together with efficiency and accuracy in loan approval automation. Two lessons are relevant here: better predictive accuracy does not by itself guarantee equitable outcomes, and fairness must be assessed with measures designed for it. The delivered prototype computes no fairness metric; it only makes the signed contributions of attributes such as gender visible to the reviewer. This is a transparency aid and not a fairness guarantee.

2.3 SUMMARY OF RELATED WORK

Table 3. Summary of related work

Ref.	Focus (as stated by the sources)	Models / explanation	Consistency or rule layer
[1]	Bank loan approval from application, previous-application and bureau data	LR, RF, XGBoost; SMOTE; AUC comparison	–
[2]	Loan approval automation: efficiency, accuracy and fairness	Machine-learning classifiers (details not restated here)	Fairness considerations
[3]	Explainable ensemble framework for loan approval	Ensemble; explanation methods	–
[4]	Credit risk of about 15,000 SMEs in peer-to-peer lending	Shapley values with correlation networks	–
[5]	Credit default on consumer loans of a Norwegian bank	LightGBM vs. logistic-regression scorecard; SHAP	–
[6]	Loan approval with heuristic risk assessment	Calibrated XGBoost; SHAP	Heuristic penalty applied to the model score
[9]	Explainable credit-risk assessment	ML models; SHAP and LIME	–
This work	Loan Prediction dataset format (614 applications in the earlier evaluation)	LR, RF, XGBoost, LightGBM; SHAP	Separate five-component score with contradiction warnings

Table 4. Qualitative comparison of loan-decision approaches (assessment by the authors, not a measured result)

Approach	Reviewer receives	Explains the model?	Checks application consistency?	Rule signal kept separate?	Validated on repayment?
Manual appraisal	Officer's judgement	Not applicable	Yes, informally	Not applicable	Not applicable
Logistic-regression scorecard [16], [17]	Score or probability	Yes, readable weights	No	Not applicable	Depends on study
Black-box classifier alone	Label or probability	No	No	Not applicable	Depends on study
Classifier with SHAP or LIME [7], [8]	Label and contributions	Yes, post hoc	No	Not applicable	Depends on study
Classifier with merged rule penalties [6]	Single modified score	Partly	Yes, through rules	No	Not established here
FBCE prototype (this work)	Label, probability, FBCE score, warnings, SHAP list	Yes (SHAP)	Yes (five components, three rules)	Yes	No

The criteria of Table 4 are the questions that motivated the design of FBCE. The entries for other approaches are generalisations from the sources reviewed and are not measurements; where a source does not state a property, the table says so.

2.4 RESEARCH GAP

The surveyed studies show that explanation techniques are now routine in credit and loan-approval work, but they describe the classifier's output. None of the sources reviewed here is

described as measuring the consistency of an application on its own, and the single hybrid design [6] folds its rules into the model score. This is the gap addressed by the project: FBCE keeps the two apart, so that the consistency score, its components and any warnings are shown alongside the model probability and not merged with it. The gap statement is limited to the sources reviewed in this chapter; it is not a claim about the whole literature.

Table 5. Research gaps and the extent to which the project addresses them

Gap	Addressed by the project?	How	What remains unresolved
A label or probability gives no reason	Partly	SHAP contributions and FBCE components accompany the prediction	Contributions are not causal and depend on the reference input
Consistency of the application is not measured	Partly	Five component scores and three warning rules	Thresholds and scale factors are unvalidated; loan and income units are mixed
Rule signals merged into the model score	Yes, by design	FBCE and the classifier output are kept separate	No study of how reviewers interpret the two signals
Protected attributes may influence the model unnoticed	Partly	Signed contributions make such weights visible	Gender and marital status remain model features; no fairness metrics
Probabilities may be unreliable	No	–	No calibration; probabilities are not validated
Evidence on real repayment outcomes	No	–	Public approval labels only; no external validation

Table 5 separates the gaps that the design addresses from those that remain open. The project does not claim to close the research gap in general; it demonstrates one way of keeping a transparent consistency signal beside a model and an explanation.

2.5 THEORETICAL BACKGROUND

This section explains the concepts used by the prototype so that the later chapters can refer to them without interruption. The explanations are general; the specific settings used by the project are given in Chapter 4.

2.5.1 Loan approval as a binary classification problem

Loan approval is treated as a supervised binary classification task. Each historical application is described by a feature vector x (applicant income, loan amount, credit history and so on) and a label y, where $y = 1$ means that the loan was approved and $y = 0$ that it was rejected. The classifier learns a function that maps x to an estimate of $P(y = 1 \mid x)$. A decision threshold, 0.5 in this prototype, turns the probability into a label. Because the labels record what lenders did and not whether borrowers repaid, a model trained on them learns past lending practice, including any bias it contained.

2.5.2 Classification algorithms used

Logistic regression. Logistic regression models the log-odds of approval as a linear function of the transformed features z:

$$P(y = 1 \mid z) = \sigma(w^T z + b) = 1 / (1 + \exp(-(w^T z + b))) \quad (2.1)$$

where w is the weight vector and b the bias. Training minimises a regularised log-loss; in scikit-learn [15] the default regularisation is L2 with strength C = 1. Because the model is linear in the log-odds, each weight has a direct interpretation and its SHAP values can be written in closed form (Section 2.5.5). In the training script the model is created with `class_weight="balanced"` and `max_iter=2000`.

Random forest. A random forest [14] builds many decision trees on bootstrap samples of the training data, considers only a random subset of features at each split and averages the trees' votes, which reduces the variance of individual trees. The training script uses 300 trees with at least two samples per leaf and balanced class weights.

Gradient-boosted trees. Gradient boosting builds trees one after another, each fitted to the errors left by the trees before it. XGBoost [12] and LightGBM [13] are two widely used implementations. XGBoost grows trees to a limited depth, whereas LightGBM grows trees leaf-wise and is controlled mainly through the number of leaves. Both use a learning rate to shrink the contribution of each new tree. The configurations used in the training script are listed in Table 18.

2.5.3 Evaluation metrics

Let TP, TN, FP and FN be the numbers of true positives, true negatives, false positives and false negatives, with approval as the positive class. The metrics used in the project are:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN) \quad (2.2)$$

$$\text{Precision} = TP / (TP + FP) \quad (2.3)$$

$$\text{Recall} = TP / (TP + FN) \quad (2.4)$$

$$F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall}) \quad (2.5)$$

$$\text{Specificity} = TN / (TN + FP) \quad (2.6)$$

The area under the ROC curve (AUC) measures how well the predicted probabilities rank approved applications above rejected ones, independently of any threshold. Specificity matters here because it states how many historically rejected applications the model also rejects; a model with high recall and low specificity approves almost everything. The training script computes accuracy, precision, recall, F1 and ROC AUC on the held-out test split; specificity is not computed by the script and appears only in the cross-validation results reported in Section 4.5.4.

2.5.4 Data preprocessing

Imputation. Real application records contain missing values. The pipeline replaces missing numeric values with the column median, which is robust to outliers such as very high incomes, and missing categorical values with the most frequent category.

One-hot encoding. Categorical attributes such as Gender, Married, Dependents, Education, Self_Employed and Property_Area cannot be used directly by the algorithms. One-hot encoding replaces each attribute with one 0/1 column per category. The encoder is created with `handle_unknown="ignore"`, so a category that was not seen during training produces an all-zero block instead of an error.

Standardisation. Numeric features are standardised so that they have zero mean and unit variance on the training data:

$$z_j = (x_j - \mu_j) / \sigma_j \tag{2.7}$$

This puts income, loan amount and term on a comparable scale, which matters for the regularised logistic regression. Tree-based models are insensitive to monotonic scaling, but a single common pipeline keeps the comparison fair.

Leakage and stratification. Imputation, encoding and scaling statistics must be learned from the training data only; otherwise information from the test set leaks into the model. The project keeps all three steps inside one scikit-learn pipeline that is fitted on the training split. The data are split in a stratified way, so that the approved/rejected ratio is the same in both parts.

2.5.5 Explainability with SHAP

SHAP assigns each feature a contribution to a particular prediction using Shapley values from cooperative game theory [7]. The prediction $f(x)$ is written as a base value plus the sum of the feature contributions:

$$f(x) = \varphi_0 + \Sigma_{j=1}^{M} \varphi_j \tag{2.8}$$

where φ_0 is the model output at the reference (background) input and φ_j is the contribution of feature j. A positive φ_j pushes the prediction towards approval and a negative φ_j towards rejection. For a linear model whose features are treated as independent, the Shapley value reduces to

$$\varphi_j = w_j(z_j - b_j) \tag{2.9}$$

where b_j is the value of feature j in the background input. The dashboard passes a background of all zeros to the linear explainer (Section 4.3), so $b_j = 0$ and each contribution is simply $w_j z_j$. For a standardised numeric feature, zero equals the training mean; for a one-hot column, zero means that the category is absent, so the reference is not the average applicant but an all-zero input. For logistic regression the contributions are expressed in log-odds, not in probability. The one-hot and scaling steps give the contributions names such as `num__ApplicantIncome`

and `cat__Education_Graduate`, where the prefixes `num__` and `cat__` come from the column transformer.

2.5.6 Financial concepts used by FBCE

Equated monthly instalment (EMI). For a loan amount L repaid over n months at a monthly interest rate r, the instalment is

$$\text{EMI} = L \cdot r \cdot (1 + r)^n / ((1 + r)^n - 1) \tag{2.10}$$

with r = (annual rate / 100) / 12. If $r = 0$ the instalment is simply L / n. The FBCE module assumes an annual rate of 10% by default, which is a project assumption and not a market rate.

Fixed obligation to income ratio (FOIR). FOIR is the share of monthly income that goes to fixed repayments. It appears in the dashboard screenshots of Section 5.5 but is not computed by the delivered `app.py`.

Loan-to-income ratio. The ratio between the requested loan amount and the household income is a simple indicator of how heavy a request is relative to the applicant's means.

Why these are called project-defined. The thresholds and scale factors used in FBCE were chosen by the project team for demonstration. They have not been fitted to repayment outcomes and they are not official banking standards.

2.5.7 Class imbalance and class weights

When one class is much rarer than the other, a classifier can reach high accuracy by favouring the majority class. The scikit-learn option `class_weight="balanced"`, used for logistic regression and random forest in the training script, weights each class inversely to its frequency:

$$w_c = n / (k \cdot n_c) \tag{2.11}$$

where n is the number of training samples, k the number of classes (two here) and n_c the number of samples of class c. For a dataset with 68.7% approvals, rejected applications receive a weight of about 1.6 and approved applications a weight of about 0.73. XGBoost and LightGBM are configured without class weights in the delivered script.

2.5.8 Regularisation

Regularisation limits the size of the weights to reduce over-fitting. For L2-regularised logistic regression with strength C, scikit-learn minimises a penalised log-loss in which a smaller C means stronger regularisation; the script leaves C at its default of 1. Because the weights are shrunk, the SHAP contributions of a regularised linear model are also shrunk, so they should be read as properties of this fitted model.

2.5.9 Hold-out and cross-validation

A single hold-out split (80% training, 20% test in the script) is simple but its result depends on which records land in the test set, and with a few hundred records the variation can exceed the differences between models. In stratified k-fold cross-validation the data are divided into k parts that keep the class ratio, each part is used once as the test set, and the metrics are averaged. The earlier evaluation phase used five folds (Section 4.5.4); the delivered script uses one split (Section 4.5.3). The two are reported separately because they are different experiments.

2.5.10 Decision thresholds, probabilities and calibration

A classifier outputs a score that is turned into a label by a threshold; the dashboard reports the label of the saved pipeline together with the predicted probability of the approval class, so a probability of 0.5009 and one of 0.95 both give "APPROVED". A probability is *calibrated* if, among applications given probability p, a fraction p are actually positive. Balanced class weights shift the predicted probabilities, and no calibration step exists in the project, so the probabilities must be read as model scores and not as measured frequencies.

2.5.11 Properties of Shapley-based attributions

Lundberg and Lee [7] identify three desirable properties of additive feature attributions: local accuracy (the contributions plus the base value equal the model output for the explained input), missingness (a feature that is absent receives no contribution) and consistency (if a model changes so that a feature matters more, its contribution does not decrease). For the linear case of (2.9), local accuracy can be checked numerically, as done for the synthetic-data model in Section 4.3. A useful reading for logistic regression follows from the log-odds scale: a contribution of φ multiplies the odds of approval by exp(φ) relative to the reference input,

$$\text{odds ratio} = \exp(\varphi_j) \tag{2.12}$$

so a contribution of +0.873 corresponds to odds about 2.4 times higher, and −0.162 to odds about 15% lower, relative to the all-zero reference. This is a statement about the fitted model and not about real applicants.

CHAPTER 3. SYSTEM DESIGN

3.1 EXISTING SYSTEM

In current practice, and in most of the studies reviewed in Chapter 2, loan screening takes one of two forms. In manual appraisal, an officer examines income, employment, credit history and repayment ability. In automated screening, a trained classifier outputs a label and possibly a probability, optionally followed by a post-hoc explanation such as SHAP or LIME. The limitations of these approaches are:

- Manual appraisal is slow, and different officers can reach different conclusions on the same file.
- A bare label or probability gives no reason, so a refused applicant cannot see what drove the outcome.
- Explanations describe the classifier and not whether the application is internally coherent.
- Rules merged into the model score make it impossible to tell whether a low value came from the model or from a rule [6].
- Historical bias in lending decisions can pass unnoticed behind an approval label.

Table 6. Existing practice compared with the proposed system

Criterion	Manual appraisal	Classifier with post-hoc explanation	Proposed FBCE prototype
Speed and repeatability	Slow; varies between officers	Fast and repeatable	Fast and repeatable
Reason for the outcome	Officer's notes	Feature contributions, if provided	SHAP contributions and FBCE components
Internal consistency of the application	Checked informally	Not checked	Five component scores and three warning rules
Visibility of disagreement between signals	Not applicable	Not applicable	Probability, FBCE score and SHAP shown side by side
Protected attributes	Subject to individual bias	May be used by the model unnoticed	Signed contributions make their weights visible
Validation against repayment	Experience of the officer	Depends on the data	None; thresholds are heuristic

Table 6 summarises where the proposed system differs from existing practice. The last row is deliberate: the proposed system adds transparency but no validation, and it should not be presented as more accurate than the alternatives.

3.2 PROPOSED SYSTEM

The proposed system processes every application along two parallel paths and presents the results side by side. On the prediction path, the application is preprocessed, scored by the saved classifier and explained with SHAP. On the consistency path, the same raw values are evaluated by transparent rules that produce five component scores, an overall FBCE score, a consistency

level and contradiction warnings. The dashboard collects the results without merging them into one number. The advantages are:

- **Transparency:** each output can be traced to the values the applicant entered or to a named feature contribution.
- **Separation of signals:** the model probability, the consistency score and the explanation can disagree, and the disagreement is visible.
- **Bias visibility:** signed SHAP contributions expose the influence of attributes such as gender.
- **Human-in-the-loop design:** the dashboard is meant to support a reviewer and carries a notice that it is not a banking decision.
- **Lightweight deployment:** the prototype runs locally on a laptop-class machine.

3.2.1 Design rationale

1. **Two independent paths.** The classifier path and the FBCE path read the same inputs but never each other's outputs. A disagreement between them is therefore informative and cannot be an artefact of one signal being tuned to the other.
2. **Raw values for FBCE.** FBCE works on what the user typed, so each component can be recomputed by hand from the form fields, as shown in Section 4.2.1.
3. **A readable model by default.** Logistic regression is competitive on small credit data [16], [17] and its weights are readable, which suits an explanation layer.
4. **One pipeline object.** Preprocessing and model are saved together, so the dashboard cannot apply a different preprocessing from the one used in training.
5. **Local execution.** No network service or database is needed, which removes a class of privacy and availability concerns for an academic prototype.
6. **Visible disclaimers.** The sidebar and the caption state that the output is not a banking decision.

3.3 SYSTEM ARCHITECTURE

Figure 1 outlines the architecture. The raw application follows two paths. It is preprocessed, scored by the saved classifier and explained with SHAP; in parallel, the consistency engine evaluates the same raw values with transparent rules. The Streamlit dashboard collects the results for the reviewer.

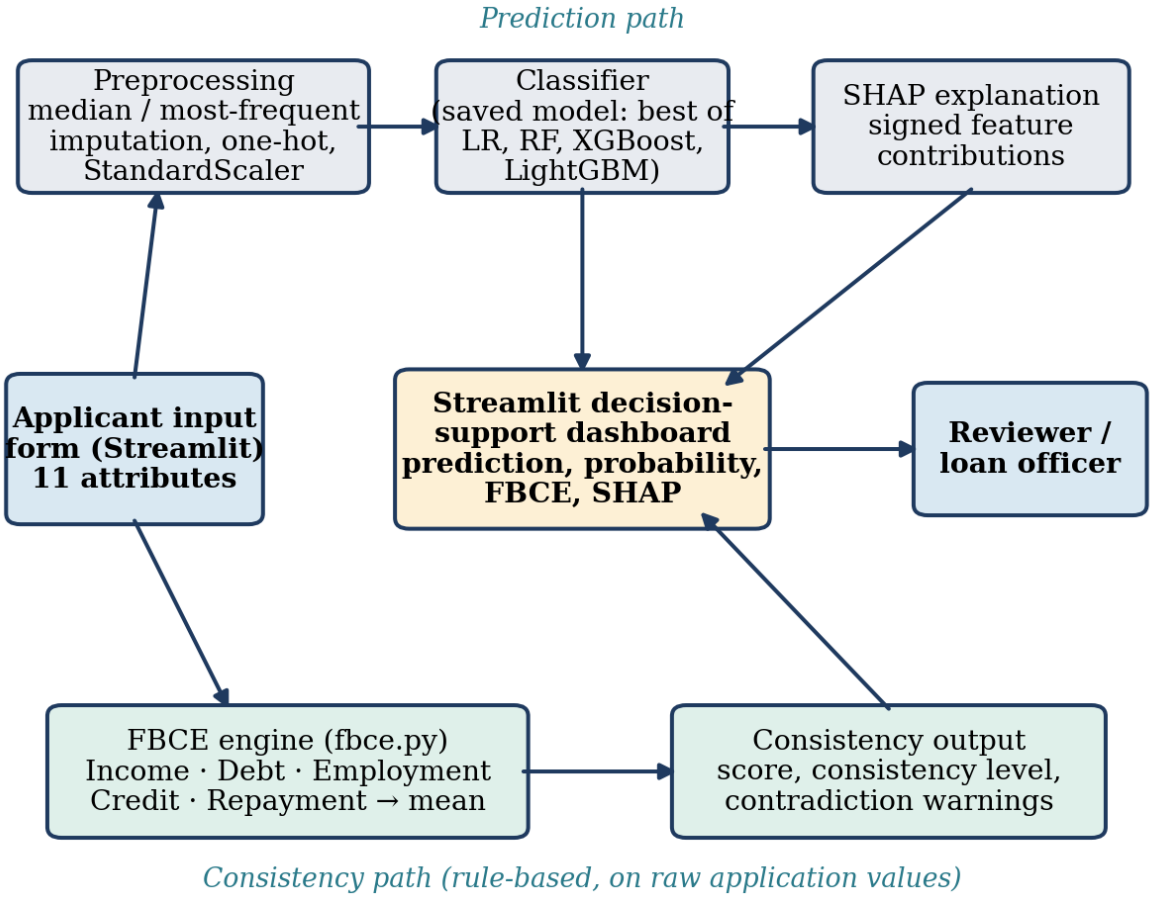


Figure 1. Architecture of the FBCE prototype

The components of the architecture are:

- **Applicant input form:** collects eleven attributes: applicant and co-applicant income, loan amount, loan term, credit history, self-employment, gender, marital status, dependents, education and property area.
- **Data preprocessing:** imputes missing values, one-hot encodes categorical attributes and standardises numeric attributes inside a single scikit-learn column transformer that is part of the saved pipeline.
- **Classification:** four classifiers are benchmarked during training and the best by F1-score (then accuracy) is saved; the dashboard loads whichever model was saved and displays its name in the sidebar.
- **Explanation layer:** SHAP gives per-feature contributions for the prediction, using a tree explainer for random forest, XGBoost and LightGBM and a linear explainer otherwise.
- **FBCE engine:** works on the raw application and returns the five component scores, the overall score, the consistency level and any contradiction warnings.
- **Dashboard:** shows the prediction, probability, FBCE results, SHAP table and chart and the lists of positive and negative contributors in separate panels.

3.3.1 Component responsibilities and interfaces

Table 7 lists what each component of Figure 1 receives and returns in the delivered code. The interfaces are Python function calls and files; no network protocol or database is involved.

Table 7. Component interfaces in the delivered implementation

Component	Implemented in	Receives	Returns	Failure behaviour
Input form	app.py (st.form)	User entries	Eleven values on submit	Widgets restrict numbers to non-negative values and categories to fixed lists
Preprocessing and classifier	Saved pipeline (train_model.py)	One-row DataFrame	Class label, probability of approval	Exception shown as "Prediction failed"
FBCE engine	fbce.py	Incomes, loan, term, credit history, employment type	Dictionary: scores, score, level, total income, warnings	Non-numeric credit history raises ValueError
Explanation layer	app.py (shap)	Preprocessor, estimator, transformed row	Top-ten contribution table, chart, lists	Exception shown as "SHAP explanation could not be generated"
Dashboard	app.py	Results of the above	Panels, summary and caption	Warning and stop if the model file is missing

3.3.2 Data items moved through the system

Table 8. Data items

Data item	Created by	Format	Used by
data/loan_data.csv	User or generate_demo_data.py	CSV with Kaggle-format columns	train_model.py
models/fbce_model.pkl	train_model.py	joblib dictionary: model, model_name, feature_columns	app.py
models/model_results.csv	train_model.py	CSV of accuracy, precision, recall, F1, ROC_AUC per model	Reader of the results
Application row	app.py form	One-row pandas DataFrame, eleven columns	Pipeline and SHAP
FBCE result	calculate_fbce	Python dictionary	app.py panels

3.4 REQUIREMENTS ANALYSIS

3.4.1 Functional requirements

- Accept the eleven application attributes through a form.
- Predict approval or rejection and report the approval probability.
- Compute the five FBCE component scores, the overall score and the consistency level.
- Raise readable contradiction warnings when rules fire.
- Display the ten largest SHAP contributions of the application, as a table, a bar chart and lists of positive and negative contributors.
- Show a notice that the FBCE score is a project-defined indicator and not a banking credit score.

3.4.2 Non-functional requirements

- **Interpretability:** every output must be traceable to inputs or to named features.
- **Reproducibility:** the training script uses a fixed random seed (42) and saves the fitted pipeline, which the application reloads.
- **Usability:** a single-page browser interface with clearly separated panels.
- **Portability:** runs locally on a standard laptop without any banking-system connection.

3.4.3 Requirements with identifiers and verification status

The requirements of this section are numbered so that the test matrix of Section 4.5.7 can refer to them.

Table 9. Functional requirements

ID	Requirement	Realised in	Verification
FR-1	Accept eleven application attributes	app.py form	Reviewed; not run
FR-2	Predict approval or rejection and report the probability	app.py, saved pipeline	Reviewed; pipeline executed on synthetic data
FR-3	Compute five component scores, overall score and level	fbce.py	Executed (T1–T13)
FR-4	Raise contradiction warnings	fbce.py	Executed (T4–T7, T13)
FR-5	Show ranked positive and negative SHAP contributors	app.py	Reviewed; not run
FR-6	Show the not-a-credit-score notice	app.py sidebar and caption	Reviewed; not run
FR-7	Train and compare four classifiers and save the best	train_model.py	LR and RF executed; XGBoost and LightGBM not run
FR-8	Generate a synthetic dataset	generate_demo_data.py	Executed

Table 10. Non-functional requirements

ID	Requirement	How it is met	Verification
NFR-1	Interpretability: outputs traceable to inputs or named features	Raw-value FBCE; named SHAP features	Worked example and tests
NFR-2	Reproducibility: fixed seeds and saved pipeline	random_state 42; joblib bundle	Executed (R1, R2)
NFR-3	Usability: single-page interface with separate panels	Streamlit layout	Not run
NFR-4	Portability: local run without banking systems	No network calls in the code	Code review
NFR-5	Honest labelling of limitations	Sidebar and caption; this report	Code review

3.4.4 Input validation and error handling as implemented

Table 11. Input validation and error handling in the delivered code

Where	Mechanism	Effect	Gap
app.py number inputs	min_value 0.0 for incomes and loan amount, 1 for the term	Negative entries cannot be typed	No upper limit and no unit check
app.py selection boxes	Fixed option lists	Only listed categories reach the model	None
app.py model loading	Existence check, warning and st.stop()	Clear instruction to train first	A corrupt file raises an exception
app.py prediction block	try/except around prediction and FBCE	Error text and a hint are displayed	Messages are generic
app.py SHAP block	Separate try/except	Prediction remains visible if SHAP fails	No fallback explanation
fbce.py	Income ≤ 0 and non-positive loan or term return 0	No division by zero	Non-numeric credit history raises ValueError
train_model.py	File and target-column checks, unmapped rows dropped	Readable errors	Silent dropping of unmapped targets

3.5 HARDWARE REQUIREMENTS

Table 12. Hardware requirements used for development

Component	Specification
Processor	Intel Core i5 or AMD Ryzen 5 class processor
Memory	8 GB RAM
Machine type	Laptop or desktop computer

The workload is light. The dataset has only a few hundred records, the classifiers are small, and SHAP for a logistic-regression model is computed from the model weights, so no GPU is required.

The hardware in the table is a development specification, not the result of a hardware study. The dataset has a few hundred rows, the models are small and the explanation of a single application is cheap, so the modest specification is a reasonable expectation. The report does not claim that other hardware configurations were tested, and the executed checks of this report were run in a different (server) environment described in Section 3.6.

3.6 SOFTWARE REQUIREMENTS

Table 13. Software tools and libraries

Software / library	Purpose	Evidence
Python 3.10 or 3.11	Programming language recommended for the project	README.md
pandas, NumPy	Data handling and numerical computation	requirements.txt; train_model.py

Software / library	Purpose	Evidence
scikit-learn	Pipeline, preprocessing, logistic regression, random forest, metrics, data split	requirements.txt; train_model.py
XGBoost, LightGBM	Gradient-boosted classifiers in the benchmark	requirements.txt; train_model.py
joblib	Saving and loading the fitted model bundle (models/fbce_model.pkl)	requirements.txt; train_model.py; app.py
SHAP	Feature-level explanation	requirements.txt; app.py
Streamlit	Browser-based dashboard	requirements.txt; app.py
Matplotlib	Bar chart of SHAP contributions in the dashboard	requirements.txt; app.py
Seaborn, VS Code, Jupyter Notebook	Plots and development environment during the earlier evaluation phase (not required by the delivered code)	Earlier phase report
Web browser	Viewing the dashboard (screenshots were captured in Microsoft Edge)	Dashboard screenshots

The file `requirements.txt` lists the nine packages pandas, numpy, scikit-learn, xgboost, lightgbm, shap, streamlit, matplotlib and joblib without version numbers. The library versions used on the authors' machine are therefore not recorded in the delivered files; a reproducible installation should record them with `pip freeze`.

The checks executed for this report used Python 3.12.3, pandas 3.0.2, NumPy 2.4.4 and scikit-learn 1.8.0. The README recommends Python 3.10 or 3.11, so these are not the versions the authors recommend. XGBoost, LightGBM, SHAP and Streamlit were not installed in that environment, which is why the corresponding code was reviewed but not executed (Section 4.5.1).

3.7 PROJECT FOLDER STRUCTURE

The README describes the folder layout of the project shown in Figure 2. Data are expected in `data/loan_data.csv`, and training writes the model and the comparison table to the `models` folder.

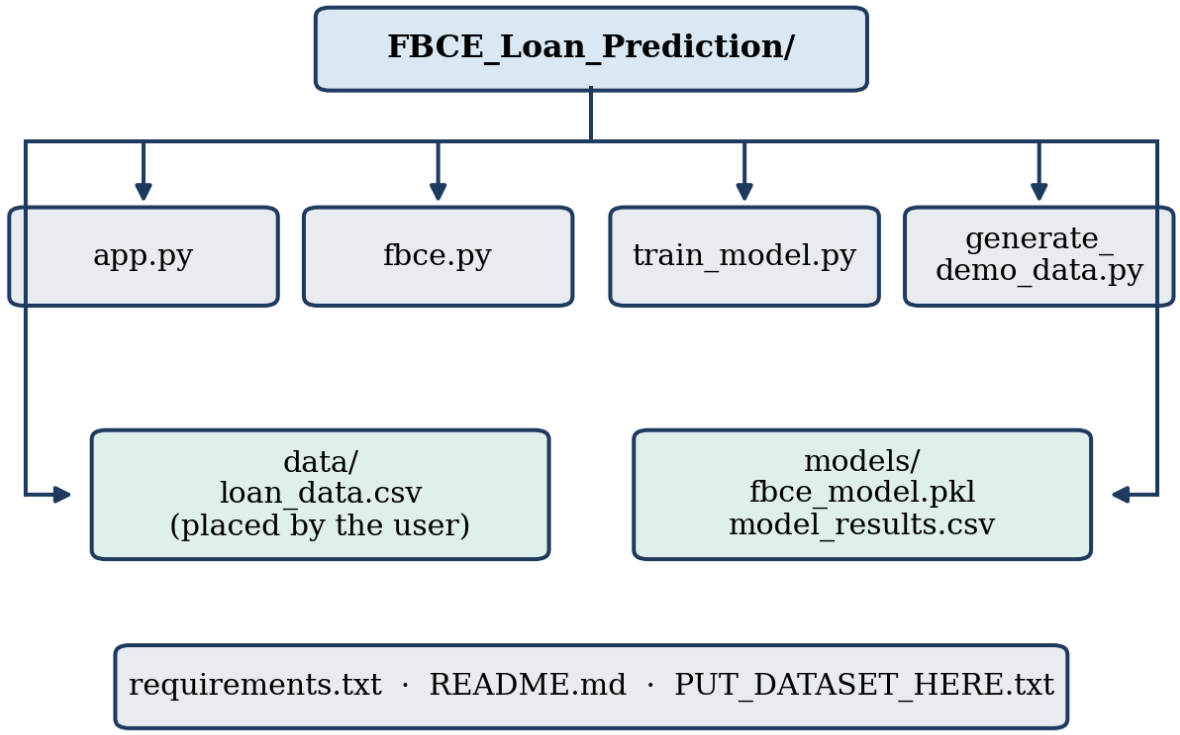


Figure 2. Project folder structure (from README.md)

Table 14. Role of each file and folder

File or folder	Role	Produced or read by
app.py	Dashboard: form, prediction, FBCE, SHAP, summary	Reads models/fbce_model.pkl and imports fbce.py
fbce.py	Five scoring functions and calculate_fbce	Imported by app.py
train_model.py	Preprocessing, four classifiers, ranking, saving	Reads data/loan_data.csv; writes models/
generate_demo_data.py	Synthetic 800-row dataset	Writes data/loan_data.csv
requirements.txt	Nine unpinned dependencies	pip
README.md	Setup, outputs and disclaimer	Reader
PUT_DATASET_HERE.txt	Dataset placement note	Reader
data/	Holds loan_data.csv	Created by the user
models/	Holds fbce_model.pkl and model_results.csv	Created by train_model.py if missing

3.8 USE-CASE, DATA-FLOW AND INTERACTION DESIGN

This section models the system from the viewpoint of its users and its data. The diagrams describe the behaviour of the delivered source files.

3.8.1 Use cases

Two kinds of users interact with the prototype. A **loan officer or reviewer** enters applicant details, requests a prediction and reads the probability, the FBCE results and the SHAP contributions. A **developer** (in this project, the student team) prepares the dataset, generates

synthetic data when the real data are missing, and trains and compares the models. The use cases are shown in Figure 3.

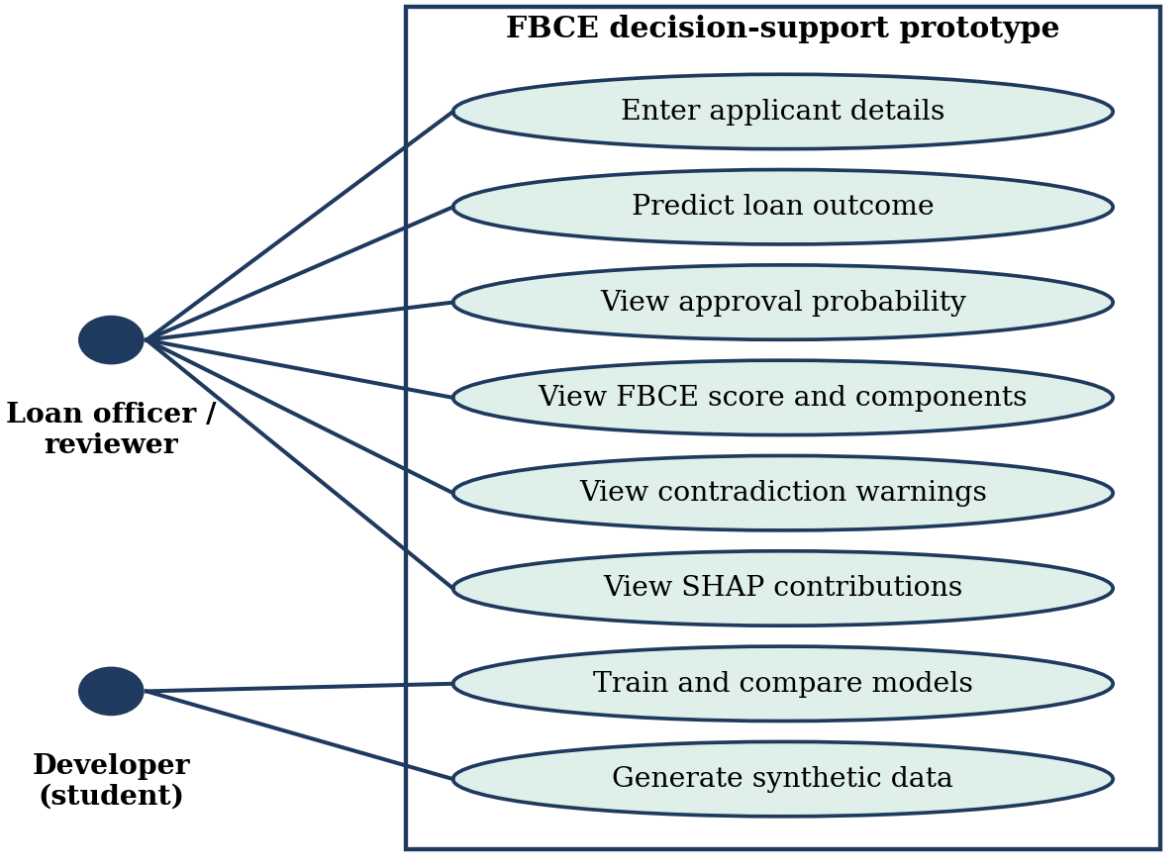


Figure 3. Use-case diagram of the FBCE prototype

3.8.2 Data flow

Figure 4 shows the data flow at the first level of detail. The reviewer's input goes to three processes that operate independently on the same raw values: preprocessing and prediction (which uses the saved pipeline in the model store), the FBCE consistency calculation, and the SHAP explanation, which uses the same saved pipeline. A fourth process composes the panels that are returned to the reviewer. The dataset is used only at training time and is therefore not shown.

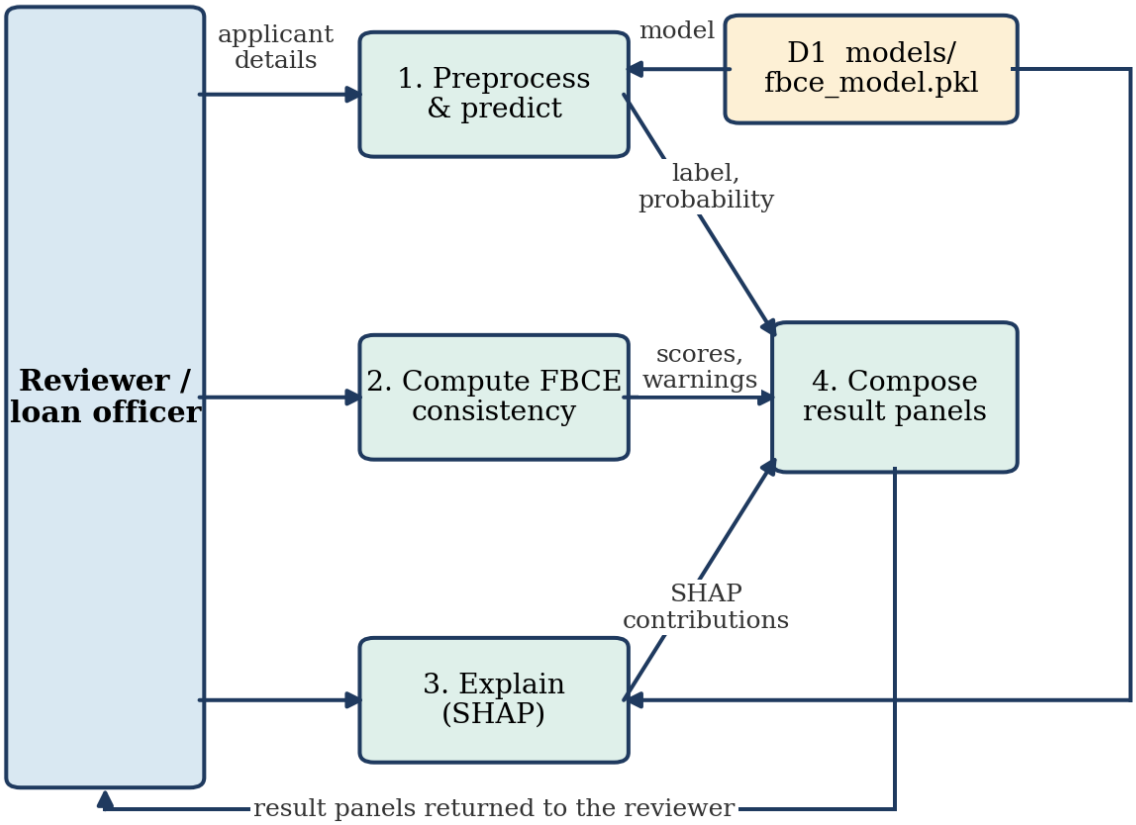


Figure 4. Level-1 data-flow diagram

3.8.3 Interaction sequence

Figure 5 shows the order of calls when the reviewer presses the “Predict Loan” button. The dashboard builds a one-row table from the form values and sends it to the saved pipeline, which returns the label and the approval probability; the dashboard then calls the FBCE function with the raw inputs, requests SHAP values for the same application and finally renders the panels. The prediction and the FBCE calculation do not depend on each other, which is what allows the two signals to disagree visibly.

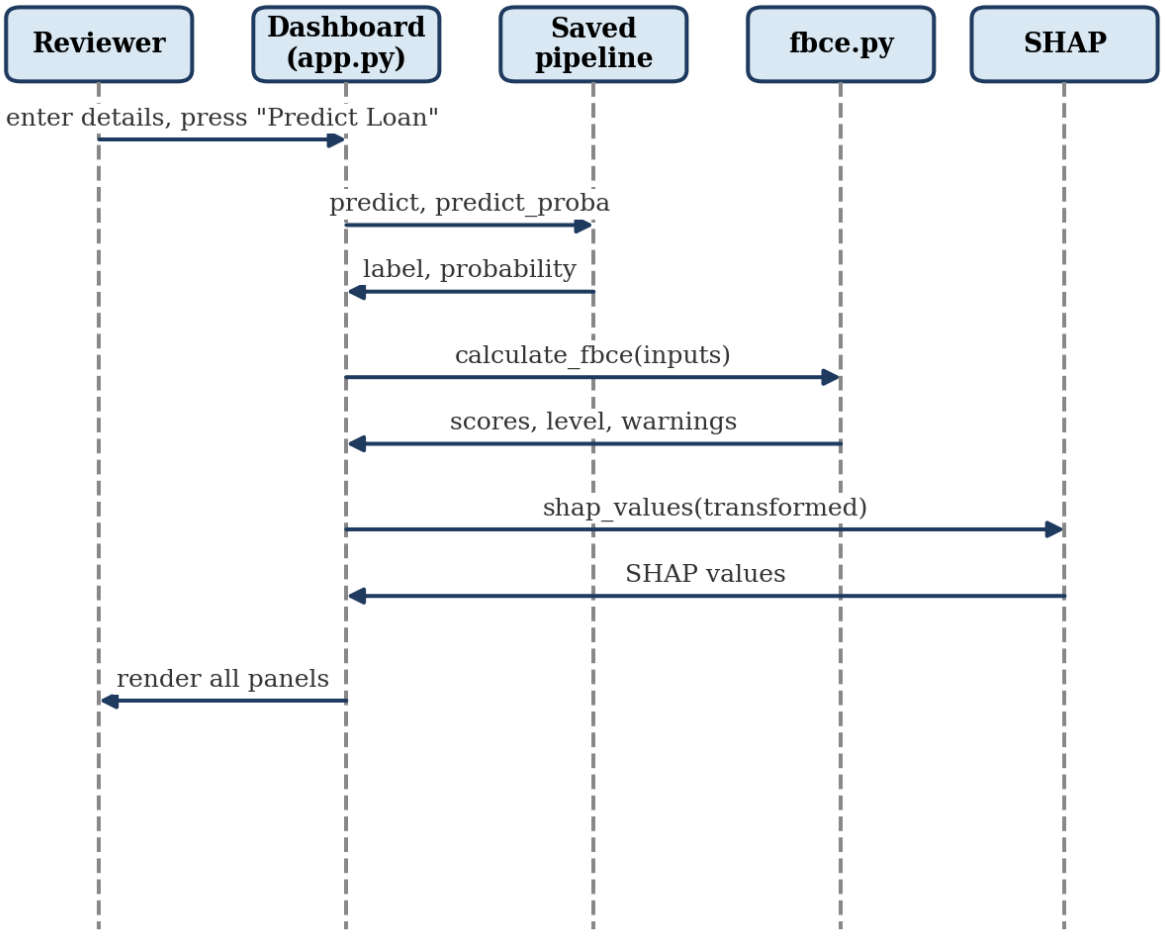


Figure 5. Sequence of interactions for one prediction request

Reading the diagrams. The use-case diagram (Figure 3) separates two roles: the reviewer uses the first six use cases, which all happen in the dashboard after one submission, and the developer uses the last two, which are command-line activities before the dashboard is started. The data-flow diagram (Figure 4) shows that the three processes receive the same applicant details and that only process 4 combines their outputs. The saved pipeline in the model store is read by the prediction process and by the explanation process, because SHAP needs the fitted preprocessor and estimator. The sequence diagram (Figure 5) adds the order of calls and makes clear that the FBCE call does not wait for, or use, the classifier result.

Error paths. Two error paths are implemented. If any step of prediction or FBCE raises an exception, the dashboard prints “Prediction failed”, the exception text and a reminder about feature names. If only the SHAP step fails, the prediction and FBCE panels are still shown and the SHAP section shows the error. Missing model files are handled before the form is displayed.

3.9 MODULE DEPENDENCIES

Figure 6 shows how the source files depend on each other. `train_model.py` reads the dataset and writes the model file; `generate_demo_data.py` writes a synthetic dataset to the same

location when the real one is missing; `fbce.py` is a self-contained module with no dependency on the model; and the dashboard combines the saved model, the FBCE module and the SHAP library. Keeping `fbce.py` independent of the classifier is what allows it to be tested on its own, as done in Section 4.5.2.

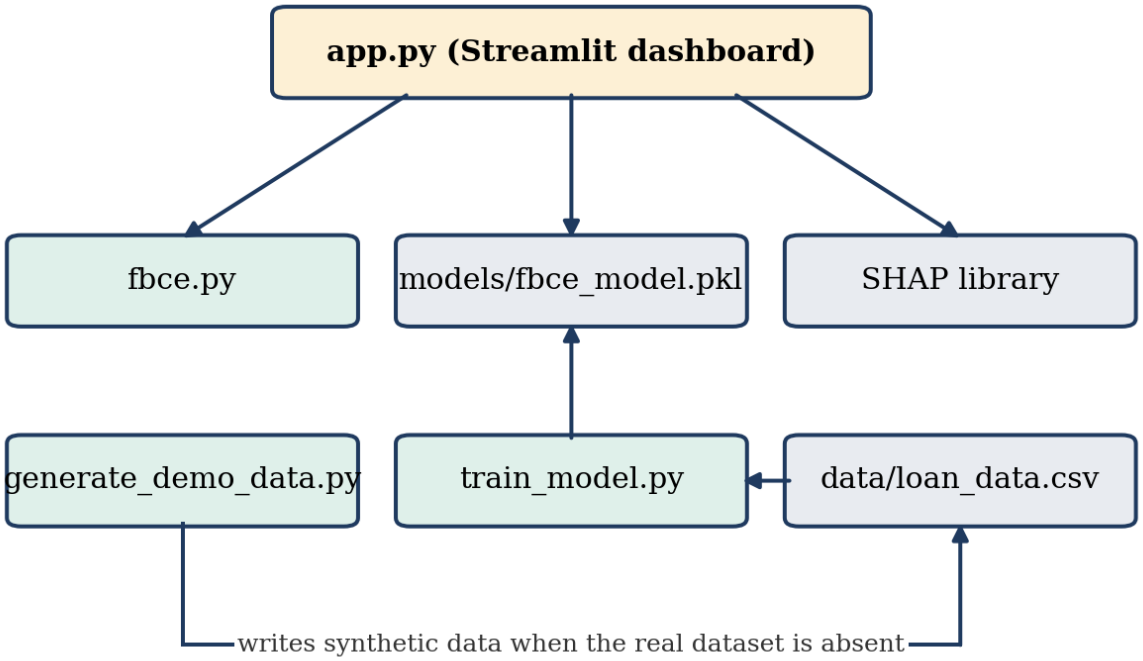


Figure 6. Dependencies between the project files

Table 15. Dependency matrix (row depends on column)

	fbce.py	train_model.py	model file	dataset	SHAP	Streamlit
app.py	imports	–	reads	–	imports	imports
train_model.py	–	–	writes	reads	–	–
generate_demo_data.py	–	–	–	writes	–	–
fbce.py	–	–	–	–	–	–

Table 15 shows that `fbce.py` depends on nothing but the Python standard library, which is why it can be tested alone. The dashboard has the largest set of dependencies and is therefore the least testable component without the external libraries; the dependence of the dashboard on a model file means that a stale or untrusted file would be loaded without any check (Section 4.8).

CHAPTER 4. IMPLEMENTATION

This chapter explains how the prototype was built and evaluated. Section 4.1 describes the methodology (dataset, preprocessing, classification pipeline and the FBCE scoring rules), Section 4.4 describes the implementation of each source file, and Section 4.5 reports testing, results and discussion. Each statement is tied to the source file or to the evidence that supports it, and results that were executed for this report are kept apart from results that are reported from the project's earlier evaluation phase.

4.1 METHODOLOGY

4.1.1 End-to-end workflow

The project follows the nine stages of Table 16. The stages are listed in the order in which they happen when the project is set up and used, and the last column states honestly how far each stage was verified for this report.

Table 16. End-to-end workflow and the verification status of each stage

Stage	Activity	File or function	Output	Status in this report
1 Data acquisition	Place the Kaggle-format CSV, or generate synthetic data	generate_demo_data .py	data/loan_data. csv	Synthetic executed; real file not available
2 Target preparation	Map Y/N to 1/0, drop Loan_ID and unnamed columns	load_data()	X, y	Executed on synthetic data
3 Preprocessing	Median or most-frequent imputation, scaling, one-hot encoding	build_preprocessor()	20 model inputs	Executed
4 Model development	Fit four classifiers on a stratified 80/20 split	build_models(), main()	Metrics per model	LR and RF executed; XGBoost and LightGBM not run
5 Selection and saving	Rank by F1 then accuracy; save the best pipeline	main()	models/fbce_m odel.pkl	Not run end to end
6 Inference	Build one-row table; predict label and probability	app.py	Label, probability	Reviewed; pipeline step executed
7 Consistency scoring	Compute five components, score, level, warnings	calculate_fbce()	Dictionary	Executed
8 Explanation	SHAP contributions of the application	app.py	Top-ten list	Reviewed; linear closed form executed
9 Evaluation	Metrics on held-out data	main(); earlier 5-fold study	Tables	Experiment B partly executed; Experiment A reported

4.1.2 Implemented, executed and proposed steps

Three categories are used consistently in this report. **Implemented** means that the feature exists in the delivered source code. **Executed** means that the code was run while preparing this report and the output is quoted. **Reported** means that a number comes from the project's earlier work and could not be reproduced from the delivered files. **Proposed** means future work that does

not exist in the code. A feature can be implemented without having been executed here, as is the case for the dashboard.

4.1.3 Dataset

The project targets the common public Loan Prediction dataset of the Kaggle format [10]. The README and `PUT_DATASET_HERE.txt` list the expected columns: Loan_ID, Gender, Married, Dependents, Education, Self_Employed, ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, Property_Area and Loan_Status. The experiments of the earlier evaluation phase used the 614-application version of this dataset: 422 applications were approved (68.7%) and 192 rejected (31.3%), and seven attributes contain missing values (Table 17).

Table 17. Attributes of the Loan Prediction dataset (as recorded in the earlier evaluation phase)

Attribute	Type and values	Missing	Used by FBCE
ApplicantIncome	Numeric, monthly	0	Yes
CoapplicantIncome	Numeric, monthly	0	Yes
LoanAmount	Numeric, in thousands	22	Yes
Loan_Amount_Term	Numeric, months	14	Yes
Credit_History	1 = meets guideline, 0 = not	50	Yes
Self_Employed	Yes, No	32	Yes
Gender	Male, Female	13	–
Married	Yes, No	3	–
Dependents	0, 1, 2, 3+	15	–
Education	Graduate, Not Graduate	0	–
Property_Area	Urban, Semiurban, Rural	0	–
Loan_Status	Target: Y (422), N (192)	0	–

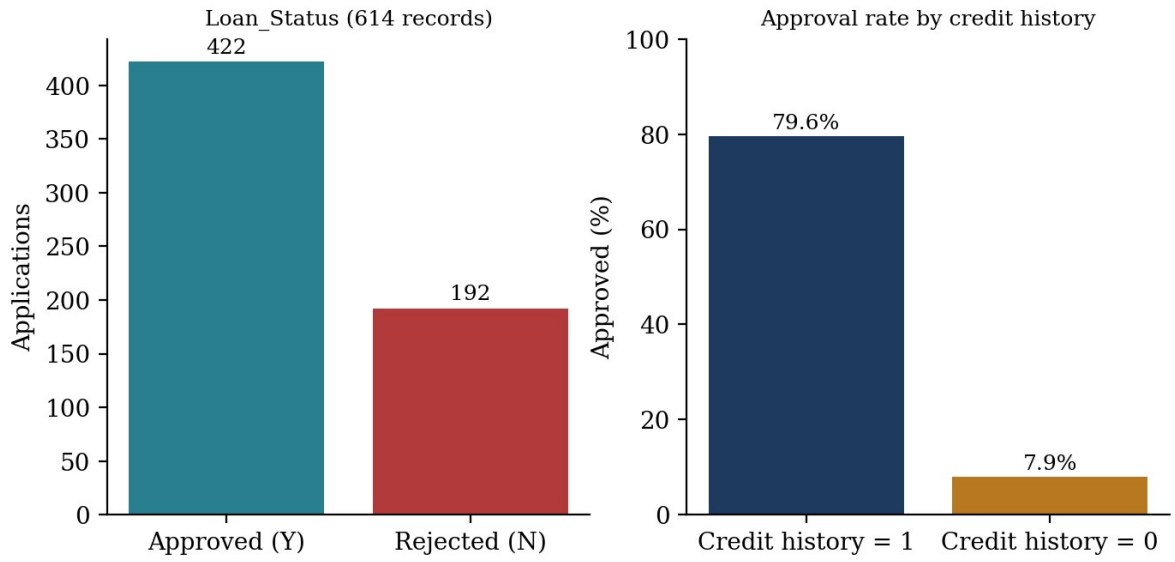


Figure 7. Class balance and approval rate by credit history (earlier exploratory analysis)

The exploratory analysis showed how strongly credit history shapes the outcome. Among records with a recorded value, 79.6% of applicants who met the credit guideline were approved, compared with 7.9% of those who did not. Self-employment made almost no difference: 68.3% of self-employed and 68.6% of salaried applicants were approved.

Important data note. The README states that the common Kaggle dataset has no explicit existing-debt column and no employment-duration column. FBCE therefore uses the requested-loan burden as an academic proxy for debt and the Self_Employed field as a proxy for employment stability. These are not real banking standards and should not be described as such. The real dataset file itself is not part of the delivered project files; it is expected at `data/loan_data.csv`.

4.1.4 Synthetic demonstration dataset

When the real dataset is unavailable, `generate_demo_data.py` creates 800 synthetic applications with the same column names. Incomes and co-applicant incomes are drawn from log-normal distributions, loan amounts from a clipped normal distribution and categorical fields from fixed probabilities. The target is produced from a score that combines credit history, incomes, loan amount, education and property area plus random noise, and is converted to Y/N at a probability of 0.5. The script itself warns that this rule is only for testing the software pipeline and is not a real-world approval rule, and the README states that the data must not be presented as real banking data or accuracy evidence.

The generator was executed for this report with its fixed random seed (42). It produced a file of 800 rows and 13 columns in which only 60 applications are labelled Y and 740 are labelled N (7.5% approved); 654 records have Credit_History = 1. This strong class imbalance is a property of the generator's threshold and not of real lending data, and it is one more reason not to quote results on this dataset as evidence of model quality.

4.1.5 Data preprocessing

`train_model.py` prepares the data as follows:

1. The CSV is read from `data/loan_data.csv`. The target column is found by looking for Loan_Status, loan_status, LoanStatus or target.
2. Target values are converted to 1 or 0: Y, YES, APPROVED and 1 map to 1; N, NO, REJECTED and 0 map to 0. Rows whose target cannot be mapped are dropped.
3. The target and the Loan_ID identifier are removed from the features, and any column whose name starts with "Unnamed" is removed.
4. Columns of type object, category or bool are treated as categorical; all other columns are numeric.

5. Numeric columns are imputed with the median and standardised with StandardScaler. Categorical columns are imputed with the most frequent value and one-hot encoded with `handle_unknown="ignore"` and dense output.
6. The data are divided into 80% training and 20% test portions with a stratified split and random seed 42.

The preprocessor is built separately for each model and is part of the same scikit-learn pipeline as the classifier, so it is fitted on the training portion only. Figure 8 shows the workflow.

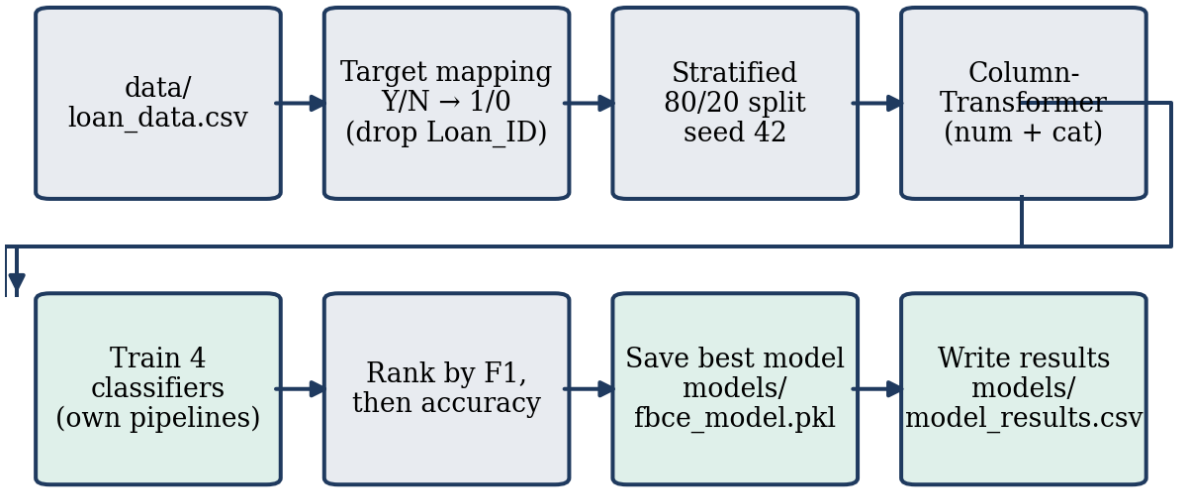


Figure 8. Model-development workflow implemented in train_model.py

4.1.6 Classification pipeline

Table 18. Classifier configurations in train_model.py

Model	Settings in the script
Logistic Regression	max_iter = 2000, class_weight = "balanced" (L2 penalty, C = 1 by scikit-learn default)
Random Forest	n_estimators = 300, min_samples_leaf = 2, class_weight = "balanced", random_state = 42
XGBoost	n_estimators = 250, max_depth = 4, learning_rate = 0.05, subsample = 0.9, colsample_bytree = 0.9, eval_metric = "logloss", random_state = 42
LightGBM	n_estimators = 250, learning_rate = 0.05, num_leaves = 31, random_state = 42, verbosity = -1

Each model is trained on the training split and scored on the test split. For each model the script records accuracy, precision, recall, F1 and ROC AUC. The rows are sorted by F1 and then by accuracy, both in descending order, and the first row is taken as the best model. The fitted pipeline, its name and the list of feature columns are saved with joblib to `models/fbce_model.pkl`, and the table is saved to `models/model_results.csv`.

Two evaluation set-ups exist in the project and are kept separate in this report. **Experiment A** is the stratified five-fold cross-validation of the earlier evaluation phase, which used XGBoost

with 300 rounds and depth 3 and LightGBM with 300 rounds and 15 leaves. **Experiment B** is the single 80/20 split implemented in the delivered `train_model.py` with the configurations of Table 18. The script for Experiment A is not part of the delivered files, so its numbers can be reported but not reproduced here (Section 4.5.4).

4.1.7 Feature set after preprocessing

With the Kaggle-format columns and default encoder settings (no category is dropped), the column transformer produces twenty model inputs: five numeric columns and fifteen one-hot columns. The count of twenty was confirmed by fitting the pipeline on the synthetic dataset. Credit_History is stored as a number (0 or 1) and is therefore treated as a numeric feature, which is why it appears as `num__Credit_History` in the dashboard. Table 19 lists the resulting names.

Table 19. Model inputs produced by the column transformer

Group	Source attribute	Output feature names
Numeric (5)	ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History	num__ApplicantIncome, num__CoapplicantIncome, num__LoanAmount, num__Loan_Amount_Term, num__Credit_History
Categorical (15)	Gender (2)	cat__Gender_Female, cat__Gender_Male
	Married (2)	cat__Married_No, cat__Married_Yes
	Dependents (4)	cat__Dependents_0, cat__Dependents_1, cat__Dependents_2, cat__Dependents_3+
	Education (2)	cat__Education_Graduate, cat__Education_Not Graduate
	Self_Employed (2)	cat__Self_Employed_No, cat__Self_Employed_Yes
	Property_Area (3)	cat__Property_Area_Rural, cat__Property_Area_Semiurban, cat__Property_Area_Urban

Because both categories of a two-valued attribute are kept, the two columns carry the same information. For logistic regression the L2 penalty handles this redundancy. In the SHAP display only the ten largest absolute contributions are shown, which is why the dashboard can show, for example, `cat__Gender_Male` without `cat__Gender_Female`.

4.2 FINANCIAL BEHAVIOUR CONSISTENCY ENGINE

The FBCE module (`fbce.py`) works on the raw application values entered in the form and not on the transformed features, so each output can be traced to what the applicant entered. Let I_A and I_C be the applicant and co-applicant incomes, $I_T = I_A + I_C$ the total income, L the loan amount, n the term in months, and let clamp(u) limit u to the range 0–100. The four formula-based component functions in the delivered file are:

$$S_{\text{income}} = \text{clamp}(100 \cdot (1 - \min((L / I_T) / 10, 1))) \quad (4.1)$$

$$S_{\text{debt}} = \text{clamp}(100 \cdot (1 - \min((L / I_T) / 8, 1))) \quad (4.2)$$

$$S_{\text{credit}} = 90 \text{ if credit history} \geq 1, \text{ otherwise } 30 \quad (4.3)$$

$$S_{\text{repay}} = \text{clamp}(100 \cdot (1 - \min((\text{EMI} / I_T) / 0.80, 1))) \quad (4.4)$$

Employment stability is 75 for "No", "not self employed" and "salaried"; 65 for "Yes" and "self employed"; and 55 for any other value. If an employment duration in years is supplied, the score is clamp(30 + 7 · min(years, 10)), which ranges from 30 to 100; the delivered dashboard does not supply this value. If $I_T \leq 0$, the income, debt and repayment components return 0, and if the loan amount or the term is not positive the repayment component also returns 0. The EMI in (4.4) is calculated with (2.10) at an annual rate of 10% unless another rate is passed to the function. The overall FBCE score is the unweighted mean of the five component scores, rounded to two decimals:

$$\text{FBCE} = (S_{\text{income}} + S_{\text{debt}} + S_{\text{employment}} + S_{\text{credit}} + S_{\text{repay}}) / 5 \quad (4.5)$$

Table 20. Consistency levels assigned by fbce.py

FBCE score	Consistency level (label in code)
80 or above	High Consistency
60 to below 80	Moderate Consistency
40 to below 60	Low Consistency
below 40	Very Low Consistency

Contradiction warnings are produced by three rules: a warning when total income is zero or unavailable; a warning that the requested loan is very high relative to income when L / I_T exceeds 8; and a warning that credit history is weak or unavailable when the credit-history value is below 1. The first two are mutually exclusive. Equal weights and all thresholds are project design choices and not banking standards, as the comments in the source code state. Figure 9 shows how the outputs are derived and Figure 10 the shape of the scoring functions.

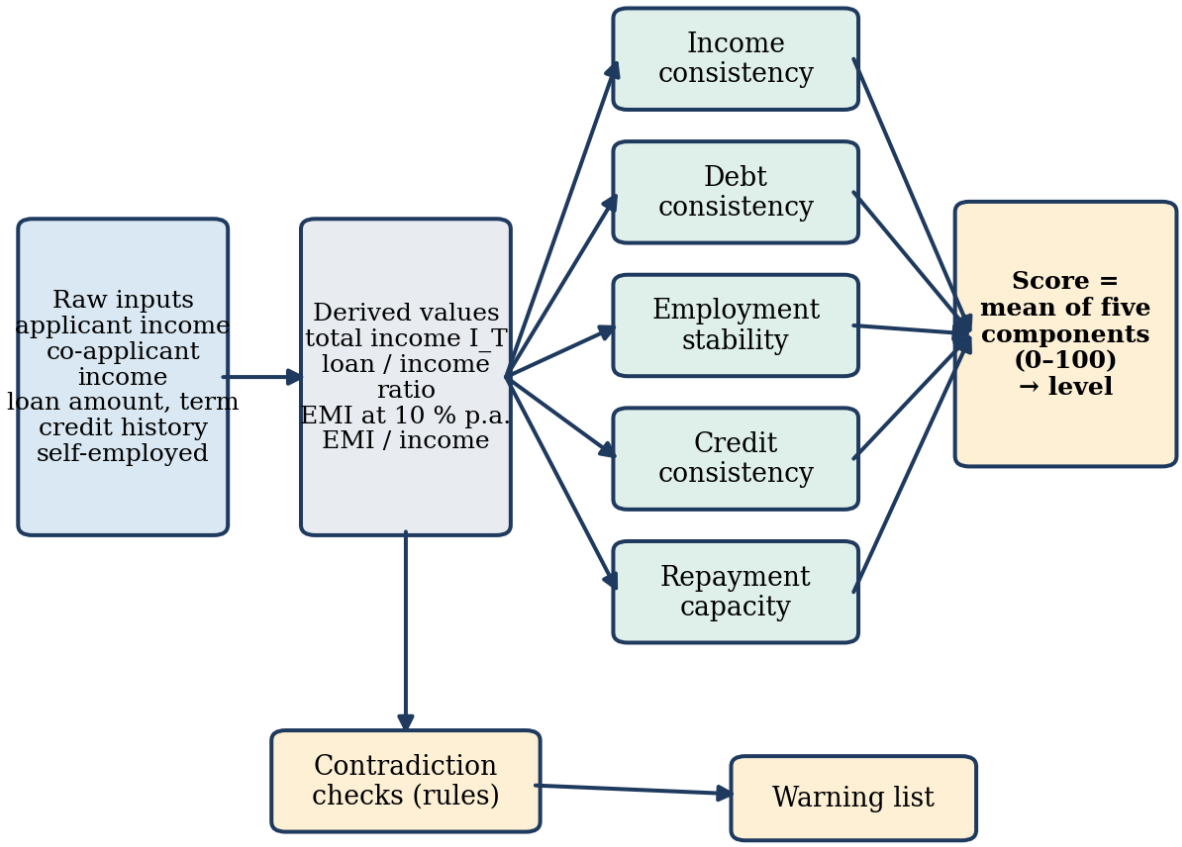


Figure 9. Derivation of the FBCE component scores, overall score and warnings (from fbce.py)

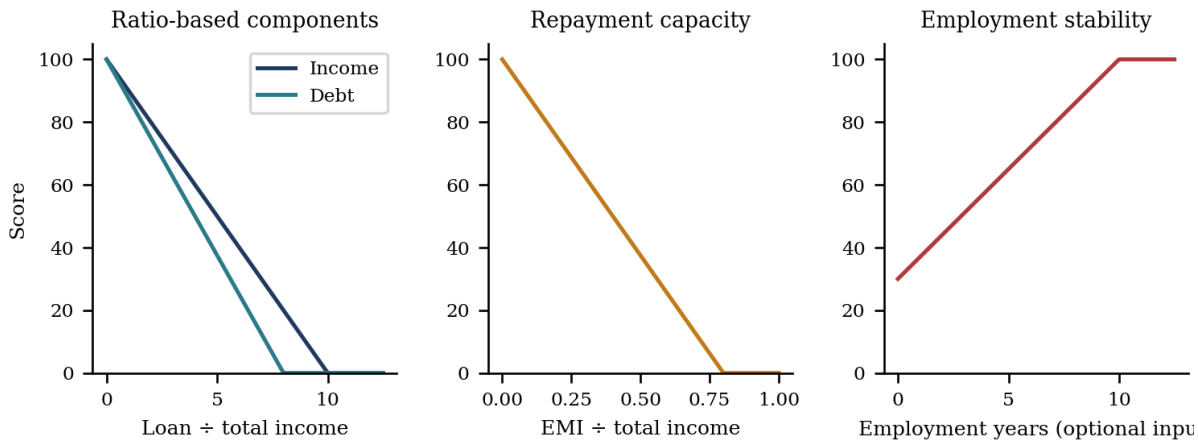


Figure 10. Behaviour of the scoring functions in fbce.py (computed by evaluating the code)

4.2.1 Worked example

The calculation of `fbce.py` is followed step by step for the case-study applicant of the earlier evaluation: applicant income 5000, co-applicant income 1500, loan amount 150, term 360 months, good credit history (1) and self-employed ("Yes"). The numbers were confirmed by running the code.

1. Total income: $I_T = 5000 + 1500 = 6500$.
2. Loan-to-income ratio: $L / I_T = 150 / 6500 = 0.02308$.

3. Income consistency: $100 \cdot (1 - 0.02308 / 10) = 99.77$.
4. Debt consistency: $100 \cdot (1 - 0.02308 / 8) = 99.71$.
5. Employment stability: self-employed "Yes" gives 65.00.
6. Credit consistency: credit history 1 gives 90.00.
7. Instalment: with $r = 0.10 / 12 = 0.008333$ and $n = 360$, $(1 + r)^n = 19.837$ and EMI $= 150 \cdot r \cdot (1 + r)^n / ((1 + r)^n - 1) = 1.316$ in the units of the loan amount.
8. Repayment capacity: EMI / $I_T = 0.00020$, so $100 \cdot (1 - 0.00020 / 0.80) = 99.97$.
9. Overall score: $(99.77 + 99.71 + 65.00 + 90.00 + 99.97) / 5 = 90.89$, which is at least 80 and therefore High Consistency.
10. Warnings: total income is positive, the ratio 0.02308 is below 8 and credit history is 1, so no rule fires.

The example shows how the score is built: three components are almost at 100, and the two categorical components (65 and 90) pull the average down. Changing the credit history to 0 would lower the credit component to 30 and add a warning (test T4 in Section 4.5.2).

4.2.2 Sensitivity analysis of the scoring functions

To understand how the scoring functions respond to their inputs, they were evaluated over ranges of values. All numbers below were obtained by running the delivered `fbce.py`.

Table 21. Income and debt consistency against the loan-to-income ratio

Loan ÷ income	Income consistency	Debt consistency
0	100.00	100.00
0.5	95.00	93.75
1	90.00	87.50
2	80.00	75.00
4	60.00	50.00
6	40.00	25.00
8	20.00	0.00
10	0.00	0.00
12	0.00	0.00

The income component falls linearly to 0 at a ratio of 10, and the debt component, which is steeper, reaches 0 at a ratio of 8. Both are exactly 100 when the loan is zero. In the Kaggle data the typical raw ratio, with the loan in thousands and income in rupees, is far below 1, so both components stay close to 100 for most applications.

Table 22. Repayment capacity against the EMI-to-income ratio

EMI ÷ income	Repayment capacity
0	100.00

EMI ÷ income	Repayment capacity
0.1	87.50
0.2	75.00
0.3	62.50
0.4	50.00
0.5	37.50
0.6	25.00
0.7	12.50
0.8	0.00

The repayment component falls linearly from 100 at no instalment to 0 when the instalment reaches 80% of income. The next two tables show how the term and the interest rate change the component when the loan is entered in rupees, for an applicant with a monthly income of 20,000 and a loan of 500,000.

Table 23. Effect of the loan term (income 20,000; loan 500,000; rate 10%)

Term (months)	EMI	Repayment capacity
60	10624	33.60
120	6608	58.70
180	5373	66.42
240	4825	69.84
360	4388	72.58
480	4246	73.46

Table 24. Effect of the annual interest rate (income 20,000; loan 500,000; term 240 months)

Annual rate (%)	Repayment capacity
0	86.98
5	79.38
10	69.84
15	58.85
20	46.91

A longer term lowers the instalment and raises the component, with diminishing returns beyond about 240 months, and a higher rate lowers it. These are the expected directions and confirm that the function behaves sensibly when the units are consistent.

Table 25. Score for a salaried-type applicant as the loan amount grows (income 5000; term 360; credit history 1; self-employed “No”)

Loan amount	Components (Inc, Debt, Emp, Cred, Rep)	Score	Level	Warn.
150	99.70, 99.62, 75.00, 90.00, 99.97	92.86	High	0
1000	98.00, 97.50, 75.00, 90.00, 99.78	92.06	High	0

Loan amount	Components (Inc, Debt, Emp, Cred, Rep)	Score	Level	Warn.
5000	90.00, 87.50, 75.00, 90.00, 98.90	88.28	High	0
10000	80.00, 75.00, 75.00, 90.00, 97.81	83.56	High	0
20000	60.00, 50.00, 75.00, 90.00, 95.61	74.12	Moderate	0
40000	20.00, 0.00, 75.00, 90.00, 91.22	55.24	Low	0

The score stays in the High band until the requested amount is of the same order as the income, because the loan and the income are compared without a unit conversion. Figure 11 shows the same behaviour as a map of the overall score over income and loan amount for a salaried-type applicant with good credit history. The score is almost constant over most of the realistic range and falls sharply only in the lower-right corner, where the loan is many times the income. A future version should convert units consistently and tie the tolerances to the observed distribution of the ratios.

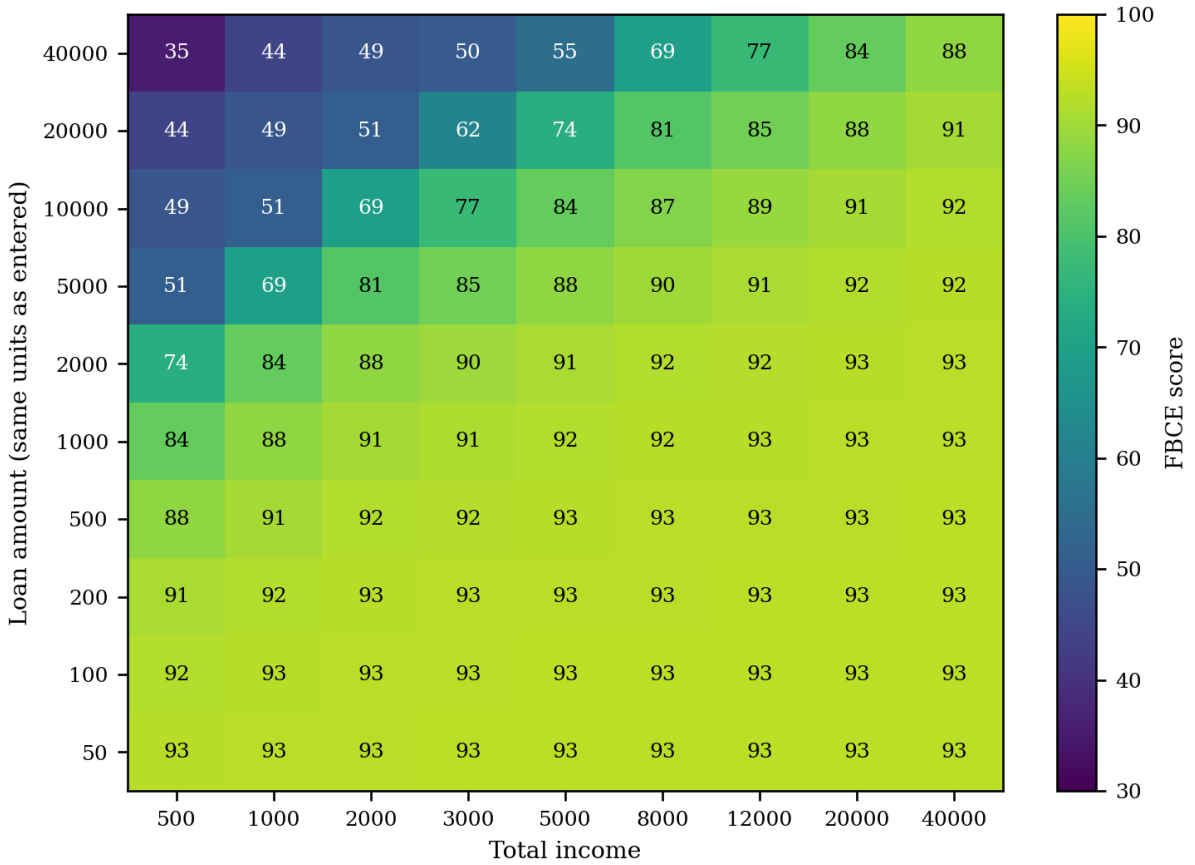


Figure 11. FBCE score as a function of total income and loan amount (computed by evaluating fbce.py; credit history 1, self-employed “No”, 360 months)

4.2.3 Purpose, inputs and outputs

The purpose of FBCE is to give the reviewer a transparent answer to the question “do the numbers in this application fit together?”, independently of what the classifier predicts. Table 26 lists the inputs and outputs of `calculate_fbce`. The function returns a dictionary and has no side effects.

Table 26. Inputs and outputs of calculate_fbce

Direction	Name	Type and meaning
Input	applicant_income, coapplicant_income	Numbers; monthly incomes entered in the form
Input	loan_amount	Number; in the unit of the dataset (thousands for the Kaggle file)
Input	loan_term_months	Number; term in months
Input	credit_history	Number; 1 or more means good, below 1 means weak or unavailable
Input	self_employed	Text; “Yes” or “No” in the dashboard
Input (optional)	annual_interest_rate, employment_years	Defaults 10.0 and None; not set by the dashboard
Output	scores	Dictionary of five component scores, 0 to 100
Output	fbce_score, risk_level	Mean rounded to two decimals; one of four level labels
Output	total_income, contradictions	Sum of incomes; list of warning texts

4.2.4 Variable definitions

Table 27. Symbols used in the FBCE equations

Symbol	Meaning	Unit	Name in the code
I_A, I_C	Applicant and co-applicant income	Currency per month	applicant_income, coapplicant_income
I_T	Total income $I_A + I_C$	Currency per month	total_income
L	Requested loan amount	Dataset unit (thousands in Kaggle data)	loan_amount
n	Loan term	Months	loan_term_months
r	Monthly interest rate (annual rate / 100) / 12	Fraction per month	r
EMI	Equated monthly instalment, (2.10)	Same unit as L	emi
S_{income} … S_{repay}	Component scores	Points, 0 to 100	keys of scores
FBCE	Mean of the five component scores	Points, 0 to 100	fbce_score

The only unit that matters for correctness is the pairing of L and I_T: both appear in the same ratio, so they must be in the same unit. The same holds for EMI and I_T in (4.4).

4.2.5 Constants, scaling and thresholds

Table 28. Constants of fbce.py (all project-defined)

Constant	Value	Used in	Status
Loan-to-income tolerance, income component	10	(4.1)	Heuristic
Loan-to-income tolerance, debt component	8	(4.2)	Heuristic (proxy for debt)
EMI-to-income tolerance	0.80	(4.4)	Heuristic
Credit scores	90 if history ≥ 1, else 30	(4.3)	Heuristic

Constant	Value	Used in	Status
Employment scores	75 (No), 65 (Yes), 55 (other)	Employment stability	Heuristic
Employment-years formula	30 + 7 · min(years, 10)	Optional input	Heuristic; unused by the dashboard
Default annual interest rate	10 %	EMI	Assumption, not a market rate
Level cut-offs	80, 60, 40	Table 20	Heuristic
High-loan warning ratio	8	Warning rule	Heuristic
Component weights	Equal (1/5)	(4.5)	Design choice

None of these constants has been estimated from repayment data, and none is an official banking standard; the comments in the source code say so. Their value is that they are explicit: a reviewer or a future team can change one number and see the effect, as in the sensitivity analysis of Section 4.2.2.

4.2.6 Units, saturation and extreme outputs

Three properties of the delivered scoring functions follow directly from the formulas and were confirmed by execution.

1. **Dependence on units.** Because the loan amount is divided by income without conversion, the same applicant can receive very different scores depending on the unit used to enter the loan. Table 29 shows the case-study applicant with the loan entered as in the dataset (150, in thousands) and with the same loan expressed in rupees (150000). The values are produced by running `fbce.py`; they illustrate the sensitivity of the function and do not describe a corrected version of the code.
2. **Saturation.** With the loan in thousands the two ratio components stay near 100 for almost all realistic applications, so the score is decided by the two categorical components.
3. **Bounded output.** The overall score cannot reach 100 or 0 for inputs the dashboard can produce. Table 30 gives the range.

Table 29. Illustrative sensitivity of the score to the unit of the loan amount (computed by evaluating fbce.py; not experimental data)

Scenario (applicant, co-applicant, loan, term, credit, self-employed)	Components (Inc, Debt, Emp, Cred, Rep)	Score	Level	Warn.
Loan as entered (thousands) (5000, 1500, 150, 360, 1, Yes)	99.77, 99.71, 65.00, 90.00, 99.97	90.89	High	0
Same loan converted to rupees (5000, 1500, 150000, 360, 1, Yes)	0.00, 0.00, 65.00, 90.00, 74.69	45.94	Low	1
Rupee loan, 20 lakh (2,000,000) (5000, 1500, 2000000, 360, 1, Yes)	0.00, 0.00, 65.00, 90.00, 0.00	31.00	Very Low	1
Rupee loan, 20 lakh, credit history 0 (5000, 1500, 2000000, 360, 0, Yes)	0.00, 0.00, 65.00, 30.00, 0.00	19.00	Very Low	2

Table 30. Range of the FBCE score (computed by evaluating fbce.py)

Case	Value	Remark
Highest score reachable from the dashboard (self-employed “No”, good credit, negligible loan)	93.0	Components 100, 100, 75, 90, 100
Lowest score reachable from the dashboard (self-employed “Yes”, weak credit, no affordability)	19.0	Components 0, 0, 65, 30, 0
Highest score if employment years = 10 were supplied	98.0	Not offered by the dashboard
Best score with weak credit history	81.0	Level still “High Consistency”; one warning

Table 30 has two consequences for interpretation. A score of 93 is the maximum, so scores in the nineties are not “almost perfect” but “at the ceiling”. And a weak credit history on its own can never push the level below High Consistency, because even then the remaining components can reach 81. The warning list is therefore an essential part of the FBCE output. These properties are recorded here as documented behaviour of the delivered code; they are not corrected, and no fix has been verified.

4.3 EXPLAINABILITY LAYER AS IMPLEMENTED

Explanations are produced in `app.py` after the prediction. The code takes the preprocessor and the estimator from the saved pipeline, transforms the one-row application and obtains the feature names from the preprocessor. Two branches are implemented:

- **Tree models** (random forest, XGBoost, LightGBM): a SHAP tree explainer is created from the estimator, and the values for the approval class are extracted, whether the library returns a list of arrays, a three-dimensional array or a two-dimensional array.
- **Other models** (logistic regression): a SHAP linear explainer is created from the estimator with a background consisting of a single row of zeros, and the values of the one application are used.

As explained in Section 2.5.5, the zero background means that for logistic regression each contribution equals the coefficient multiplied by the transformed feature value, in log-odds, relative to an all-zero input and not relative to the average applicant. The scale of the tree-model contributions depends on the model and the SHAP library defaults. The explanation is therefore a feature-attribution of the model output; it is not a causal explanation of why a lender would approve or reject an applicant.

The ten features with the largest absolute contribution are kept. The dashboard shows them in a table, in a horizontal bar chart and in two lists, “Positive contributors” and “Negative contributors”, with the signed value to four decimals. When no contribution of one sign is among the ten, the corresponding list states so. If SHAP cannot be computed for a model or input, the dashboard reports the error text instead of stopping.

The SHAP library is not installed in the environment used to prepare this report, so this branch of the code was reviewed but not executed here; the SHAP figures in this report come from dashboard runs of the earlier phase (Section 4.5.5).

4.3.1 Step-by-step workflow in the dashboard

1. The submitted form values are assembled into a one-row table x.
2. The preprocessor of the saved pipeline transforms x into the vector z of twenty inputs.
3. The estimator predicts from z; the label and the probability are shown.
4. The model name decides the branch: tree models use the tree explainer; logistic regression uses the linear explainer with a zero background.
5. The explainer returns one contribution per transformed feature; the values of the approval class are extracted.
6. The ten contributions with the largest absolute value are kept and sorted.
7. They are shown as a table, a horizontal bar chart and two lists, one for positive and one for negative contributions.

4.3.2 How to read the contributions

A positive contribution raised the model output for this application relative to the reference input, and a negative contribution lowered it. For logistic regression the output is the log-odds of approval, so the contributions add up on that scale, and exp(φ) is the factor by which a feature multiplies the odds of approval ((2.12)). The length of a bar says how much the model used a feature for this application, not how important the feature is across the whole population. The names carry the prefixes `num__` and `cat__` of the column transformer, and a contribution such as `cat__Gender_Male` refers to the one-hot column for that category.

4.3.3 Illustrative contributions of the synthetic-data model

The SHAP library is not installed in the environment used for this report, so the dashboard's explanation code could not be run. For logistic regression the linear explainer with a zero background has a closed form, (2.9) with $b_j = 0$, which can be evaluated directly. Table 31 and Figure 12 apply it to the demonstration input of Section 4.5.6 (applicant income 40000, no co-applicant income, loan 170, 360 months, good credit, graduate, self-employed, married male, no dependents, urban) with the logistic regression trained by the delivered pipeline on the synthetic dataset. The values are computed by the closed-form expression and not by the SHAP library; they are an illustration of the method and say nothing about real applicants.

Table 31. Ten largest linear contributions for the demonstration input (synthetic-data logistic regression; closed form, log-odds)

Feature	Contribution φ	exp(φ)	Direction
num__ApplicantIncome	+1.1779	3.25	raises approval

Feature	Contribution φ	exp(φ)	Direction
num__Credit_History	+0.8730	2.39	raises approval
cat__Education_Graduate	+0.3933	1.48	raises approval
num__LoanAmount	-0.3066	0.74	lowers approval
num__CoapplicantIncome	+0.2202	1.25	raises approval
cat__Gender_Male	-0.1624	0.85	lowers approval
cat__Married_Yes	+0.1367	1.15	raises approval
cat__Property_Area_Urban	-0.0600	0.94	lowers approval
cat__Dependents_0	+0.0589	1.06	raises approval
num__Loan_Amount_Term	-0.0457	0.96	lowers approval

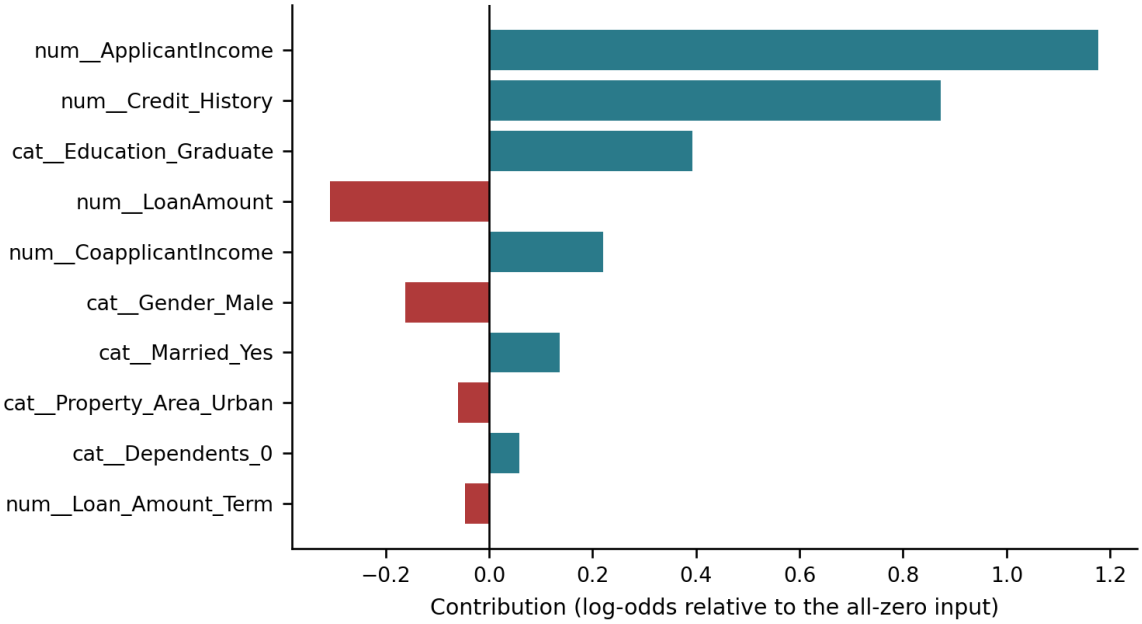


Figure 12. Ten largest linear contributions for the demonstration input (synthetic-data model; closed form)

The table and the figure show the expected pattern. Applicant income (+1.1779) and credit history (+0.8730) raise the log-odds most, graduate education adds +0.3933, and the loan amount lowers it (−0.3066). Male gender contributes −0.1624. The probability of this model for the input is 0.7648, which agrees with the 76.5% displayed in the dashboard screenshot of Section 5.5, and the contributions agree with the values shown there to the four decimals displayed. The same model also reproduces the case study of Section 4.5.5. This agreement shows that the screenshots and the case study were produced by a model trained on the synthetic dataset and not on the 614-record Kaggle data.

4.3.4 Relationship between model output and explanation

Local accuracy (Section 2.5.11) can be verified for this example. The intercept of the model is -1.0933, the sum of all twenty contributions is 2.2724, and their total 1.1790 equals the model's log-odds 1.1790; the numerical difference is 4.4e-16. The ten displayed contributions sum to

2.2851, so the remaining ten features contribute -0.0128 together. The dashboard shows only the ten largest values and does not display the intercept, so a reviewer cannot reconstruct the total log-odds from the display alone.

4.3.5 Limits of interpretation

- **Reference input.** With a zero background, contributions are measured from an all-zero vector. For one-hot columns this means "category absent", and the pair of columns of a two-valued attribute is not symmetric in the display.
- **Model-specific scale.** Contributions of tree models can be on a different scale from those of logistic regression, so numbers should not be compared across models.
- **Not causal.** A contribution describes how the fitted model responds to the input; it does not say that changing the input would change a real lending decision [20].
- **Truncation.** Only ten contributions are shown, and the remainder is not summarised.
- **Dependence on training data.** A different dataset or a retrained model changes the weights and therefore every contribution. The gender contribution of the synthetic-data model is an example: gender plays no role in the rule that generates the synthetic target (Section 4.1.4), so a non-zero weight is the fit to random variation in this sample.
- **Explanation is not justification.** A tidy explanation of a prediction does not show that the prediction is correct or fair.

4.4 IMPLEMENTATION OF THE MODULES

The complete source code of the delivered files is reproduced without change in Section 5.3, and Table 14 lists the role of each file.

4.4.1 Training module (train_model.py)

The script defines four functions. `load_data()` checks that the dataset exists, finds the target column, maps the target to 1/0, removes Loan_ID and unnamed columns and returns the features *X* and the target *y*; a missing file raises an error that tells the user to save a dataset as `data/loan_data.csv`. `build_preprocessor(X)` creates the numeric and categorical pipelines described in Section 4.1.5. `build_models()` returns a dictionary of the four classifiers. `main()` splits the data, trains each model inside a pipeline, computes the metrics, ranks the models, saves the best one and prints the comparison table.

The saved file is a dictionary with three entries: the fitted pipeline (`model`), its name (`model_name`) and the feature column names (`feature_columns`). The dashboard can therefore load a single file and apply preprocessing and prediction in one call. When a model has no `predict_proba` method, or when the test set contains a single class, the ROC AUC is stored as NaN.

4.4.2 FBCE module (fbce.py)

The module contains the helper functions `clamp`, `income_consistency`, `debt_consistency`, `employment_stability`, `credit_consistency` and `repayment_capacity`, and the main function `calculate_fbce`. The main function accepts the raw inputs, adds the two incomes, evaluates the five components and returns a dictionary with the component scores, the overall score, the consistency level, the total income and the list of contradiction warnings. Comments in the code state that the debt component is a proxy because the public dataset has no debt column, that equal weights are a design choice, and that the scale is an academic prototype scale.

An important detail concerns units. The loan amount is divided directly by the total income. In the Kaggle dataset the loan amount is in thousands and incomes are in rupees per month, so this ratio is about a thousand times smaller than a ratio computed in common units. The form in `app.py` even instructs the user to enter the loan amount in the same unit as the training dataset. Section 4.5.2 shows how this affects the scores.

4.4.3 Data generator (generate_demo_data.py)

The generator fixes the NumPy random seed to 42 and produces 800 rows. It writes the file to `data/loan_data.csv` relative to the script's own folder and prints the first five rows. It does not create the data folder, so the folder must exist before the script is run; the folder structure in the README includes it.

4.4.4 Dashboard (app.py)

The Streamlit dashboard is organised as follows. It configures a wide page titled "FBCE - Explainable Loan Prediction", loads the saved model bundle once through a cached function and, if the model file does not exist, shows a warning with the training command and stops. The sidebar describes the project, states that the FBCE score is a project-defined consistency indicator and not a banking credit score, and names the loaded model.

The form "Applicant Financial Details" has eleven fields in three columns: applicant income (default 5000), co-applicant income (default 1500), loan amount (default 150), loan term in months (default 360), credit history (Good / Available = 1.0 or Weak / Unavailable = 0.0), gender, marital status, dependents, education, self-employment and property area. When the "Predict Loan" button is pressed, the values are assembled into a one-row table whose column names match the Kaggle format, and the model returns the predicted class and the approval probability. `calculate_fbce` is called with the incomes, loan amount, term, credit history and self-employment value. The results are shown as three metrics (prediction, approval probability and FBCE score), the consistency level, a table and bar chart of the five components, the contradiction warnings (or a success message when none is raised) and the total income, then the SHAP section (Section 4.3) and a decision summary. The summary states

that the model predicts approval or rejection and asks the reviewer to examine the FBCE score and the SHAP factors; a caption states that the prototype is for academic demonstration only and does not provide a real banking decision. Prediction errors are caught and displayed together with a hint to use the Kaggle feature names.

4.4.5 Setup and execution

The README gives the following sequence: install Python 3.10 or 3.11; install the libraries with `pip install -r requirements.txt`; place the dataset at `data/loan_data.csv`; train the models with `python train_model.py`; and start the dashboard with `streamlit run app.py`. If the dataset is not available, `python generate_demo_data.py` followed by the training and dashboard commands runs the pipeline on synthetic data.

4.4.6 Function reference

Table 32, Table 33 and Table 34 summarise the functions of the modules with their inputs and behaviour as visible in the source code.

Table 32. Functions in fbce.py

Function	Inputs	Returns / behaviour
clamp(value, low, high)	number, bounds 0 and 100	Limits a value to the range 0–100.
income_consistency(total_income, loan_amount)	total income, loan amount	0 if income ≤ 0; otherwise 100·(1 − min(ratio/10, 1)).
debt_consistency(total_income, loan_amount)	total income, loan amount	0 if income ≤ 0; otherwise 100·(1 − min(ratio/8, 1)); the loan is a proxy for debt.
employment_stability(self_employed, employment_years)	category text, optional years	30 + 7·min(years, 10) if years are given; else 75 (No / salaried), 65 (Yes / self employed), 55 (other).
credit_consistency(credit_history)	credit-history value	90 if the value is at least 1; otherwise 30 (also 30 if conversion to a number fails).
repayment_capacity(total_income, loan_amount, loan_term_months, annual_interest_rate)	amounts, months, rate (default 10)	0 if income, loan or term ≤ 0; otherwise 100·(1 − min((EMI/income)/0.80, 1)); EMI = loan/n when the rate is 0.
calculate_fbce(...)	incomes, loan, term, credit history, self-employment, rate, optional years	Dictionary with scores, fbce_score (2 decimals), risk_level, total_income and the list of contradiction warnings.

Table 33. Functions in train_model.py

Function	Behaviour
load_data()	Reads data/loan_data.csv, raises FileNotFoundError if absent, finds the target column, maps it to 1/0, drops unmapped rows, Loan_ID and “Unnamed” columns; returns X and y.
build_preprocessor(X)	Splits columns into categorical (object, category, bool) and numeric; builds median-impute + StandardScaler and most-frequent-impute + OneHotEncoder pipelines inside a ColumnTransformer with remainder dropped.
build_models()	Returns a dictionary with the four configured classifiers (Table 18).
main()	Stratified 80/20 split (seed 42); fits a pipeline per model; computes accuracy, precision, recall, F1 and ROC AUC; sorts by F1 then accuracy; saves the best pipeline with its name and feature columns; writes model_results.csv; prints the comparison.

Table 34. Structure of app.py

Part	Behaviour
load_model() (cached)	Loads models/fbce_model.pkl with joblib; returns None if the file is missing.
Sidebar	Project description, FBCE disclaimer and the name of the loaded model.
Form "loan_form"	Eleven input widgets and the "Predict Loan" submit button.
Prediction block	Builds the one-row DataFrame, calls predict and predict_proba, then calculate_fbce.
Result panels	Metrics, consistency level, component table and chart, contradiction warnings, total income.
SHAP block	Tree or linear explainer, top-ten table, bar chart, positive and negative lists; errors are displayed.
Decision summary	Approval or rejection message and the academic-demonstration caption.

4.4.7 Design observations on the source code

Reading the delivered code closely gives the following observations. They are recorded so that the report describes the implementation accurately and so that they can be addressed in future work.

- **Leakage-safe design.** The preprocessing is inside the pipeline of every model, so statistics are learned from the training split only.
- **Class imbalance handling.** Logistic regression and random forest use balanced class weights; XGBoost and LightGBM do not, which is one reason their precision–recall trade-off differs.
- **Single split.** The script evaluates on one 80/20 split, so differences of a point or two between models are within the variation seen in the five-fold cross-validation of the earlier phase.
- **Model choice by F1.** Ranking by F1 favours models with high recall; given the low specificity noted in the earlier evaluation, a reviewer may prefer a different criterion.
- **Protected attributes remain features.** Only the target and Loan_ID are removed, so Gender and Married are used by the models.
- **Output folder handling.** The models folder is created if it is missing, but the data folder is not, so it must exist before `generate_demo_data.py` is run.
- **FBCE units and validation.** The loan-to-income ratio mixes units, and the warning check converts the credit-history value with `float()` without a guard, so a non-numeric value would raise an error there; the dashboard passes a numeric value.
- **SHAP reference input.** The linear explainer uses an all-zero background, so the contributions are relative to an all-zero input rather than to the average applicant (Section 4.3).
- **Unpinned dependencies.** `requirements.txt` contains no version numbers.

4.4.8 Code walkthroughs

This section explains selected excerpts of the delivered code line by line. The excerpts are printed exactly as in the source files with their original line numbers; the complete files are in Section 5.3.

4.4.9 fbce.py: repayment capacity and the warning rules

```
 43  def repayment_capacity(total_income, loan_amount, loan_term_months,
annual_interest_rate=10.0):
 44      if total_income <= 0 or loan_amount <= 0 or loan_term_months <= 0:
 45          return 0.0
 46
 47      r = (annual_interest_rate / 100.0) / 12.0
 48      n = float(loan_term_months)
 49
 50      if r == 0:
 51          emi = loan_amount / n
 52      else:
 53          emi = loan_amount * r * ((1 + r) ** n) / (((1 + r) ** n) - 1)
 54
 55      emi_ratio = emi / total_income
 56
 57      # Academic prototype scale: lower EMI-to-income ratio is more consistent.
 58      return clamp(100 * (1 - min(emi_ratio / 0.80, 1.0)))
```

- Lines 44–45: the function returns 0 when income, loan or term is not positive, which avoids a division by zero in the instalment formula.
- Lines 47–48: the monthly rate r is derived from the annual percentage and the term n is converted to a float.
- Lines 50–53: with a zero rate the instalment is the loan divided by the term; otherwise (2.10) is applied.
- Lines 55–58: the ratio of the instalment to income is scaled by 0.80 and limited to 1, then turned into a score between 0 and 100 by `clamp`.

```
 83      # Equal weights are a project-design choice, not a banking standard.
 84      fbce_score = sum(scores.values()) / len(scores)
 85
 86      if fbce_score >= 80:
 87          risk = "High Consistency"
 88      elif fbce_score >= 60:
 89          risk = "Moderate Consistency"
 90      elif fbce_score >= 40:
 91          risk = "Low Consistency"
 92      else:
 93          risk = "Very Low Consistency"
```

The mean of the five scores is computed on the first line of this excerpt, and the level labels follow Table 20. The comment in the code states that equal weights are a design choice and not a banking standard.

```
 95      contradictions = []
 96
 97      if total_income <= 0:
 98          contradictions.append("Reported total income is zero or unavailable.")
 99      elif loan_amount / total_income > 8:
100          contradictions.append("Requested loan is very high relative to reported
income.")
101
```

```
102      if float(credit_history) < 1:
103          contradictions.append("Credit history is marked as weak/unavailable.")
```

- The first rule fires when the total income is not positive; otherwise (`elif`) the second rule fires when the loan-to-income ratio exceeds 8. The two cannot both fire.
- The third rule converts the credit history with `float()` and fires when it is below 1. This conversion has no guard, so a non-numeric value raises `ValueError` (test V2).

4.4.10 train_model.py: target preparation, preprocessing and ranking

```
44      # Convert common Y/N target to 1/0.
45      y = df[target].astype(str).str.strip().str.upper().map({
46          "Y": 1, "YES": 1, "APPROVED": 1, "1": 1,
47          "N": 0, "NO": 0, "REJECTED": 0, "0": 0
48      })
49
50      valid = y.notna()
51      df = df.loc[valid].copy()
52      y = y.loc[valid].astype(int)
```

- The target column is converted to text, stripped and upper-cased, then mapped through a dictionary; any value outside the dictionary becomes missing.
- `valid` marks the rows with a mapped target; the other rows are dropped without a message (test P6).
- The labels are converted to integers for the classifiers.

```
66  def build_preprocessor(X):
67      categorical = X.select_dtypes(include=["object", "category",
"bool"]).columns.tolist()
68      numerical = [c for c in X.columns if c not in categorical]
69
70      num_pipe = Pipeline([
71          ("imputer", SimpleImputer(strategy="median")),
72          ("scaler", StandardScaler())
73      ])
74
75      cat_pipe = Pipeline([
76          ("imputer", SimpleImputer(strategy="most_frequent")),
77          ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
78      ])
79
80      preprocessor = ColumnTransformer([
81          ("num", num_pipe, numerical),
82          ("cat", cat_pipe, categorical)
83      ], remainder="drop")
84
85      return preprocessor
```

- Categorical columns are those of type object, category or bool; all remaining columns are numeric, so `Credit_History` is numeric.
- The numeric pipeline imputes the median and standardises ((2.7)); the categorical pipeline imputes the most frequent value and one-hot encodes with `handle_unknown="ignore"`.
- `remainder="drop"` discards any column that is neither numeric nor categorical.

```
159      results_df = pd.DataFrame(results).sort_values(
160          ["F1", "Accuracy"], ascending=False
161      )
162
```

```
163     best_name = results_df.iloc[0]["Model"]
164     best_model = trained[best_name]
165
166     MODEL_PATH.parent.mkdir(exist_ok=True)
167     joblib.dump(
168         {
169             "model": best_model,
170             "model_name": best_name,
171             "feature_columns": X.columns.tolist()
172         },
173         MODEL_PATH
174     )
175
176     results_df.to_csv(RESULT_PATH, index=False)
```

- The results are sorted by F1 and then accuracy, so the first row is the saved model.
- `MODEL_PATH.parent.mkdir(exist_ok=True)` creates the models folder if needed, and `joblib.dump` writes a dictionary with the pipeline, its name and the feature columns.
- The comparison table is written to `model_results.csv`.

4.4.11 generate_demo_data.py: the synthetic target

```
23  score = (
24      1.6 * credit
25      + 0.00002 * income
26      + 0.00001 * co_income
27      - 0.0035 * loan
28      + 0.25 * (education == "Graduate")
29      + 0.10 * (property_area == "Semiurban")
30      + np.random.normal(0, 0.7, N)
31  )
32  prob = 1 / (1 + np.exp(-score + 2.5))
33  status = (prob > 0.5).astype(int)
```

The target is a deterministic threshold of a score that depends on credit history, the two incomes, the loan amount, graduate education and semi-urban property, plus Gaussian noise. Gender, marital status, dependents and self-employment do not enter the score. Any weight that a model later assigns to them is therefore a property of the random sample. The threshold 0.5 on a sigmoid shifted by 2.5 produces the low approval rate of 7.5%.

4.4.12 app.py: loading, inference and the linear explanation

```
27  @st.cache_resource
28  def load_model():
29      if not MODEL_PATH.exists():
30          return None
31      return joblib.load(MODEL_PATH)
32
33  bundle = load_model()
```

The model is loaded once and cached by Streamlit. If the file does not exist the function returns `None` and the page shows a warning and stops. Because `joblib.load` unpickles the file, only a trusted model file should be placed in the models folder (Section 4.8).

```
179                 # Linear-model explanation.
180                 background = np.zeros((1, transformed.shape[1]))
181                 explainer = shap.LinearExplainer(estimator, background)
182                 shap_values = explainer.shap_values(transformed)
183                 values = np.asarray(shap_values)
184                 if values.ndim == 2:
```

```
185                     values = values[0]
```

- A background of one row of zeros with as many columns as the transformed input defines the reference input of (2.9).
- The linear explainer is created from the fitted estimator and applied to the transformed application.
- The returned array is flattened to the one row of contributions that the table and chart use.

4.4.13 Module interfaces

Table 35. Inputs, outputs, side effects and errors of the modules

Module	Inputs	Outputs	Side effects	Errors raised or shown
train_model.py	data/loan_data.csv	Printed comparison; two files	Creates models/ if missing	FileNotFoundError, ValueError
fbce.py	Eight arguments	Result dictionary	None	ValueError, TypeError on bad types
generate_demo_data.py	None (seed 42)	data/loan_data.csv	Overwrites the CSV	Fails if data/ is missing
app.py	Form values; model file	Browser panels	Reads the model file	Warnings and error boxes

4.5 TESTING, RESULTS AND DISCUSSION

This section reports the evidence that exists for the prototype. Results that were executed while preparing this report are kept apart from results reported from the project's earlier evaluation phase, because the latter cannot be reproduced from the delivered files. Nothing in this section is presented as evidence of real-world performance unless it is explicitly identified as such.

4.5.1 Evidence overview

Table 36. Evidence used in this chapter and its status

Evidence	Status	Basis
Syntax check of the four Python scripts	Executed for this report	py_compile reported no errors
FBCE test cases T1–T13 and the sensitivity tables	Executed for this report	Delivered fbce.py run directly
Synthetic dataset generation	Executed for this report	Delivered generate_demo_data.py, seed 42
Training pipeline on synthetic data (logistic regression and random forest)	Executed for this report	Functions of the delivered train_model.py; XGBoost and LightGBM libraries not installed in the environment, so these two models were not run
Dashboard (app.py) and SHAP code	Reviewed, not executed	Streamlit and SHAP not installed in the environment
Five-fold cross-validation results (Experiment A)	Reported from the earlier evaluation phase	Script or notebook not among the delivered files

Evidence	Status	Basis
Output of train_model.py on the real dataset (Experiment B)	Not available	Real dataset and models/model_results.csv not among the delivered files
Case-study and screenshot probability and SHAP values	Reproduced for this report	The logistic regression trained by the delivered pipeline on the synthetic dataset gives 50.09% and 76.5% and the listed contributions
Screenshot FBCE values and extra panels	Recapture required	Earlier dashboard build differs from the delivered app.py and fbce.py

4.5.2 Testing of the FBCE module

The FBCE functions are deterministic, so they can be tested by evaluating them on chosen inputs. Table 37 lists thirteen test cases that were executed against the delivered `fbce.py`; every output in the table is the value returned by the code. The default 10% interest rate was used unless stated. The inputs are applicant income, co-applicant income, loan amount, term in months, credit history and self-employment.

Table 37. FBCE test cases executed on the delivered fbce.py

ID	Scenario and inputs	Components (Inc, Debt, Emp, Cred, Rep)	Score	Level	Warn.
T1	Case-study applicant (5000, 1500, 150, 360, 1, Yes)	99.77, 99.71, 65.00, 90.00, 99.97	90.89	High	none
T2	Dashboard demonstration input (40000, 0, 170, 360, 1, Yes)	99.96, 99.95, 65.00, 90.00, 100.00	90.98	High	none
T3	Salaried applicant, good credit (6000, 2000, 120, 360, 1, No)	99.85, 99.81, 75.00, 90.00, 99.98	92.93	High	none
T4	Credit history = 0 (6000, 2000, 120, 360, 0, No)	99.85, 99.81, 75.00, 30.00, 99.98	80.93	High	1
T5	Very large loan relative to income (ratio 20) (1000, 0, 20000, 360, 1, No)	0.00, 0.00, 75.00, 90.00, 78.06	48.61	Low	1
T6	Loan-to-income ratio just above 8 (1000, 0, 8100, 360, 1, No)	19.00, 0.00, 75.00, 90.00, 91.11	55.02	Low	1
T7	Zero total income (0, 0, 100, 360, 1, No)	0.00, 0.00, 75.00, 90.00, 0.00	33.00	Very Low	1
T8	Short term (60 months) (2000, 0, 400, 60, 1, No)	98.00, 97.50, 75.00, 90.00, 99.47	91.99	High	none
T9	Employment years supplied (10) (5000, 0, 100, 240, 1, No), years=10	99.80, 99.75, 100.00, 90.00, 99.98	97.91	High	none
T10	Employment years supplied (0) (5000, 0, 100, 240, 1, No), years=0	99.80, 99.75, 30.00, 90.00, 99.98	83.91	High	none
T11	Interest rate 0 % (5000, 0, 100, 240, 1, No), rate=0	99.80, 99.75, 75.00, 90.00, 99.99	92.91	High	none
T12	Unknown employment label (5000, 0, 100, 240, 1, Unknown)	99.80, 99.75, 55.00, 90.00, 99.98	88.91	High	none
T13	Low-consistency stress case (900, 0, 9000, 60, 0, Yes)	0.00, 0.00, 65.00, 30.00, 73.44	33.69	Very Low	2

The results confirm the intended behaviour of the code:

- **Reproducibility of the case study (T1).** The code returns an overall score of 90.89 with components 99.77, 99.71, 65.00, 90.00 and 99.97, exactly the values reported for the case study of the earlier evaluation.
- **Warnings fire as designed.** T5, T6 and T13 trigger the high-loan rule (ratio above 8); T4 and T13 trigger the weak-credit rule; T7 triggers the zero-income rule and drives the income, debt and repayment components to 0.
- **Employment rules.** Salaried applicants score 75, self-employed 65 and unknown values 55 (T3, T1, T12); supplying employment years replaces the category rule, giving 100 for ten years (T9) and 30 for zero years (T10).
- **Interest-rate handling.** With a 0% rate the instalment is the loan divided by the term (T11) and the function returns a valid score without a division error.
- **Boundary behaviour.** At a loan-to-income ratio just above 8 (T6) the debt component is already 0 and the warning fires, whereas the income component is still 19.

The tests also expose three limitations of the delivered version:

- **Mixed units.** The loan amount is compared with income without converting thousands to rupees. For realistic inputs the ratio-based components therefore stay close to 100 (T1, T2, T8), and the overall score is dominated by the two categorical components.
- **A weak credit history does not lower the level much.** T4 differs from T3 only in credit history, yet the score falls from 92.93 to 80.93 and the level stays High Consistency, although a warning is shown. The warning list is therefore important for interpreting the score.
- **Input validation.** `calculate_fbce` converts the credit-history value with `float()` in the warning check without a guard, so a non-numeric value would raise an error; the dashboard passes a numeric value.

4.5.3 Experiment B: the training script on synthetic data

To confirm that the training pipeline runs end to end, the synthetic dataset of Section 4.1.4 (800 records, 60 approved) was passed through the delivered functions `load_data()`, `build_preprocessor()` and `build_models()` with the split of the script (test size 0.20, seed 42, stratified), which gives 640 training and 160 test records, of which 12 test records are approved. The environment used for this report has no XGBoost or LightGBM library, so only logistic regression and random forest were fitted; the two boosted models were **not** executed. Table 38 lists the metrics obtained.

Table 38. Results of the training pipeline on the synthetic dataset (160 test records, 12 approved; logistic regression and random forest only)

Model	Accuracy	Precision	Recall	F1	ROC AUC	TN / FP / FN / TP
Logistic Regression	0.6250	0.1250	0.6667	0.2105	0.7337	92 / 56 / 4 / 8
Random Forest	0.9187	0.0000	0.0000	0.0000	0.6959	147 / 1 / 12 / 0
XGBoost	not executed	–	–	–	–	–
LightGBM	not executed	–	–	–	–	–

These numbers demonstrate only that the pipeline executes. They are **not** evidence of model quality, because the target of the synthetic data is produced by an arbitrary scoring rule, only 7.5% of the records are approved and the test set contains just twelve approved records. The random forest, for example, reaches an accuracy of 91.9% only because it predicts rejection for almost every record (no approved record is found), whereas the balanced logistic regression finds eight of the twelve approved records at the price of many false approvals. By the ranking rule of the script (F1, then accuracy), logistic regression would be the saved model on this dataset, but this ordering must not be generalised to real data.

4.5.4 Experiment A: five-fold cross-validation reported from the earlier phase

Table 39 lists the fold means recorded in the earlier evaluation phase for the four classifiers, obtained with stratified five-fold cross-validation (shuffled, seed 42) on the 614-record Loan Prediction dataset, with approval as the positive class. Figure 13 compares accuracy, F1-score and specificity. The script or notebook that produced these numbers is not among the delivered files, so the numbers are reported as recorded and could not be reproduced for this report. This set-up differs from the delivered `train_model.py` in the evaluation scheme (five folds instead of one split) and in the XGBoost and LightGBM settings (Section 4.1.6), so Experiments A and B must not be compared or combined.

Table 39. Stratified five-fold cross-validation results, fold means (Experiment A, earlier evaluation phase)

Model	Acc. (%)	Prec. (%)	Rec. (%)	Spec. (%)	F1 (%)	AUC
LR (selected)	80.30	79.01	97.16	43.28	87.14	0.751
RF	79.15	78.84	95.26	43.81	86.26	0.749
XGBoost	78.18	79.53	91.94	47.98	85.25	0.752
LightGBM	75.40	79.00	87.43	48.97	82.97	0.740

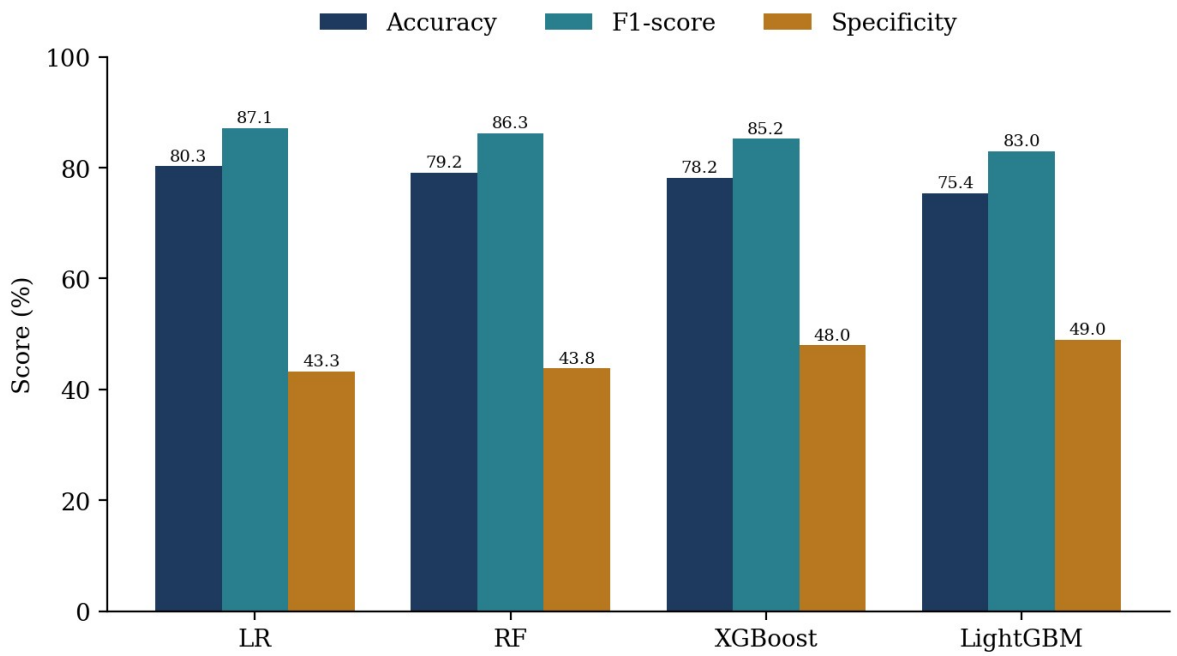


Figure 13. Accuracy, F1-score and specificity of the four classifiers (plotted from Table 39)

In this evaluation logistic regression achieved the highest accuracy (80.30%), recall (97.16%) and F1-score (87.14%), and its AUC of 0.751 practically equalled those of XGBoost (0.752) and random forest (0.749). The earlier phase also recorded fold standard deviations of 1.1 to 5.3 percentage points, which exceed most gaps between the models, so the four classifiers are close on data of this size. This is in line with the logistic-regression result reported in [1] for a different dataset. Logistic regression was selected for its F1-score and because its SHAP values are cheap to compute and have a direct reading.

Specificity is less comfortable. No model rejected even half of the historically rejected applications (43.28% for logistic regression to 48.97% for LightGBM), and the boosted models gained a little specificity only by losing recall. The earlier phase also recorded that, across the test folds, logistic regression approved 109 of the 192 rejected applications and that 89.0% of those had a positive credit flag, so the classifiers lean heavily on credit history. This is the situation in which an independent consistency check is useful.

The output of the delivered training script on the real dataset (Experiment B) is not available in the delivered files. Table 40 is provided for the values of `models/model_results.csv`.

Table 40. Output of train_model.py on the real dataset (values to be inserted from models/model_results.csv)

Model	Accuracy	Precision	Recall	F1	ROC_AUC
Logistic Regression	[ ]	[ ]	[ ]	[ ]	[ ]
Random Forest	[ ]	[ ]	[ ]	[ ]	[ ]
XGBoost	[ ]	[ ]	[ ]	[ ]	[ ]
LightGBM	[ ]	[ ]	[ ]	[ ]	[ ]

4.5.5 Case study (synthetic-data model)

Table 41 gives the inputs and outputs for the case-study applicant. The FBCE values in the table were reproduced exactly by running the delivered `fbce.py` (test T1). The prediction, the probability and the SHAP contributions were reproduced for this report: the logistic regression trained by the delivered pipeline on the synthetic dataset of Section 4.1.4 (seed 42) gives an approval probability of 50.09% for this applicant and the contributions of Figure 15 (for example credit history +0.873, graduate education +0.393, male gender −0.162, loan amount −0.092 and self-employed −0.013), computed from the closed form of (2.9) with a zero background. The case study therefore illustrates the behaviour of the prototype on a model trained on synthetic data. It is not a result on the real lending data, and an earlier version of this report that attributed it to the real dataset has been corrected (Section 5.4).

Table 41. Case-study inputs and prototype outputs (synthetic-data model; FBCE values from fbce.py)

Input	Value	Output	Value
Applicant income	5000	Prediction	Approved
Co-applicant income	1500	Approval probability	50.09%
Loan amount (thousands)	150	Income consistency	99.77
Loan term (months)	360	Debt consistency	99.71
Credit history	Good (1)	Employment stability	65.00
Self-employed	Yes	Credit consistency	90.00
Education	Graduate	Repayment capacity	99.97
Gender, married	Male, Yes	FBCE score	90.89
Dependents	0	Consistency level	High Consistency
Property area	Urban	Contradiction warnings	None

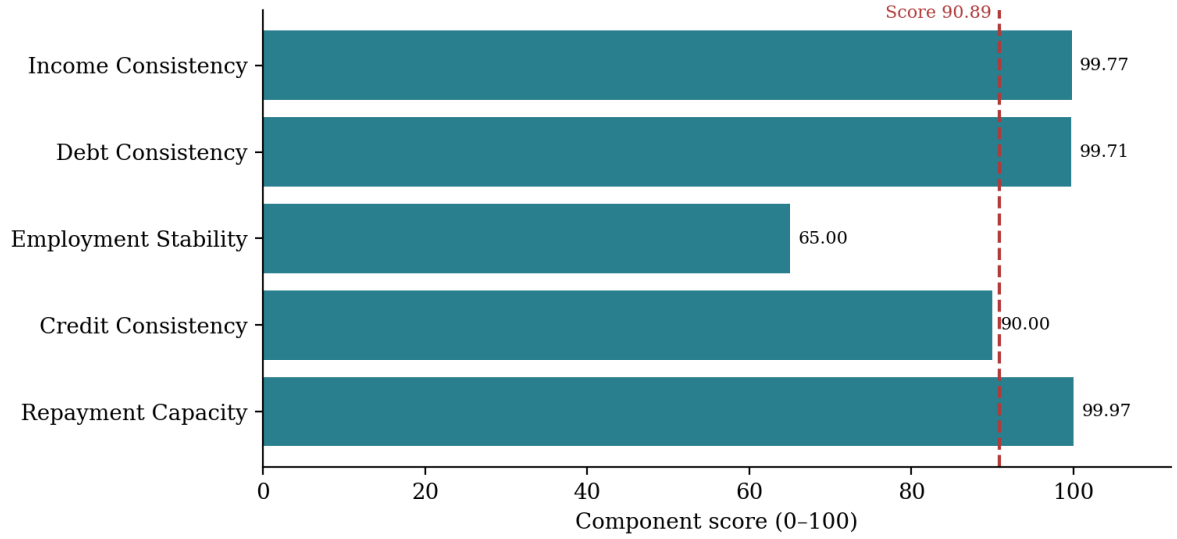


Figure 14. FBCE component scores for the case-study applicant (computed by running fbce.py)

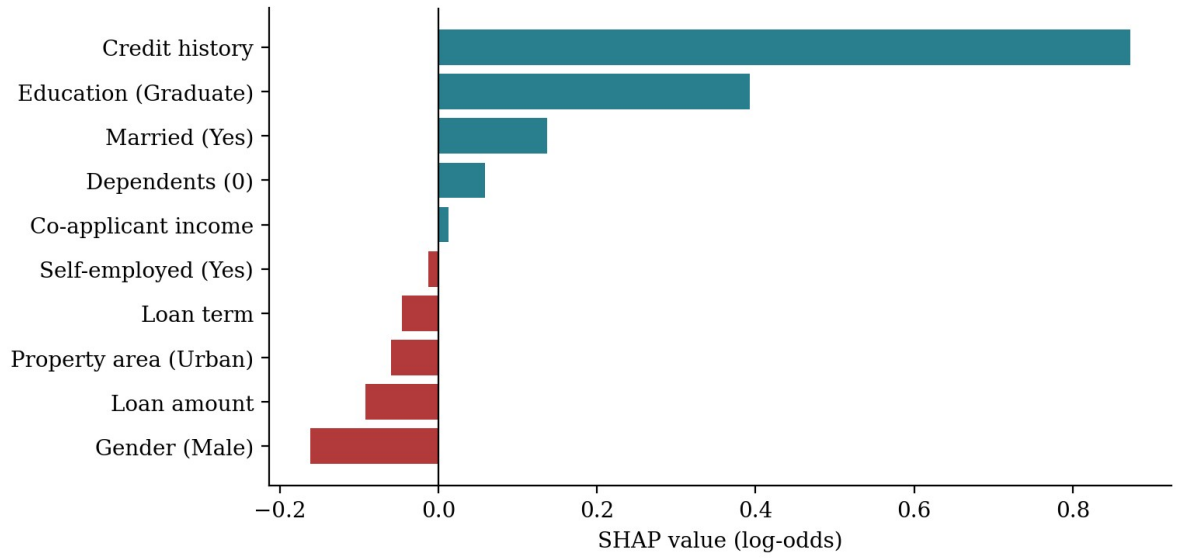


Figure 15. Signed SHAP contributions for the case-study applicant (values reproduced with the synthetic-data model)

The synthetic-data classifier approved the application with a probability of 50.09%, only 0.09 percentage points above the threshold. FBCE was far more certain: three components exceeded 99.7, credit consistency was 90 and employment stability 65, giving a score of 90.89 (High Consistency) with no warnings for a household income of 6,500. In the SHAP view, the positive credit history added 0.873 to the log-odds and graduate education 0.393, while male gender (−0.162) and the loan amount (−0.092) were the largest negative terms.

4.5.6 Dashboard demonstration and screenshots

Section 5.5 reproduces dashboard screenshots for an illustrative input: applicant income 40,000, no co-applicant income, loan amount 170 (thousand), 360-month term, good credit history, a graduate, self-employed, married male applicant with no dependents in an urban area. These screenshots were captured from an earlier build of the dashboard and **do not match the delivered** `app.py` and `fbce.py`. Table 42 lists the differences. The approval probability of 76.5% and the SHAP contributions shown in the screenshots (for example applicant income +1.1779, credit history +0.8730 and loan amount −0.3066) are reproduced by the logistic regression trained on the synthetic dataset (Section 4.3.3), which shows that the screenshots display a model trained on the synthetic data and not on the real Kaggle file.

Table 42. Differences between the earlier dashboard screenshots and the delivered code

Item	Screenshot build	Delivered code
Loan-amount field label	"Loan Amount (in thousands)"	"Loan Amount" with the hint to use the unit of the training dataset
Key indicators	Prediction, probability, FBCE score, estimated EMI and FOIR	Prediction, probability and FBCE score only
Extra panels	Loan approval diagnostic, financial health advisory, financial behaviour analysis profile	Not present in app.py

Item	Screenshot build	Delivered code
FBCE components for the input (40000, 0, 170, 360, 1, Yes)	92.9167, 89.375, 65, 90, 92.5406 (score 86.0)	99.96, 99.95, 65.00, 90.00, 100.00 (score 90.98; test T2)
Consistency-check panel	Debt obligation (FOIR) and leverage ratio	Warnings (or success message) and total income
Model behind probability and SHAP values	Logistic regression; values reproduced by the synthetic-data model (76.5%)	Whichever model train_model.py saves
SHAP list headings	"Positive contributors (Helped Approval)" and "Negative contributors (Risk Factors / Caused Rejection)"	"Positive contributors" and "Negative contributors"

The screenshot build therefore used a different version of the scoring code and of the dashboard, which cannot be reconstructed from the delivered files. The screenshots are retained in Section 5.5 as a record of the earlier build and are labelled as requiring recapture from the delivered code with the trained model; the values displayed in them have not been altered. The FBCE values that the delivered `fbce.py` returns for the same input are given in test T2.

4.5.7 Consolidated test matrix

This section records every check that was considered for the prototype, with its identifier, the requirement to which it relates, the condition or input, the expected behaviour, the observed behaviour and the status. **PASS** means the check was executed and matched the expectation. **GAP CONFIRMED** means the check was executed and confirmed a documented limitation of the code. **NOT RUN** means the check was not executed in the preparation environment and no result is claimed. No check is marked as passed unless it was executed. The checks are described below in groups.

4.5.8 Scoring-function tests T1 to T13

The first group exercises the five-component scoring function `calculate_fbce` with reference profiles, boundary cases and parameter variants (requirements FR-3 and FR-4). Each test was executed on the delivered `fbce.py`, and the observed values are those returned by the code. The reference profiles T1 to T4 cover the case-study applicant, the dashboard demonstration input, a salaried applicant and a weak credit history.

T1 (FR-3/4) examined Case-study applicant (5000, 1500, 150, 360, 1, Yes). Expected: Case-study components 99.77, 99.71, 65, 90, 99.97; High; no warning. Observed: 99.77, 99.71, 65.00, 90.00, 99.97; 90.89; High; 0 warning(s). Status: PASS. **T2** (FR-3/4) examined Dashboard demonstration input (40000, 0, 170, 360, 1, Yes). Expected: Ratio near zero: High; no warning. Observed: 99.96, 99.95, 65.00, 90.00, 100.00; 90.98; High; 0 warning(s). Status: PASS.

T3 (FR-3/4) examined Salaried applicant, good credit (6000, 2000, 120, 360, 1, No). Expected: Salaried (75) with good credit: High; no warning. Observed: 99.85, 99.81, 75.00, 90.00, 99.98; 92.93; High; 0 warning(s). Status: PASS. **T4** (FR-3/4) examined Credit history = 0 (6000, 2000,

120, 360, 0, No). Expected: Credit 30; one warning; level may stay High. Observed: 99.85, 99.81, 75.00, 30.00, 99.98; 80.93; High; 1 warning(s). Status: PASS.

The boundary and stress cases T5, T6, T7 and T13 test very large loans relative to income, the threshold of the high-loan rule, zero income and a low-consistency combination.

T5 (FR-3/4) examined Very large loan relative to income (ratio 20) (1000, 0, 20000, 360, 1, No). Expected: Ratio 20: income and debt 0; one warning. Observed: 0.00, 0.00, 75.00, 90.00, 78.06; 48.61; Low; 1 warning(s). Status: PASS. **T6** (FR-3/4) examined Loan-to-income ratio just above 8 (1000, 0, 8100, 360, 1, No). Expected: Ratio just above 8: debt 0, income 19; one warning. Observed: 19.00, 0.00, 75.00, 90.00, 91.11; 55.02; Low; 1 warning(s). Status: PASS.

T7 (FR-3/4) examined Zero total income (0, 0, 100, 360, 1, No). Expected: Zero income: three components 0; one warning. Observed: 0.00, 0.00, 75.00, 90.00, 0.00; 33.00; Very Low; 1 warning(s). Status: PASS. **T13** (FR-3/4) examined Low-consistency stress case (900, 0, 9000, 60, 0, Yes). Expected: Low consistency; two warnings. Observed: 0.00, 0.00, 65.00, 30.00, 73.44; 33.69; Very Low; 2 warning(s). Status: PASS.

The parameter variants T8 to T12 test a short loan term, the optional employment-years input, a zero interest rate and an unknown employment label.

T8 (FR-3/4) examined Short term (60 months) (2000, 0, 400, 60, 1, No). Expected: Short term still affordable: High. Observed: 98.00, 97.50, 75.00, 90.00, 99.47; 91.99; High; 0 warning(s). Status: PASS. **T9** (FR-3/4) examined Employment years supplied (10) (5000, 0, 100, 240, 1, No), years=10. Expected: Ten years gives employment 100. Observed: 99.80, 99.75, 100.00, 90.00, 99.98; 97.91; High; 0 warning(s). Status: PASS. **T10** (FR-3/4) examined Employment years supplied (0) (5000, 0, 100, 240, 1, No), years=0. Expected: Zero years gives employment 30. Observed: 99.80, 99.75, 30.00, 90.00, 99.98; 83.91; High; 0 warning(s). Status: PASS.

T11 (FR-3/4) examined Interest rate 0 % (5000, 0, 100, 240, 1, No), rate=0. Expected: Zero rate: EMI = loan / term; valid score. Observed: 99.80, 99.75, 75.00, 90.00, 99.99; 92.91; High; 0 warning(s). Status: PASS. **T12** (FR-3/4) examined Unknown employment label (5000, 0, 100, 240, 1, Unknown). Expected: Unknown label gives employment 55. Observed: 99.80, 99.75, 55.00, 90.00, 99.98; 88.91; High; 0 warning(s). Status: PASS.

4.5.9 Input-handling checks V1 to V10

The second group examines how the scoring functions treat unusual or invalid inputs (requirement FR-3). The checks V1, V2, V3, V7, V8 and V9 concern types and special values.

V1 (FR-3) examined credit_consistency('abc'). Expected: 30.0 (documented fallback). Observed: 30.0. Status: PASS. **V2** (FR-3) examined calculate_fbce(..., credit_history='abc'). Expected: Error (float() unguarded). Observed: ValueError: could not convert string to float: 'abc'. Status: GAP CONFIRMED. **V3** (FR-3) examined numeric strings ('5000','1500','150') accepted. Expected: Accepted (float conversion). Observed: 90.89. Status: PASS.

V7 (FR-3) examined applicant income None. Expected: Error (no guard). Observed: TypeError: float() argument must be a string or a real number, not 'NoneType'. Status: GAP CONFIRMED. **V8** (FR-3) examined self_employed=None. Expected: Unknown label 55. Observed: 55.0. Status: PASS. **V9** (FR-3) examined credit_history=0.5 (between 0 and 1). Expected: 30 (threshold is 1). Observed: 30.0. Status: PASS.

The checks V4, V5, V6 and V10 concern the sign and range of numerical inputs.

V4 (FR-3) examined negative loan amount (-100), income 5000. Expected: Rejected ideally; code gives income 100. Observed: (100.0, 0.0). Status: GAP CONFIRMED. **V5** (FR-3) examined loan amount 0, income 5000. Expected: Repayment 0 for non-positive loan. Observed: (100.0, 0.0). Status: PASS. **V6** (FR-3) examined loan term 0 months. Expected: Repayment 0 for non-positive term. Observed: 0.0. Status: PASS. **V10** (FR-3) examined negative income (-100) with positive co-income 100. Expected: Zero-income warning. Observed: ['Reported total income is zero or unavailable.']. Status: PASS.

4.5.10 Training-pipeline and reproducibility checks

The third group uses the functions of the delivered `train_model.py` and `generate_demo_data.py` on the synthetic dataset (requirements FR-2, FR-7 and FR-8). The checks P1, P1b, P2 and P3 concern data loading, the removal of identifier and target columns, the stratified split and the number of model inputs.

P1 (FR-2/7/8) examined load_data() on synthetic CSV. Expected: 800 rows, 11 features, 60 approved. Observed: X shape (800, 11), y approved 60 of 800. Status: PASS. **P1b** (FR-2/7/8) examined Loan_ID and target removed from features. Expected: Both columns removed. Observed: True. Status: PASS.

P2 (FR-2/7/8) examined stratified 80/20 split (seed 42). Expected: 640 train, 160 test. Observed: train 640 (approved 48), test 160 (approved 12). Status: PASS. **P3** (FR-2/7/8) examined build_preprocessor feature count. Expected: 20 features. Observed: 20. Status: PASS.

The checks P4, P5 and P6 concern the error messages for a missing file and a missing target column and the dropping of rows whose target cannot be mapped.

P4 (FR-2/7/8) examined missing data file. Expected: FileNotFoundError with instruction. Observed: FileNotFoundError: Dataset not found: /tmp/none/loan_data.csv .... Status: PASS. **P5** (FR-2/7/8) examined no target column. Expected: ValueError naming candidates. Observed: ValueError: Target column not found. Expected one of: Loan_Status, . Status: PASS. **P6** (FR-2/7/8) examined unmappable targets ('maybe','x') dropped. Expected: Unmappable rows dropped. Observed: rows kept 3 of 5, y=[1, 0, 1]. Status: PASS.

The checks R1 and R2 concern reproducibility (requirement NFR-2): the generator was run twice and the files compared, and the logistic-regression pipeline was fitted twice and the predicted probabilities compared.

R1 (NFR-2) examined generate_demo_data.py run twice, SHA-256 of CSV. Expected: Identical file. Observed: identical (d96922dd86333885…). Status: PASS. **R2** (NFR-2) examined two LR fits give identical probabilities. Expected: Identical probabilities. Observed: True. Status: PASS.

4.5.11 Checks that were not run

Eleven checks could not be executed because XGBoost, LightGBM, SHAP, Streamlit or the real dataset were not available in the preparation environment. They are listed so that the planned coverage is visible; no result is claimed for any of them, and the observed result of each is recorded as not executed.

The three training checks P7, P8 and P9 are as follows. **P7** (FR-1/2/5/6/7) examined Fit XGBoost and LightGBM in the pipeline. Expected: Two more rows in the result table. Observed: Not executed. Status: NOT RUN. **P8** (FR-1/2/5/6/7) examined main() end to end, saving fbce_model.pkl and model_results.csv. Expected: Files written; best model named. Observed: Not executed. Status: NOT RUN. **P9** (FR-1/2/5/6/7) examined train_model.py on the real Kaggle file. Expected: Experiment B results. Observed: Not executed. Status: NOT RUN.

The eight dashboard checks A1 to A8 are as follows. **A1** (FR-1/2/5/6/7) examined Start app.py with no model file. Expected: Warning and stop. Observed: Not executed. Status: NOT RUN. **A2** (FR-1/2/5/6/7) examined Form shows eleven fields with defaults 5000, 1500, 150, 360. Expected: As coded. Observed: Not executed. Status: NOT RUN. **A3** (FR-1/2/5/6/7) examined Press Predict Loan with defaults. Expected: Label, probability, FBCE metrics. Observed: Not executed. Status: NOT RUN. **A4** (FR-1/2/5/6/7) examined FBCE panel for the defaults. Expected: Components table and chart; no warning. Observed: Not executed. Status: NOT RUN.

A5 (FR-1/2/5/6/7) examined SHAP section for logistic regression. Expected: Top-ten table, chart and lists. Observed: Not executed. Status: NOT RUN. **A6** (FR-1/2/5/6/7) examined SHAP section for a tree model. Expected: Tree explainer branch runs. Observed: Not executed. Status: NOT RUN. **A7** (FR-1/2/5/6/7) examined Induced prediction error (missing column). Expected: Error box and hint. Observed: Not executed. Status: NOT RUN. **A8** (FR-1/2/5/6/7) examined Sidebar notice and caption visible. Expected: Both texts shown. Observed: Not executed. Status: NOT RUN.

4.5.12 Explanation check S1

S1 (FR-5) examined Local accuracy of the closed-form linear contributions on the synthetic-data model. Expected: Intercept + sum of contributions = log-odds. Observed: Difference 4.4e-16. Status: PASS.

4.5.13 Summary of the checks

Of the 44 checks described above, 30 were executed and passed, 3 were executed and confirmed a documented limitation, and 11 were not run. The not-run checks are all those that need Streamlit, SHAP, XGBoost, LightGBM or the real dataset, which were unavailable. The gaps confirmed are the unguarded conversions in `fbce.py` (V2 and V7) and the acceptance of a negative loan amount by the scoring function (V4), which the dashboard prevents only through the minimum value of its input widgets.

4.6 DISCUSSION

Different levels of confidence. The three outputs carry different information. Read alone, the label "Approved" points to a routine approval, whereas the probability of 50.09% in the case study (synthetic-data model) is close to a coin toss; the FBCE score then indicates that nothing in the application is incoherent, and the SHAP list shows which inputs the model weighed. Read together, they favour approval after review. The same layout would also expose the opposite situation, a confident approval accompanied by a low FBCE score or an active warning, which the low specificity of the earlier evaluation suggests could be common.

Gender-related contribution. In the case study and in the demonstration input the SHAP output assigns a negative contribution to male gender (−0.162 and −0.1624). Both examples come from the logistic regression trained on the synthetic dataset, in whose generating rule gender plays no role (Section 4.4.11). The coefficient is therefore the fit to random variation in a sample of 800 records and not evidence of bias in real lending decisions. The observation is still useful for a different reason: it shows that a model can assign a non-zero weight to a protected attribute whenever the attribute is available as a feature, and the delivered training script removes only the target and Loan_ID, so Gender and Married remain model features. They should be removed, or at least audited with fairness measures as considered in [2] and [21].

Different treatment of self-employment. The FBCE rule lowers the employment component for self-employed applicants by design. In the synthetic-data case study the classifier gave the attribute almost no weight (−0.013), and in the real Kaggle data the approval rates of the two groups are nearly identical (68.3% and 68.6%). A rule that is not matched by the data is a lending assumption that should be calibrated against repayment outcomes.

Score saturation. For realistic inputs the ratio-based components stay close to 100 in the delivered version (Section 4.5.2), so the overall score is driven by the categorical components.

Tolerances tied to the empirical distributions of the ratios, and consistent units, would give these components a wider spread.

Interpretation of SHAP values. Because the linear explainer uses an all-zero background, the contributions describe the distance of the application from an all-zero input, not from the average applicant. Reviewers should therefore read them as a ranking of the features that moved the log-odds of this particular prediction, not as a causal reason for approval.

4.6.1 What the project demonstrates

The project demonstrates a software design and not a validated lending model. It shows that a prediction, a transparent consistency score and a feature explanation can be produced from the same eleven inputs and displayed without being merged; that the FBCE rules can be tested in isolation, with thirteen executed cases and a sensitivity analysis; and that the delivered scoring functions have properties (unit dependence, a ceiling of 93, and a weak-credit floor of 81) that are visible only because the rules are explicit. It does not demonstrate that FBCE improves lending decisions, that its thresholds are right, or that the classifier generalises.

4.6.2 Practical value

For a student team, an examiner or a future developer the practical value is a small, readable code base in which every number on the screen can be traced to a formula or a coefficient. For a reviewer in a lending setting, the intended value would be a prompt to look at a file again when the signals disagree. That value has not been tested with users.

4.6.3 How FBCE and explainability change interpretation

Table 43. Reading the three signals together (interpretive guide, not an empirical result)

Classifier	FBCE score and warnings	Suggested reading
High approval probability	High score, no warning	Routine file; check that the SHAP list relies on sensible features
High approval probability	Low score or warning	Disagreement; examine the amounts, term and credit history before relying on the approval
Probability near 0.5	High score, no warning	Borderline model output with a coherent file; human judgement matters most
Low approval probability	High score, no warning	Check whether a protected attribute or a single feature drives the rejection
Low approval probability	Low score or warning	Both signals concern the file; review the reasons before any decision

The guide in Table 43 describes how the design intends the signals to be used. It rests on the logic of the system and not on a user study. Because the score has a ceiling of 93 and a weak credit history cannot lower the level below High on its own (Table 30), the warnings should be read before the level.

4.6.4 Remaining uncertainties

- How the classifier and FBCE behave on the real Kaggle data with the delivered training script (Experiment B) is not known from the delivered files.
- The five-fold results of Experiment A cannot be reproduced without the original script.
- The dashboard and SHAP code were not executed in the preparation environment.
- Whether the FBCE thresholds are sensible for real lending has not been examined.
- How reviewers would use or misread the three signals has not been studied.

4.7 LIMITATIONS

4.7.1 Data limitations

- The Kaggle-format dataset is small (614 applications) and records approval decisions, not repayment, so the classifier learns past lending practice, bias included.
- There is no column for existing debt or employment duration, so FBCE uses the requested loan and the Self_Employed flag as proxies.
- The synthetic generator produces a heavily imbalanced target (7.5% approved) from an arbitrary rule and is suitable only for testing that the software runs.

4.7.2 Implementation limitations

- The loan amount and the income are compared without unit conversion, so the ratio components saturate near 100 for realistic inputs.
- The FBCE score is bounded between 19 and 93 for dashboard inputs, and a weak credit history cannot lower the level below High Consistency on its own.
- The SHAP reference input is an all-zero vector and only ten contributions are displayed.

4.7.3 Model limitations

- The reported cross-validation used default hyper-parameters and one fold assignment; its script is not among the delivered files.
- Probabilities are not calibrated, and Gender and Married remain model features.
- The balanced class weights of logistic regression and random forest are not used by XGBoost and LightGBM.

4.7.4 Reliability and input risks

- The dashboard checks only that numbers are non-negative; it does not check plausibility or units.
- Non-numeric or missing values passed directly to `calculate_fbce` raise errors (V2, V7).
- Error messages are generic and do not tell the user which field is wrong.

4.7.5 Generalisation limits

- Results cannot be generalised to production-scale lending portfolios or to other countries or products.
- The FBCE constants were chosen by the team and have not been compared with any lender's policy.

4.7.6 Reproducibility limits

- The real dataset and the output of the delivered training script on it are not available.
- XGBoost, LightGBM, SHAP and Streamlit were not executed for this report, and `requirements.txt` has no version pins.
- The dashboard screenshots come from an earlier build whose FBCE panel and extra panels differ from the delivered code (Table 42).

4.7.7 Deployment limits

- The prototype has no authentication, access control, logging or audit trail.
- The model file is unpickled without verification.
- The dashboard has not been tested with loan officers or accessibility requirements.

4.8 RESPONSIBLE USE, FAIRNESS AND ETHICAL CONSIDERATIONS

A system that influences credit decisions raises ethical questions even when it is only a prototype. This section records how the project treats them and what remains to be done.

4.8.1 Protected attributes and proxies

The dataset contains Gender and Married, which describe the applicant and not the application. In the examples of this report, which come from the synthetic-data model, male gender received a small negative contribution (Section 4.5.5). This is not evidence of discrimination in real lending, because gender plays no part in the synthetic generating rule; it shows that the model can use an attribute that should be irrelevant to creditworthiness. Removing the attributes is the first step; the second is to check whether the remaining features act as proxies for them, because a model can reconstruct a protected attribute from correlated fields. Disparate-impact measures provide a way to check this and are listed as future work.

4.8.2 Learning from past decisions

The labels in the dataset are approval decisions made by lenders. A model trained on them reproduces lending practice, including its inconsistencies and biases, and cannot say whether a rejected applicant would have repaid. The high recall and low specificity of the earlier benchmark are consistent with a model that learned to approve most applicants with a good credit record. For this reason the report presents the classifier as one signal among three and

the FBCE score as a different kind of signal, based on affordability rather than on past approvals.

4.8.3 Transparency and human oversight

Each output of the prototype can be traced: the classifier output to a saved pipeline, the SHAP values to named features, and the FBCE score to explicit formulas and thresholds that are published in this report and in the source code. The sidebar states that the FBCE score is not a banking credit score, and the caption of the result panel states that the prototype does not provide a real banking decision. The design leaves the decision with a human reviewer, which is the appropriate role for a system whose scale factors have not been validated against repayment outcomes.

4.8.4 Data protection

The prototype runs locally and has no connection to a banking system, and the public dataset used for training does not identify individuals. If it were ever applied to real applicants, the inputs would be personal financial data and would require consent, access control, retention rules and an audit trail, none of which the prototype provides.

4.8.5 Honest reporting

Three practices were followed in this report. Results were reported only where a source exists for them; synthetic data were labelled as such and not used as accuracy evidence; and places where the available evidence disagrees or is missing were identified instead of being hidden (Section 5.4).

4.8.6 Data minimisation and privacy

The form collects eleven attributes, including gender, marital status and dependents, which describe the person and not the loan. Data minimisation would drop attributes that are not needed for the stated purpose; for a loan-prediction prototype the protected attributes are the first candidates. The delivered code does not write the entered values to disk, but it also has no consent screen, retention rule or audit trail. If it were applied to real applicants, these controls and the legal requirements of the applicable jurisdiction would have to be added. The report does not make any claim about legal compliance.

4.8.7 Security considerations

- **Unpickling.** `joblib.load` can execute code contained in a malicious file, so the models folder must be writable only by trusted users.
- **Unpinned dependencies.** Installing the latest version of each package makes the environment hard to reproduce and to audit.
- **No authentication.** Anyone who can open the local page can use the dashboard.

- **Local use only.** The prototype should not be exposed to a network in its present form.

4.8.8 Demographic bias and proxy features

Removing Gender and Married from the feature list is necessary but not sufficient, because other attributes such as property area or dependents can act as proxies. Fuster et al. [21] show that more flexible models can change which groups benefit even when restricted variables are handled carefully, so fairness has to be measured, not assumed. The negative weight of male gender in the synthetic-data model of this report is an artefact of random variation, since the generating rule does not use gender; it is nevertheless a reminder that a model fitted to small data can assign a weight to a protected attribute whenever it is available as a feature.

4.8.9 Responsible lending and the limits of automated decisions

An explanation is not a justification. A tidy list of contributions does not show that a prediction is correct, that the training labels were fair or that the applicant has been treated equitably [19], [20]. The output of this prototype must therefore not be treated as an unquestionable lending decision. A human reviewer should be able to override it, applicants should be able to ask for a reconsideration, and decisions should be recorded. None of these procedures is implemented in the prototype; they describe the setting in which a decision-support tool of this kind should be used.

4.8.10 Safeguards

Table 44. Safeguards present in the prototype and safeguards still needed

Safeguard	Present?	Note
Statement that the FBCE score is not a banking credit score	Yes	Sidebar of app.py
Statement that the output is not a real banking decision	Yes	Caption of app.py
Signed contributions that expose attribute weights	Yes	SHAP section
Removal of protected attributes from the features	No	Gender and Married remain
Formal fairness metrics	No	Future work
Probability calibration	No	Future work
Access control, logging, audit trail	No	Not needed for the academic demonstration
Human review of every decision	Design intent only	Outside the software

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

Problem and approach. Loan classifiers return labels without reasons and do not check whether the details of an application agree with one another. This project built FBCE, a decision-support prototype that places three outputs side by side: the prediction of a saved classifier, a separate five-component consistency score with warnings, and a SHAP-based list of feature contributions. The signals are deliberately not merged, so that a disagreement between them stays visible.

What was implemented. The delivered code contains a training script that compares logistic regression, random forest, XGBoost and LightGBM inside one preprocessing pipeline and saves the best model; the FBCE module with income, debt, employment, credit and repayment components, equal weights and four consistency levels; a synthetic-data generator; and a Streamlit dashboard that combines the saved model, the FBCE module and the SHAP explanation, with visible notices that the output is not a banking decision.

What the evidence supports. The FBCE module behaves as designed in thirteen executed test cases and ten executed validation checks, which also confirmed three documented gaps. Its sensitivity analysis shows that the delivered version depends on the unit of the loan amount, saturates near 100 for realistic inputs in the dataset's units, and cannot score above 93, and that a weak credit history alone cannot lower the level below High Consistency. The training pipeline ran end to end for logistic regression and random forest on a labelled synthetic dataset. Executing the delivered pipeline on that dataset also reproduced the case-study probability of 50.09%, the demonstration probability of 76.5% and the listed SHAP contributions, which shows that the case study and the dashboard screenshots illustrate a model trained on synthetic data and not a model trained on the 614-record Kaggle file. The five-fold cross-validation reported from the earlier phase, in which logistic regression matched the ensembles (80.30% accuracy, 87.14% F1, AUC 0.751), remains reported and not reproduced.

Limits. The negative contribution of male gender in the examples is not evidence of bias in real lending decisions, because gender plays no role in the synthetic generating rule; it illustrates that a fitted model can weight a protected attribute that remains available as a feature. The prototype is an academic demonstration. It supports human review and does not replace it, and its consistency score is a project-defined indicator and not a banking credit score. The limitations of Section 4.7 define the work that remains.

5.2 FUTURE WORK

The following items are proposals. None of them is implemented in the delivered prototype. For each item, the proposed step, the reason for proposing it and the way in which it would be verified are stated below.

5.2.1 Data Validation and Units

The proposed step is to convert the loan amount and the income to common units and to validate ranges and types in the form and in `calculate_fbce`. It is proposed because it removes the saturation and the unit dependence observed in tests T1 and T5 and in the input-handling checks V2, V4 and V7. It would be verified by repeating the test matrix and by adding unit tests for each validation rule.

5.2.2 Data

The proposed step is to use data with liabilities, tenure and repayment outcomes. It is proposed because the FBCE constants could then be fitted and not set by hand. It would be verified by an out-of-sample comparison on authorised data.

5.2.3 Calibration

The proposed step is to apply Platt scaling or isotonic regression. It is proposed so that the probabilities could be read as probabilities. It would be verified by a calibration curve on held-out data.

5.2.4 Broader Evaluation

The proposed step is to use cross-validation in the training script, repeated folds and a hyper-parameter search. It is proposed because the single split varies by several points on small data. It would be verified by the mean and the spread over folds, stored with the report.

5.2.5 External Validation

The proposed step is to evaluate the prototype on an appropriate, authorised external dataset. It is proposed because it tests generalisation beyond one file. It would be verified by a documented dataset, split and metrics.

5.2.6 Unit and Integration Testing

The proposed step is to write automated tests for `fbce.py`, `train_model.py` and the app flow. It is proposed because it makes the test matrix repeatable. It would be verified by a continuous test run with stored results.

5.2.7 Fairness Evaluation

The proposed step is to remove protected attributes, to add disparate-impact and error-rate comparisons and to check proxies. It is proposed because explanations alone do not guarantee fairness [21]. It would be verified by reported metrics per group.

5.2.8 Explanation

The proposed step is to use a background drawn from the training data and to show the intercept and the remainder of the contributions. It is proposed because the contributions would then be

relative to the average applicant and could be reconstructed. It would be verified by comparison with the SHAP library output.

5.2.9 Interface

The proposed step is to give specific validation messages, to use neutral headings for the contribution lists and to offer an optional EMI and FOIR display. It is proposed because it gives clearer feedback and headings that are correct for approved applications. It would be verified by a user walkthrough with recorded observations.

5.2.10 Reproducibility

The proposed step is to pin library versions, to keep the results files and to script the screenshot capture. It is proposed because it allows others to repeat the work. It would be verified by a fresh-environment installation test.

5.2.11 Monitoring and Deployment Safeguards

The proposed step is to add logging, access control, a model-file integrity check and a review workflow. It is proposed because these are needed before any real use. It would be verified by a security review and a pilot with oversight.

5.3 SOURCE CODE

The source files are listed in the order of the project folder. In use they are run in a different order, shown in Table 46. The listings are exact copies of the delivered files; no line has been changed. Table 45 summarises the purpose, inputs, outputs and dependencies of each file.

Table 45. Summary of the delivered source files

File	Purpose	Inputs and outputs	Dependencies
app.py	Streamlit dashboard	In: form values entered in the browser; models/fbce_model.pkl. Out: browser panels: prediction, probability, FBCE result, SHAP table, chart and lists, decision summary.	streamlit, shap, joblib, pandas, numpy, matplotlib, fbce.py
fbce.py	FBCE scoring functions	In: applicant and co-applicant income, loan amount, term, credit history, self-employment (optional rate and years). Out: dictionary with component scores, overall score, level, total income and warnings.	Python standard library only (math is imported but not used)
train_model.py	Model training and comparison	In: data/loan_data.csv. Out: models/fbce_model.pkl, models/model_results.csv and a printed comparison.	pandas, numpy, scikit-learn, xgboost, lightgbm, joblib
generate_demo_data.py	Synthetic dataset generator	In: none; NumPy seed 42. Out: data/loan_data.csv with 800 rows and 13 columns.	numpy, pandas
requirements.txt	Dependency list	In: read by pip. Out: installed packages.	pip
README.md	Setup and usage guide	In: read by people. Out: instructions and disclaimer.	none
PUT_DATASET_HERE.txt	Dataset placement note	In: read by people. Out: instruction to save the CSV as loan_data.csv.	none

Table 46. Execution sequence of the project files

Step	Command	File used	Result
1	pip install -r requirements.txt	requirements.txt	Libraries installed
2	python generate_demo_data.py (optional)	generate_demo_data.py	Synthetic data/loan_data.csv
3	python train_model.py	train_model.py	models/fbce_model.pkl and model_results.csv
4	streamlit run app.py	app.py, fbce.py	Dashboard in the browser

5.3.1 app.py

```
from pathlib import Path
import warnings
warnings.filterwarnings("ignore")

import joblib
import pandas as pd
import numpy as np
import streamlit as st
import shap
import matplotlib.pyplot as plt

from fbce import calculate_fbce

BASE = Path(__file__).resolve().parent
MODEL_PATH = BASE / "models" / "fbce_model.pkl"
RESULT_PATH = BASE / "models" / "model_results.csv"

st.set_page_config(
    page_title="FBCE - Explainable Loan Prediction",
    page_icon="",
    layout="wide"
)

st.title("Explainable Loan Prediction using FBCE")
st.caption("Financial Behaviour Consistency Engine — academic prototype")

@st.cache_resource
def load_model():
    if not MODEL_PATH.exists():
        return None
    return joblib.load(MODEL_PATH)

bundle = load_model()

st.sidebar.header("Project")
st.sidebar.write("FBCE combines machine-learning prediction, financial consistency checks and SHAP
explanations.")
st.sidebar.info(
    "The FBCE score is a project-defined consistency indicator. "
    "It is not a banking credit score."
)

if bundle is None:
    st.warning("Model is not trained yet.")
    st.code("python train_model.py", language="bash")
    st.stop()

model = bundle["model"]
model_name = bundle["model_name"]

st.sidebar.success(f"Loaded model: {model_name}")

with st.form("loan_form"):
    st.subheader("Applicant Financial Details")

    c1, c2, c3 = st.columns(3)

    with c1:
        applicant_income = st.number_input(
            "Applicant Income", min_value=0.0, value=5000.0, step=500.0
        )
        coapplicant_income = st.number_input(
```

```
            "Co-applicant Income", min_value=0.0, value=1500.0, step=500.0
        )
        loan_amount = st.number_input(
            "Loan Amount", min_value=0.0, value=150.0, step=10.0,
            help="Use the same unit used by your training dataset."
        )

    with c2:
        loan_term = st.number_input(
            "Loan Term (months)", min_value=1, value=360, step=12
        )
        credit_history = st.selectbox(
            "Credit History", [1.0, 0.0],
            format_func=lambda x: "Good / Available" if x == 1.0 else "Weak / Unavailable"
        )
        gender = st.selectbox("Gender", ["Male", "Female"])
        married = st.selectbox("Married", ["Yes", "No"])

    with c3:
        dependents = st.selectbox("Dependents", ["0", "1", "2", "3+"])
        education = st.selectbox("Education", ["Graduate", "Not Graduate"])
        self_employed = st.selectbox("Self Employed", ["Yes", "No"])
        property_area = st.selectbox("Property Area", ["Urban", "Semiurban", "Rural"])

    submitted = st.form_submit_button("Predict Loan")

if submitted:
    # These columns match the common Kaggle Loan Approval dataset.
    input_df = pd.DataFrame([{
        "Gender": gender,
        "Married": married,
        "Dependents": dependents,
        "Education": education,
        "Self_Employed": self_employed,
        "ApplicantIncome": applicant_income,
        "CoapplicantIncome": coapplicant_income,
        "LoanAmount": loan_amount,
        "Loan_Amount_Term": loan_term,
        "Credit_History": credit_history,
        "Property_Area": property_area
    }])

    try:
        prediction = int(model.predict(input_df)[0])
        probability = float(model.predict_proba(input_df)[0][1])

        fbce = calculate_fbce(
            applicant_income=applicant_income,
            coapplicant_income=coapplicant_income,
            loan_amount=loan_amount,
            loan_term_months=loan_term,
            credit_history=credit_history,
            self_employed=self_employed
        )

        st.divider()
        st.subheader("Prediction Result")

        r1, r2, r3 = st.columns(3)
        with r1:
            st.metric("Loan Prediction", "APPROVED" if prediction == 1 else "REJECTED")
        with r2:
            st.metric("Approval Probability", f"{probability * 100:.2f}%")
        with r3:
            st.metric("FBCE Score", f"{fbce['fbce_score']:.2f}/100")

        st.write(f"**Consistency Level:** {fbce['risk_level']}")

        left, right = st.columns(2)

        with left:
            st.subheader("FBCE Components")
            score_df = pd.DataFrame(
                list(fbce["scores"].items()),
                columns=["Component", "Score"]
            )
            st.dataframe(score_df, use_container_width=True, hide_index=True)
            st.bar_chart(score_df.set_index("Component"))

        with right:
            st.subheader("Financial Consistency Checks")
            if fbce["contradictions"]:
```

```
            for item in fbce["contradictions"]:
                st.warning(item)
        else:
            st.success("No major project-defined financial consistency contradiction detected.")

        st.write(f"**Total Income:** {fbce['total_income']:.2f}")

    st.divider()
    st.subheader("SHAP Explanation")

    try:
        preprocessor = model.named_steps["preprocessor"]
        estimator = model.named_steps["model"]
        transformed = preprocessor.transform(input_df)

        feature_names = preprocessor.get_feature_names_out()

        if model_name in ["Random Forest", "XGBoost", "LightGBM"]:
            explainer = shap.TreeExplainer(estimator)
            shap_values = explainer.shap_values(transformed)

            if isinstance(shap_values, list):
                values = shap_values[1][0]
            else:
                values = np.asarray(shap_values)
                if values.ndim == 3:
                    values = values[0, :, 1]
                elif values.ndim == 2:
                    values = values[0]

            explanation_df = pd.DataFrame({
                "Feature": feature_names,
                "SHAP Value": values
            })
        else:
            # Linear-model explanation.
            background = np.zeros((1, transformed.shape[1]))
            explainer = shap.LinearExplainer(estimator, background)
            shap_values = explainer.shap_values(transformed)
            values = np.asarray(shap_values)
            if values.ndim == 2:
                values = values[0]

            explanation_df = pd.DataFrame({
                "Feature": feature_names,
                "SHAP Value": values
            })

        explanation_df["Absolute"] = explanation_df["SHAP Value"].abs()
        explanation_df = explanation_df.sort_values("Absolute", ascending=False).head(10)

        st.dataframe(
            explanation_df[["Feature", "SHAP Value"]],
            use_container_width=True,
            hide_index=True
        )

        fig, ax = plt.subplots(figsize=(9, 5))
        plot_df = explanation_df.sort_values("SHAP Value")
        ax.barh(plot_df["Feature"], plot_df["SHAP Value"])
        ax.axvline(0, linewidth=1)
        ax.set_xlabel("SHAP contribution")
        ax.set_title("Top SHAP Feature Contributions")
        fig.tight_layout()
        st.pyplot(fig)

        positive = explanation_df[explanation_df["SHAP Value"] > 0]
        negative = explanation_df[explanation_df["SHAP Value"] < 0]

        p1, p2 = st.columns(2)
        with p1:
            st.write("**Positive contributors**")
            if len(positive):
                for _, row in positive.iterrows():
                    st.write(f"• {row['Feature']}: +{row['SHAP Value']:.4f}")
            else:
                st.write("No positive contributors in the displayed top features.")

        with p2:
            st.write("**Negative contributors**")
            if len(negative):
                for _, row in negative.iterrows():
```

```
                        st.write(f"• {row['Feature']}: {row['SHAP Value']:.4f}")
                else:
                    st.write("No negative contributors in the displayed top features.")

        except Exception as e:
            st.error("SHAP explanation could not be generated for this model/input.")
            st.code(str(e))

        st.divider()
        st.subheader("Decision Summary")

        if prediction == 1:
            st.success(
                "The machine-learning model predicts approval. "
                "Review the FBCE score and SHAP factors before treating the output as decision
support."
            )
        else:
            st.error(
                "The machine-learning model predicts rejection. "
                "Review the FBCE score and SHAP factors to understand the major contributing
features."
            )

        st.caption(
            "For academic demonstration only. This prototype does not provide a real banking
decision."
        )

    except Exception as e:
        st.error("Prediction failed.")
        st.code(str(e))
        st.info(
            "Make sure your dataset uses the same feature names as the common "
            "Loan Approval Prediction dataset."
        )
```

5.3.2 fbce.py

```
import math

def clamp(value, low=0.0, high=100.0):
    return max(low, min(high, float(value)))

def income_consistency(total_income, loan_amount):
    # Uses loan-to-income ratio as a simple academic consistency indicator.
    if total_income <= 0:
        return 0.0
    ratio = loan_amount / total_income
    # Lower requested-loan-to-income ratio gets a higher score.
    return clamp(100 * (1 - min(ratio / 10.0, 1.0)))

def debt_consistency(total_income, loan_amount):
    # The common Kaggle Loan Approval dataset does not contain an
    # explicit existing-debt column. Therefore this component uses
    # requested-loan burden as a proxy and is clearly labeled as such.
    if total_income <= 0:
        return 0.0
    burden = loan_amount / total_income
    return clamp(100 * (1 - min(burden / 8.0, 1.0)))

def employment_stability(self_employed, employment_years=None):
    # If years are supplied, use them; otherwise use the available
    # Self_Employed field from the common public dataset.
    if employment_years is not None:
        return clamp(30 + min(float(employment_years), 10) * 7)

    value = str(self_employed).strip().lower()
    if value in ("no", "not self employed", "salaried"):
        return 75.0
    if value in ("yes", "self employed"):
        return 65.0
    return 55.0

def credit_consistency(credit_history):
    try:
        value = float(credit_history)
    except Exception:
        value = 0.0
    return 90.0 if value >= 1 else 30.0
```

```
def repayment_capacity(total_income, loan_amount, loan_term_months, annual_interest_rate=10.0):
    if total_income <= 0 or loan_amount <= 0 or loan_term_months <= 0:
        return 0.0

    r = (annual_interest_rate / 100.0) / 12.0
    n = float(loan_term_months)

    if r == 0:
        emi = loan_amount / n
    else:
        emi = loan_amount * r * ((1 + r) ** n) / (((1 + r) ** n) - 1)

    emi_ratio = emi / total_income

    # Academic prototype scale: lower EMI-to-income ratio is more consistent.
    return clamp(100 * (1 - min(emi_ratio / 0.80, 1.0)))

def calculate_fbce(
    applicant_income,
    coapplicant_income,
    loan_amount,
    loan_term_months,
    credit_history,
    self_employed,
    annual_interest_rate=10.0,
    employment_years=None
):
    total_income = float(applicant_income) + float(coapplicant_income)
    loan_amount = float(loan_amount)

    scores = {
        "Income Consistency": income_consistency(total_income, loan_amount),
        "Debt Consistency": debt_consistency(total_income, loan_amount),
        "Employment Stability": employment_stability(self_employed, employment_years),
        "Credit Consistency": credit_consistency(credit_history),
        "Repayment Capacity": repayment_capacity(
            total_income, loan_amount, loan_term_months, annual_interest_rate
        ),
    }

    # Equal weights are a project-design choice, not a banking standard.
    fbce_score = sum(scores.values()) / len(scores)

    if fbce_score >= 80:
        risk = "High Consistency"
    elif fbce_score >= 60:
        risk = "Moderate Consistency"
    elif fbce_score >= 40:
        risk = "Low Consistency"
    else:
        risk = "Very Low Consistency"

    contradictions = []

    if total_income <= 0:
        contradictions.append("Reported total income is zero or unavailable.")
    elif loan_amount / total_income > 8:
        contradictions.append("Requested loan is very high relative to reported income.")

    if float(credit_history) < 1:
        contradictions.append("Credit history is marked as weak/unavailable.")

    return {
        "scores": scores,
        "fbce_score": round(fbce_score, 2),
        "risk_level": risk,
        "total_income": round(total_income, 2),
        "contradictions": contradictions,
    }
```

5.3.3 train_model.py

```
from pathlib import Path
import warnings
warnings.filterwarnings("ignore")

import joblib
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
```

```
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

BASE = Path(__file__).resolve().parent
DATA_PATH = BASE / "data" / "loan_data.csv"
MODEL_PATH = BASE / "models" / "fbce_model.pkl"
RESULT_PATH = BASE / "models" / "model_results.csv"

# Works with the common Kaggle Loan Approval Prediction dataset.
TARGET_CANDIDATES = ["Loan_Status", "loan_status", "LoanStatus", "target"]

def load_data():
    if not DATA_PATH.exists():
        raise FileNotFoundError(
            f"Dataset not found: {DATA_PATH}\n"
            "Download a public loan approval dataset and save it as data/loan_data.csv"
        )

    df = pd.read_csv(DATA_PATH)

    target = next((c for c in TARGET_CANDIDATES if c in df.columns), None)
    if target is None:
        raise ValueError(
            "Target column not found. Expected one of: "
            + ", ".join(TARGET_CANDIDATES)
        )

    # Convert common Y/N target to 1/0.
    y = df[target].astype(str).str.strip().str.upper().map({
        "Y": 1, "YES": 1, "APPROVED": 1, "1": 1,
        "N": 0, "NO": 0, "REJECTED": 0, "0": 0
    })

    valid = y.notna()
    df = df.loc[valid].copy()
    y = y.loc[valid].astype(int)

    # Loan_ID is an identifier, not a useful predictive feature.
    drop_cols = [target]
    if "Loan_ID" in df.columns:
        drop_cols.append("Loan_ID")

    X = df.drop(columns=drop_cols)

    # Remove accidental empty columns.
    X = X.loc[:, ~X.columns.astype(str).str.startswith("Unnamed")]

    return X, y

def build_preprocessor(X):
    categorical = X.select_dtypes(include=["object", "category", "bool"]).columns.tolist()
    numerical = [c for c in X.columns if c not in categorical]

    num_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ])

    cat_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
    ])

    preprocessor = ColumnTransformer([
        ("num", num_pipe, numerical),
        ("cat", cat_pipe, categorical)
    ], remainder="drop")

    return preprocessor

def build_models():
    return {
        "Logistic Regression": LogisticRegression(max_iter=2000, class_weight="balanced"),
        "Random Forest": RandomForestClassifier(
            n_estimators=300,
```

```
            random_state=42,
            class_weight="balanced",
            min_samples_leaf=2
        ),
        "XGBoost": XGBClassifier(
            n_estimators=250,
            max_depth=4,
            learning_rate=0.05,
            subsample=0.9,
            colsample_bytree=0.9,
            eval_metric="logloss",
            random_state=42
        ),
        "LightGBM": LGBMClassifier(
            n_estimators=250,
            learning_rate=0.05,
            num_leaves=31,
            random_state=42,
            verbosity=-1
        )
    }

def main():
    X, y = load_data()

    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.20,
        random_state=42,
        stratify=y
    )

    results = []
    trained = {}

    for name, model in build_models().items():
        preprocessor = build_preprocessor(X_train)

        pipe = Pipeline([
            ("preprocessor", preprocessor),
            ("model", model)
        ])

        pipe.fit(X_train, y_train)

        pred = pipe.predict(X_test)
        if hasattr(pipe, "predict_proba"):
            prob = pipe.predict_proba(X_test)[:, 1]
        else:
            prob = None

        row = {
            "Model": name,
            "Accuracy": accuracy_score(y_test, pred),
            "Precision": precision_score(y_test, pred, zero_division=0),
            "Recall": recall_score(y_test, pred, zero_division=0),
            "F1": f1_score(y_test, pred, zero_division=0),
        }

        if prob is not None and len(np.unique(y_test)) == 2:
            row["ROC_AUC"] = roc_auc_score(y_test, prob)
        else:
            row["ROC_AUC"] = np.nan

        results.append(row)
        trained[name] = pipe

    results_df = pd.DataFrame(results).sort_values(
        ["F1", "Accuracy"], ascending=False
    )

    best_name = results_df.iloc[0]["Model"]
    best_model = trained[best_name]

    MODEL_PATH.parent.mkdir(exist_ok=True)
    joblib.dump(
        {
            "model": best_model,
            "model_name": best_name,
            "feature_columns": X.columns.tolist()
        },
        MODEL_PATH
```

```
    )

    results_df.to_csv(RESULT_PATH, index=False)

    print("\nMODEL COMPARISON")
    print(results_df.to_string(index=False))
    print(f"\nBest model: {best_name}")
    print(f"Saved model: {MODEL_PATH}")

if __name__ == "__main__":
    main()
```

5.3.4 generate_demo_data.py

```
from pathlib import Path
import numpy as np
import pandas as pd

np.random.seed(42)
N = 800

income = np.random.lognormal(mean=8.4, sigma=0.55, size=N).astype(int)
co_income = np.random.lognormal(mean=7.0, sigma=0.8, size=N).astype(int)
loan = np.clip(np.random.normal(140, 55, N), 20, 400).round(0)
term = np.random.choice([120, 180, 240, 300, 360, 480], N)
credit = np.random.choice([0, 1], N, p=[0.18, 0.82])

education = np.random.choice(["Graduate", "Not Graduate"], N, p=[0.78, 0.22])
self_emp = np.random.choice(["Yes", "No"], N, p=[0.15, 0.85])
gender = np.random.choice(["Male", "Female"], N, p=[0.80, 0.20])
married = np.random.choice(["Yes", "No"], N, p=[0.65, 0.35])
dependents = np.random.choice(["0", "1", "2", "3+"], N, p=[0.58, 0.17, 0.16, 0.09])
property_area = np.random.choice(["Urban", "Semiurban", "Rural"], N)

# Synthetic target only for testing the software pipeline.
# It is NOT a real-world approval rule.
score = (
    1.6 * credit
    + 0.00002 * income
    + 0.00001 * co_income
    - 0.0035 * loan
    + 0.25 * (education == "Graduate")
    + 0.10 * (property_area == "Semiurban")
    + np.random.normal(0, 0.7, N)
)
prob = 1 / (1 + np.exp(-score + 2.5))
status = (prob > 0.5).astype(int)

df = pd.DataFrame({
    "Loan_ID": [f"D{100000+i}" for i in range(N)],
    "Gender": gender,
    "Married": married,
    "Dependents": dependents,
    "Education": education,
    "Self_Employed": self_emp,
    "ApplicantIncome": income,
    "CoapplicantIncome": co_income,
    "LoanAmount": loan,
    "Loan_Amount_Term": term,
    "Credit_History": credit,
    "Property_Area": property_area,
    "Loan_Status": np.where(status == 1, "Y", "N")
})

out = Path(__file__).resolve().parent / "data" / "loan_data.csv"
df.to_csv(out, index=False)
print(f"Created demo dataset: {out}")
print(df.head())
```

5.3.5 requirements.txt

```
pandas
numpy
scikit-learn
xgboost
lightgbm
shap
streamlit
matplotlib
joblib
```

5.3.6 README.md

````
# Explainable Loan Prediction using Financial Behaviour Consistency Engine (FBCE)

## Project idea

This project combines:

1. Machine learning loan prediction
2. Financial Behaviour Consistency Engine (FBCE)
3. SHAP-based explainability
4. Streamlit dashboard

The FBCE prototype contains five components:

- Income Consistency
- Debt Consistency
- Employment Stability
- Credit Consistency
- Repayment Capacity

The final FBCE score is normalized to a 0–100 style project score.

## Important dataset note

The common Kaggle Loan Approval Prediction dataset does not contain an explicit existing-debt column
or employment-duration column.

Therefore:

- Debt Consistency uses requested-loan burden as an academic proxy.
- Employment Stability uses the available Self_Employed field unless employment years are added.

Do not describe these as real banking standards.

## Folder structure

```text
FBCE_Loan_Prediction/
│
├── app.py
├── fbce.py
├── train_model.py
├── generate_demo_data.py
├── requirements.txt
├── README.md
│
├── data/
│   └── loan_data.csv
│
└── models/
    ├── fbce_model.pkl
    └── model_results.csv
```

## Step 1: Install Python

Use Python 3.10 or 3.11 for an easier student setup.

## Step 2: Install libraries

Open CMD inside the project folder:

```bash
pip install -r requirements.txt
```

## Step 3: Add dataset

Download a public Loan Approval Prediction dataset from Kaggle and place the CSV here:

```text
data/loan_data.csv
```

The training script expects a target column named `Loan_Status` (Y/N) or one of the documented
alternatives.

## Step 4: Train all models

```bash
````
````

```
python train_model.py
```

The program compares:

- Logistic Regression
- Random Forest
- XGBoost
- LightGBM

It saves the best model based primarily on F1 score and then accuracy.

Step 5: Start the dashboard

```bash
streamlit run app.py
```

If you do not have the dataset yet

You can create a synthetic demo dataset:

```bash
python generate_demo_data.py
python train_model.py
streamlit run app.py
```

The synthetic data is only for testing the software pipeline and must not be presented as real banking data or real-world accuracy evidence.

Outputs

The dashboard shows:

- Loan prediction
- Approval probability
- FBCE score
- Five FBCE component scores
- Consistency warnings
- SHAP feature contributions
- Positive and negative contributors

Academic disclaimer

This is an academic prototype for explainable decision support. The FBCE score and thresholds are project-defined and are not official banking credit standards.

5.3.7 PUT_DATASET_HERE.txt

```
Place your public loan approval CSV here and rename it to:

loan_data.csv

Recommended columns:
Loan_ID, Gender, Married, Dependents, Education, Self_Employed,
ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term,
Credit_History, Property_Area, Loan_Status
```

5.4 RESULTS

This section records what was executed while preparing this report, what was only inspected, which results are available and which are not, and every known mismatch between the report, the screenshots and the source code. Nothing listed as unverified has been promoted to verified.

5.4.1 Files supplied

File	Supplied	Used for
app.py, fbce.py, train_model.py, generate_demo_data.py	Yes	Description, excerpts, execution (not app.py)
requirements.txt, README.md, PUT_DATASET_HERE.txt	Yes	Dependencies and procedure

File	Supplied	Used for
Real Kaggle dataset (data/loan_data.csv)	No	–
models/model_results.csv and fbce_model.pkl	No	–
Script or notebook of the five-fold study	No	–
Dashboard screenshots (seven images)	Yes	Section 5.5

5.4.2 Reproducibility of the reported results

Table 47. Verification status of the main evidence

Item	Status	Note
FBCE tests, sensitivity tables, worked example	Executed	Delivered fbce.py (identical to Section 5.3)
Syntax of the four scripts	Executed	py_compile without errors
Synthetic dataset	Executed	800 rows, 13 columns, 60 approved, 654 with Credit_History = 1
Training pipeline, logistic regression and random forest, synthetic data	Executed	Test set of 160 records
Training pipeline, XGBoost and LightGBM	Not executed	Libraries not available
app.py with Streamlit and SHAP	Not executed	Reviewed statically
Case-study probability and SHAP values	Reproduced	By the synthetic-data model; not real-data results
Screenshot probability and SHAP values	Reproduced	By the synthetic-data model
Screenshot FBCE values and extra panels	Recapture required	Differ from delivered code (Table 42)
Five-fold cross-validation, Table 39	Reported, not reproduced	Script not delivered; configuration differs from train_model.py
train_model.py on the real dataset, Table 40	Missing	Real dataset and results file not delivered
References [1], [2], [4], [5], [16]–[21]	Confirmed	Checked against publisher, repository or index records
References [3], [6], [9], [10]	Unconfirmed	Bibliographic details not confirmed
References [7], [8], [11]–[15]	Standard works	Not re-checked online for this report
Library versions of the authors	Not recorded	requirements.txt has no version pins

5.4.3 Register of known mismatches

Table 48. Known mismatches between report, screenshots and code

ID	Observation	Status
M1	Screenshot FBCE component values (92.9167, 89.375, 65, 90, 92.5406; score 86.0) differ from fbce.py (99.96, 99.95, 65, 90, 100; score 90.98)	Open: recapture
M2	EMI, FOIR, diagnostic, advisory and behaviour-profile panels appear in screenshots but not in app.py	Open: recapture or remove from claims
M3	SHAP list headings differ between screenshots and app.py	Open (wording)
M4	Probability and SHAP values of the case study and screenshots are reproduced by the synthetic-data model, although an earlier version of this report attributed them to the real dataset	Corrected in this report
M5	Five-fold study (XGBoost 300 rounds depth 3, LightGBM 300 rounds 15 leaves) differs from train_model.py (250 and depth 4; 250 and 31 leaves)	Open: script not supplied
M6	Output of train_model.py on the real dataset unavailable	Open

ID	Observation	Status
M7	Loan amount and income compared without unit conversion in fbce.py	Documented; not fixed
M8	requirements.txt has no version pins	Open
M9	XGBoost, LightGBM, SHAP and Streamlit code not executed	Open
M10	References [3], [6], [9] and [10] not confirmed	Open

5.5 USER INTERFACE OUTPUTS

5.5.1 Dashboard screenshots

The screenshots on the following pages were supplied as dashboard output for the demonstration input of Section 4.5.6. They were captured from an earlier build of the dashboard. Executing the delivered training pipeline on the synthetic dataset reproduces their approval probability (76.5%) and their SHAP contributions to four decimals (Section 4.3.3), so the model shown is the logistic regression trained on the synthetic data. Their FBCE component values and several panels do not correspond to the delivered `fbce.py` and `app.py` (Table 42). They are therefore kept as a record of the earlier build and are labelled as **requiring recapture** from the delivered code. No value in them has been altered; the images are reproduced in full and are not cropped. To recapture them: run `python generate_demo_data.py` or place the real dataset, run `python train_model.py`, start `streamlit run app.py`, enter the input of Section 4.5.6 and capture the page.

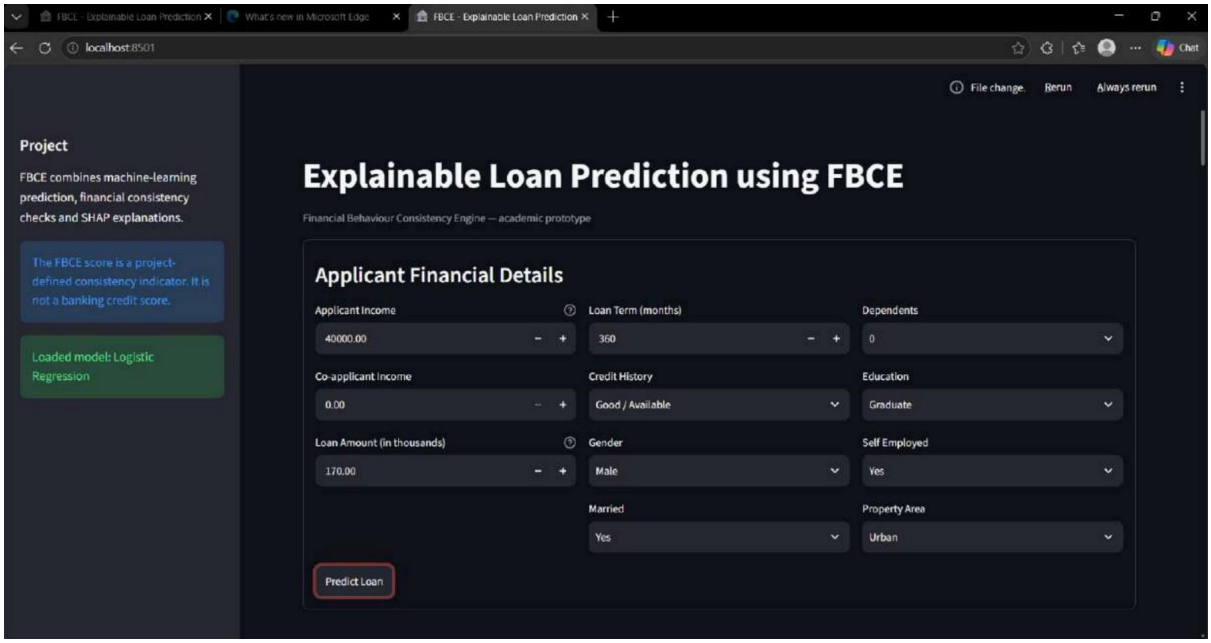


Figure 16. Dashboard input form: Applicant Financial Details (earlier build)

The form has three columns of inputs, as in the delivered code: applicant income 40000, co-applicant income 0, loan amount 170, term 360 months, credit history “Good / Available”, gender Male, married Yes, dependents 0, education Graduate, self-employed Yes and property area Urban, and the “Predict Loan” button. The sidebar carries the project description, the notice that the FBCE score is not a banking credit score and “Loaded model: Logistic Regression”. **Difference:** the loan-amount label reads “in thousands” here, whereas the delivered form shows only “Loan Amount” with a hint to use the dataset’s unit.

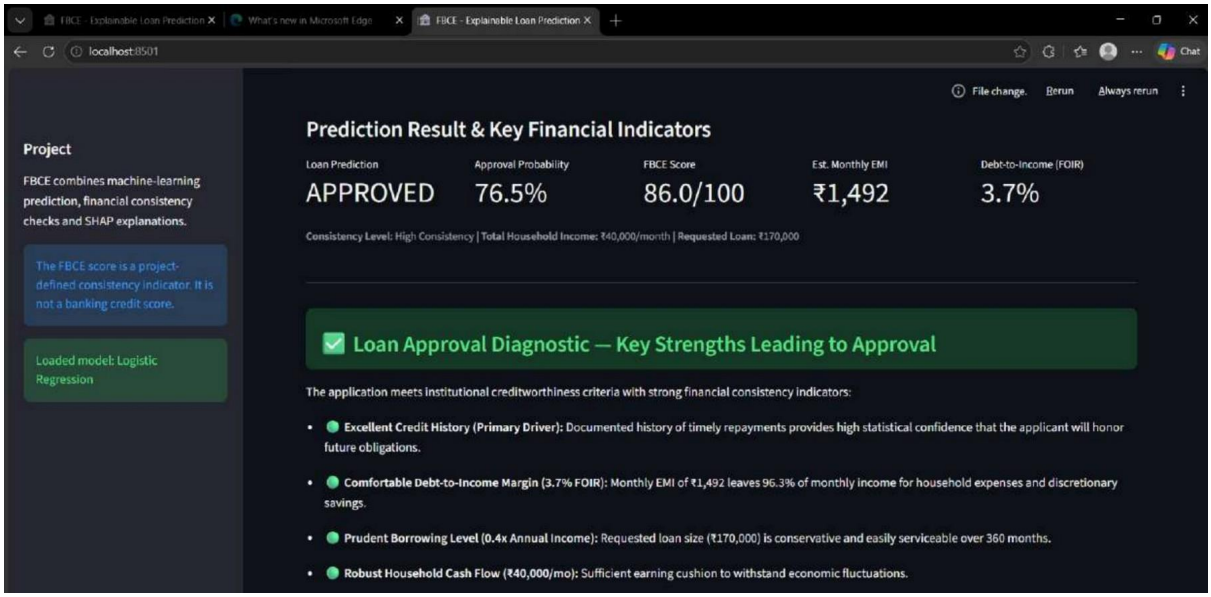


Figure 17. Prediction result and key financial indicators (earlier build)

The result row shows APPROVED, an approval probability of 76.5% and an FBCE score of 86.0/100, followed by an estimated monthly EMI and a FOIR figure and the start of a loan approval diagnostic. **Difference:** the probability agrees with the synthetic-data model; the FBCE score does not (the delivered `fbce.py` gives 90.98 for this input, test T2); the EMI, FOIR and diagnostic panels do not exist in the delivered `app.py`.

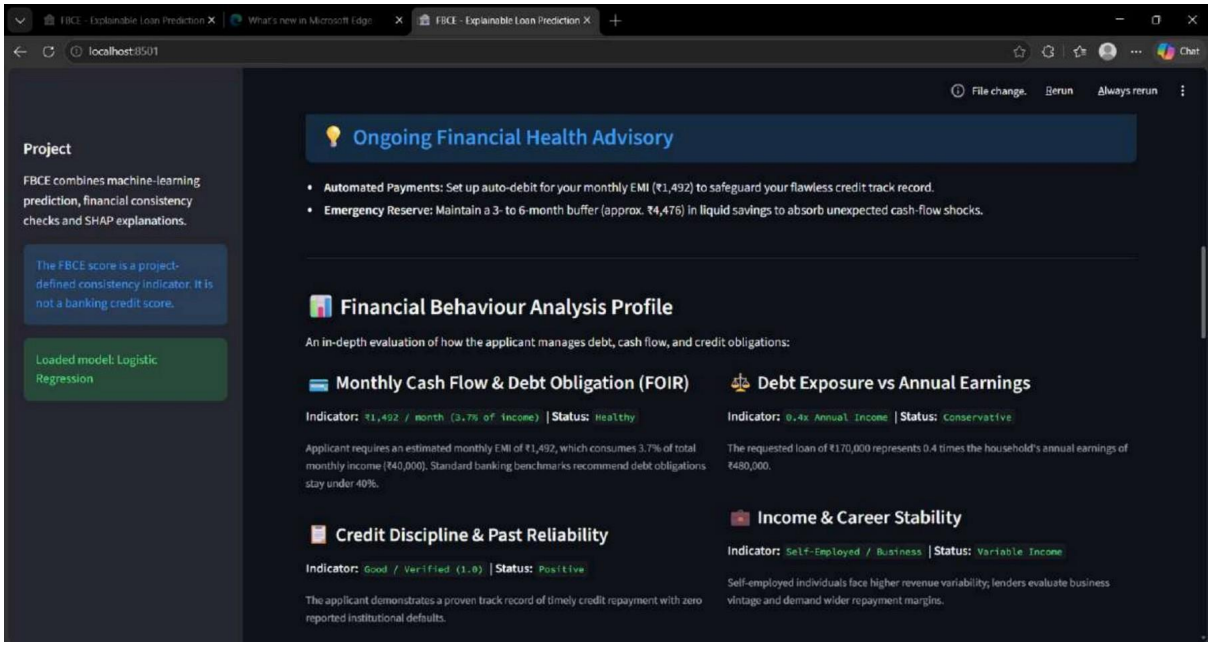


Figure 18. Financial health advisory and Financial Behaviour Analysis Profile (earlier build)

This page shows advisory text and a profile of cash flow, debt exposure, credit discipline and income stability. **Difference:** none of these panels exists in the delivered `app.py`, and their statements are not produced by `fbce.py`. They must not be attributed to the delivered system.

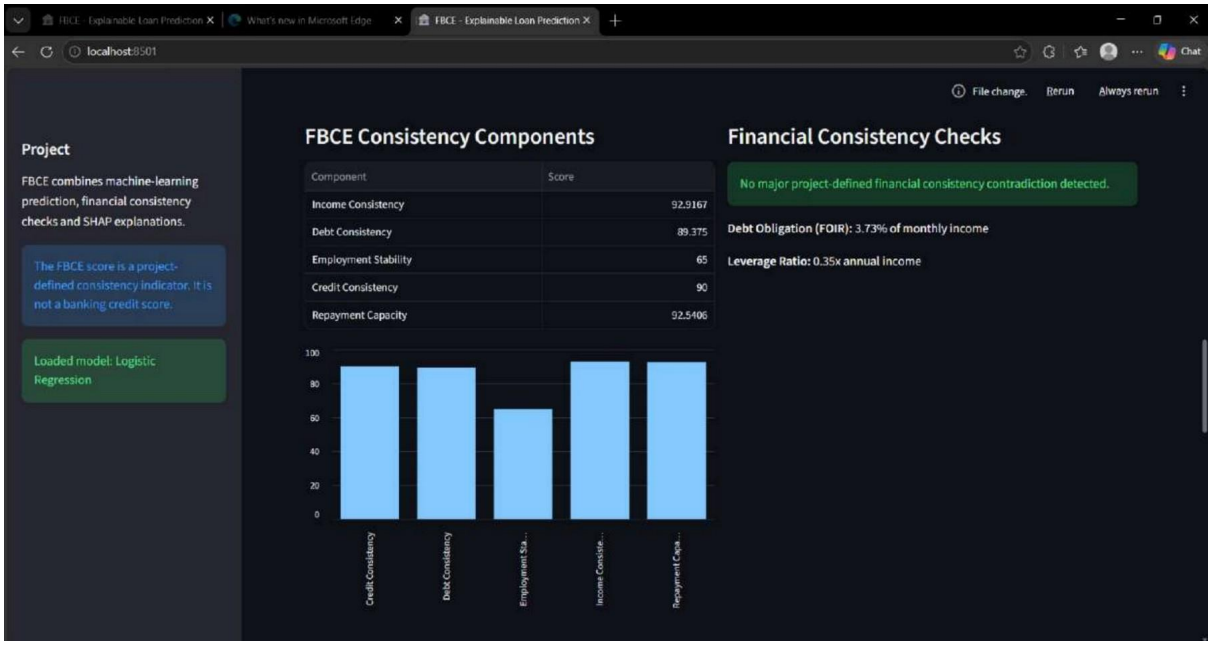


Figure 19. FBCE consistency components and financial consistency checks (earlier build)

The left panel lists five component scores (income 92.9167, debt 89.375, employment 65, credit 90, repayment 92.5406) with a bar chart; the right panel states that no contradiction was detected and shows a debt obligation and a leverage ratio. **Difference:** the delivered `fbce.py` returns 99.96, 99.95, 65.00, 90.00 and 100.00 for this input, and the delivered panel shows warnings and the total income, not FOIR or leverage. This is the clearest evidence that the screenshot build used a different scoring module.



Figure 20. SHAP explainability table (feature impact; earlier build)

The table lists the ten largest contributions by absolute value, such as num__ApplicantIncome +1.1779, num__Credit_History +0.873, cat__Education_Graduate +0.3933 and num__LoanAmount −0.3066. **Correspondence:** the structure matches the delivered code, and the values are reproduced by the synthetic-data model (Table 31).

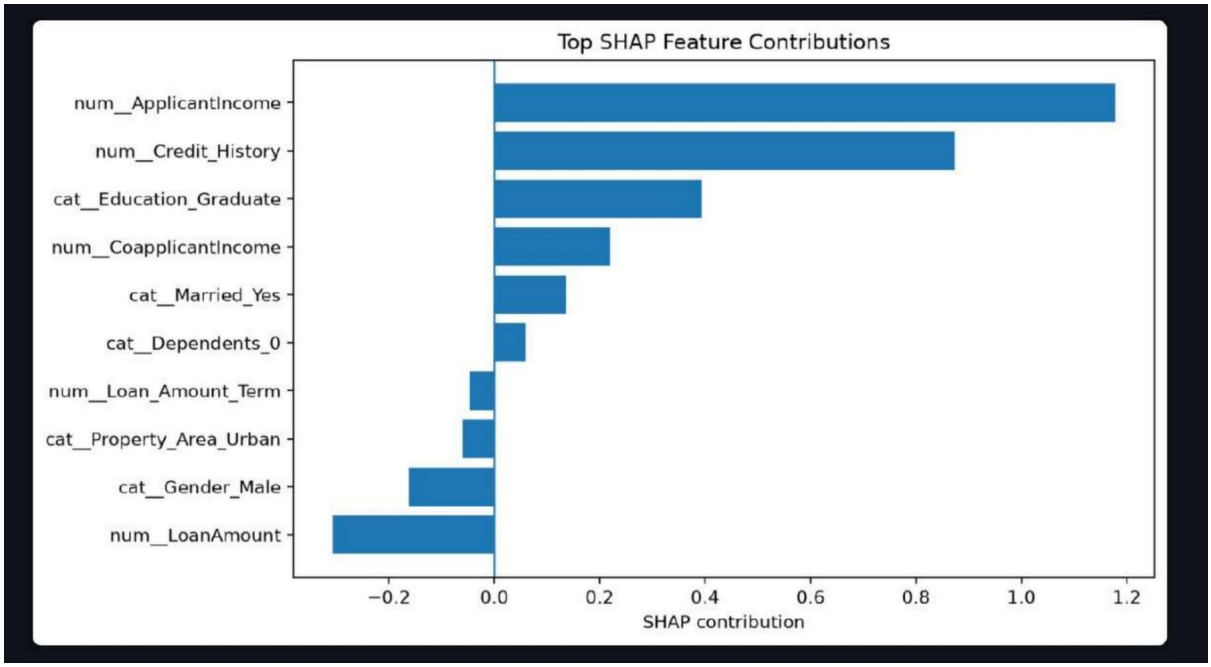


Figure 21. Top SHAP feature contributions for the demonstration input (earlier build)

The horizontal bar chart plots the same ten contributions, with positive bars to the right of zero. **Correspondence:** the delivered code draws the same kind of chart; the chart title and axis label match the code ("Top SHAP Feature Contributions", "SHAP contribution").



Figure 22. Positive and negative contributors with the academic-demonstration disclaimer (earlier build)

Two lists separate the positive and negative contributions, followed by the sentence that the prototype is for academic demonstration only. **Difference:** the list headings in the screenshot carry the additional words "(Helped Approval)" and "(Risk Factors / Caused Rejection)", which the delivered code does not print; the disclaimer wording also differs slightly from the delivered caption.

5.5.2 Installation and execution

Table 49 lists what is needed. Steps that appear in the README are marked "README"; steps added by this guide are marked "suggested". The commands were not all executed for this report (Section 5.4).

Table 49. Prerequisites

Requirement	Detail	Source
Python	Version 3.10 or 3.11	README
Libraries	pandas, numpy, scikit-learn, xgboost, lightgbm, shap, streamlit, matplotlib, joblib	requirements.txt
Dataset	A loan-approval CSV in Kaggle format saved as data/loan_data.csv, or the synthetic file	README, PUT_DATASET_HERE.txt
Command prompt	Opened inside the project folder	README
Virtual environment	Optional isolated environment (python -m venv venv)	Suggested; not in README

The installation and execution sequence is the one described in Section 4.4.5. In addition, the training script expects a target column named Loan_Status (Y/N) or one of loan_status, LoanStatus or target; the dashboard opened at localhost:8501 in the screenshots; and without a dataset `python generate_demo_data.py` is run first (the `data` folder must exist), in which case all results are treated as pipeline tests only. An optional virtual environment (`python -m venv venv`) is suggested by this guide and is not part of the README.

5.5.3 Using the dashboard

1. Check that the sidebar shows “Loaded model” followed by a model name. If a warning says that the model is not trained, run the training command first.
2. Enter applicant income, co-applicant income, loan amount and loan term in months. Use the same unit for the loan amount as the training dataset (thousands for the Kaggle dataset).
3. Choose credit history, gender, marital status, dependents, education, self-employment and property area.
4. Press “Predict Loan”.
5. Read the prediction and the approval probability. A probability close to 0.5 is a borderline output.
6. Read the FBCE score, the consistency level, the five component scores and any warnings. Read the warnings before the level (Table 30).
7. Read the SHAP table, chart and lists. Positive values raised the model output and negative values lowered it; only the ten largest are shown.
8. Treat the result as decision support for a human reviewer and not as a credit decision.

The three outputs answer different questions: the label and probability say what the model predicts, the FBCE panel says whether the entered values fit together, and the SHAP section says which features the model used. Table 43 suggests how to read them together.

5.5.4 Messages produced by the training script

Table 50. Messages produced by train_model.py and what to do

Situation	Message (from the code)	Action
data/loan_data.csv is missing	"Dataset not found: … Download a public loan approval dataset and save it as data/loan_data.csv"	Place the CSV or generate the synthetic file
No target column found	"Target column not found. Expected one of: Loan_Status, loan_status, LoanStatus, target"	Rename the target column
Successful run	"MODEL COMPARISON", the table, "Best model: …", "Saved model: …"	Start the dashboard

Table 51. Troubleshooting

Symptom	Likely cause	Remedy	Verified?
Dashboard says the model is not trained	models/fbce_model.pkl is missing	Run python train_model.py	From code; not run
ModuleNotFoundError for xgboost, lightgbm, shap or streamlit	Libraries not installed	Run pip install -r requirements.txt	Observed here for the four libraries
FileNotFoundError from generate_demo_data.py	data folder does not exist	Create the data folder first	From code
"Prediction failed" with an error text	Input columns differ from the training data	Use Kaggle-format feature names	From code; not run
"SHAP explanation could not be generated"	Library or model incompatibility	Check library versions	From code; not run
Unexpected FBCE scores	Loan amount and income in different units	Enter both in comparable units	Executed (units what-if)

REFERENCES

[1] H. Putra and R. Rumini, "Comparative study of logistic regression, random forest, and XGBoost for bank loan approval classification," *J. Appl. Inform. Comput.*, vol. 9, no. 5, Oct. 2025, doi: 10.30871/jaic.v9i5.10862.

[2] M. Ghahremani, H. Otieno, M. Kazreen, and S. Shiaeles, "Machine learning-based loan approval automation: Enhancing efficiency, accuracy and fairness in credit decision-making," *Expert Syst.*, vol. 43, no. 7, Art. no. e70307, 2026.

[3] K. Gawande, Pallavi, D. Thakur, and P. Saurabh, "An explainable ensemble learning framework for transparent and accurate loan approval prediction," *Discover Artif. Intell.*, vol. 6, Art. no. 1070, 2026.

[4] N. Bussmann, P. Giudici, D. Marinelli, and J. Papenbrock, "Explainable machine learning in credit risk management," *Comput. Econ.*, vol. 57, no. 1, pp. 203–216, 2021, doi: 10.1007/s10614-020-10042-0.

[5] P. E. de Lange, B. Melsom, C. B. Vennerød, and S. Westgaard, "Explainable AI for credit assessment in banks," *J. Risk Financial Manag.*, vol. 15, no. 12, Art. no. 556, 2022.

[6] V. H. Shah, S. V. Bhat, and V. D. Sharma, "An explainable and robust loan approval prediction system using hybrid XGBoost and heuristic risk assessment," *Int. J. Eng. Res. Technol.*, vol. 14, no. 12, Dec. 2025.

[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.

[8] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 1135–1144.

[9] D. O. Oyeyemi, O. O. Okosieme, O. Idowu-Kunlere, E. A. Julius, and I. P. Nwinyi, "Explainable AI for credit risk assessment: Integrating machine learning with business analytics," *IOSR J. Econ. Finance*, vol. 16, no. 5, pp. 63–72, 2025.

[10] "Loan prediction problem dataset," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset

[11] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nat. Mach. Intell.*, vol. 1, pp. 206–215, 2019.

[12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794.

[13] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 3146–3154.

[14] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[15] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[16] B. Baesens, T. Van Gestel, S. Viaene, M. Stepanova, J. Suykens, and J. Vanthienen, "Benchmarking state-of-the-art classification algorithms for credit scoring," *J. Oper. Res. Soc.*, vol. 54, no. 6, pp. 627–635, 2003.

[17] S. Lessmann, B. Baesens, H.-V. Seow, and L. C. Thomas, "Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research," *Eur. J. Oper. Res.*, vol. 247, no. 1, pp. 124–136, 2015, doi: 10.1016/j.ejor.2015.05.030.

[18] A. E. Khandani, A. J. Kim, and A. W. Lo, "Consumer credit-risk models via machine-learning algorithms," *J. Bank. Finance*, vol. 34, no. 11, pp. 2767–2787, 2010, doi: 10.1016/j.jbankfin.2010.06.001.

[19] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Comput. Surv.*, vol. 51, no. 5, Art. no. 93, 2019, doi: 10.1145/3236009.

[20] I. E. Kumar, S. Venkatasubramanian, C. Scheidegger, and S. A. Friedler, "Problems with Shapley-value-based explanations as feature importance measures," in *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, PMLR vol. 119, 2020, pp. 5491–5500.

[21] A. Fuster, P. Goldsmith-Pinkham, T. Ramadorai, and A. Walther, "Predictably unequal? The effects of machine learning on credit markets," *J. Finance*, vol. 77, no. 1, pp. 5–47, 2022, doi: 10.1111/jofi.13090.